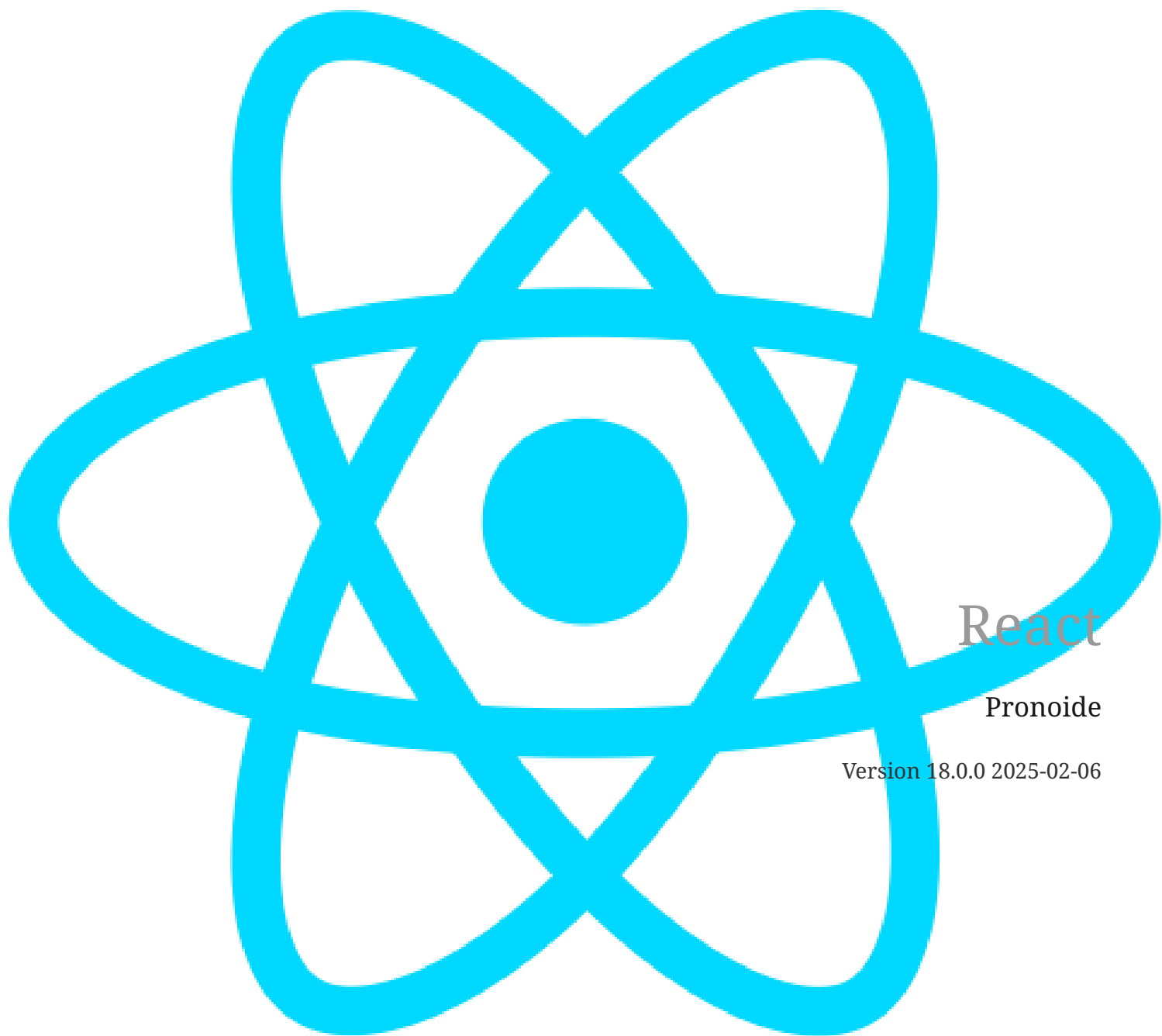




React

Pronoide

Version 18.0.0 2025-02-06

Contenidos

1. ¿Qué es React?	1
2. Requisitos	2
3. Composición	3
4. Modelo declarativo	7
5. Virtual DOM	9
6. React vs jQuery vs ...	10
7. Create React App	12
7.1. Lab: Create React App	13
7.2. Scripts del proyecto	15
7.3. Estructura del proyecto	16
8. Proyecto inicial desde 0	17
8.1. Lab: Proyecto inicial desde 0	18
9. ¿Qué es JSX?	30
10. Componente sin JSX	33
11. Componentes	35
11.1. Componentes de clase	36
11.2. Componentes funcionales	38
12. Expresiones	39
13. Estilos	40
14. Propiedades	41
14.1. Valores por defecto	42
14.2. PropTypes (validación de propiedades)	43
14.3. Validación de propiedades personalizadas	45
14.4. Lab: Propiedades	46
15. Renderizado condicional	50
16. Listas	52
16.1. Atributo key	53
16.2. Lab: Lista de noticias	55
17. Eventos	58
17.1. Tipos de eventos	59
17.2. Lab: Eventos	60
18. Estados	62
18.1. Lab: Estado	64
19. Flujo de datos	79
20. Ciclo de vida	80
20.1. constructor	81
20.2. getDerivedStateFromProps	82
20.3. componentDidMount	83

20.4. shouldComponentUpdate	84
20.5. getSnapshotBeforeUpdate	85
20.6. componentDidUpdate	86
20.7. componentWillUnmount	87
20.8. Lab: Ciclo de vida	88
21. Referencias	96
21.1. Lab: Referencias	97
22. Propiedad children	105
22.1. Lab: Propiedad children	106
23. Fragments	111
24. Context API	113
24.1. Lab: Context API	115
25. Higher Order Components (HOC)	119
25.1. Lab: Añadir estilos con un HOC	120
25.2. Lab: Cargar datos con un HOC	126
26. Suspense (lazy loading)	133
26.1. Lab: Suspense	134
27. Portals	138
27.1. Lab: Portals	139
28. React Hooks	142
28.1. Reglas de los React Hooks	142
28.2. useState	143
28.2.1. Lab: useState	144
28.3. useEffect	147
28.3.1. Lab: useEffect	149
28.4. useContext	153
28.4.1. Lab: useContext	154
28.5. useMemo	158
28.5.1. Lab: useMemo	159
28.6. useRef	164
28.6.1. Lab: useRef	165
28.7. useReducer	172
28.7.1. Lab: useReducer	173
28.8. Crear un hook	178
28.8.1. Lab: Crear un hook	179
29. Redux	183
29.1. Instalación	184
29.2. Estado	185
29.3. Actions	186
29.3.1. Action creators	186
29.4. Reducers	187

29.5. Store	188
29.5.1. Funciones del Store	189
29.6. Lab: Redux	190
30. React-Redux	197
30.1. Instalación	198
30.2. Provider	199
30.3. connect	200
30.4. Hooks de react-redux	202
30.5. Lab: react-redux	203
30.6. Middleware	208
30.6.1. redux-thunk	209
31. React Router	211
31.1. Instalación	212
31.2. BrowserRouter	213
31.3. Routes y Route	214
31.4. Link	215
31.5. Navegación por código (useNavigate)	216
31.6. Rutas con parámetros	217
31.7. Rutas anidadas	219
31.8. Redirigir rutas	221
31.9. Guards	222
31.10. Ruta comodín	223
31.11. Query Params	224
31.12. Lab: React Router	226

Chapter 1. ¿Qué es React?

React es una librería open source de JavaScript para crear interfaces de usuario que sigue el paradigma de la programación orientada a componentes.

Fue desarrollada por el equipo de Facebook y la construyeron para crear componentes visuales para las aplicaciones. El problema de Facebook es que su aplicación es demasiado grande y tenían muchos elementos que pintar en la pantalla, por lo que al navegador le llevaba un tiempo en renderizarlos todos ellos y sacaron React como solución para renderizar todos estos elementos pero de una forma mucho más eficiente.

Hay que decir que no es un **framework**, es decir, que no vamos a tener modelos, controladores, plantillas, rutas... Todos los frameworks intentan empaquetar un conjunto de funcionalidades imponiendo la forma en la que hay que desarrollar con ellos, te dan todo lo necesario para poder construir una aplicación completa. React solo se centra en la interfaz de usuario por lo que tenemos libertad de elegir con que otras tecnologías lo podemos usar.

Actualmente en 2020, React es la librería más utilizada para construir SPAs, siendo usada por aplicaciones como Facebook, Netflix, Instagram, Airbnb, Khan Academy...

Chapter 2. Requisitos

Para empezar a usar React, vamos a necesitar:

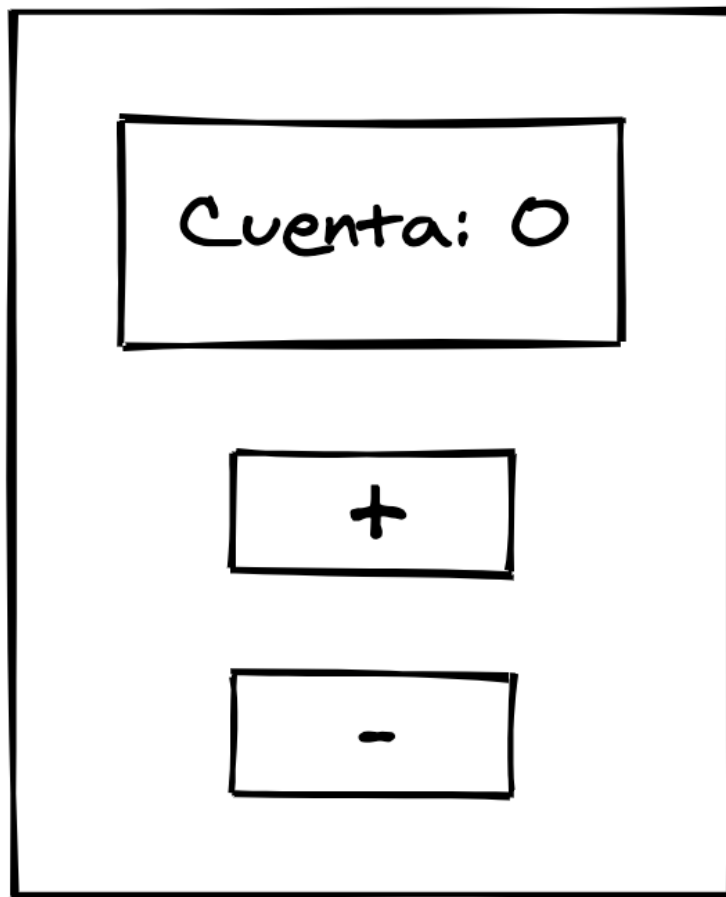
- Editor de código: VSCode (<https://code.visualstudio.com/>), Atom, Sublime Text...
- Node y NPM: <https://nodejs.org/en/>
- Un navegador en sus últimas versiones.

Chapter 3. Composición

En las aplicaciones hechas con React, todo son **componentes**. En la programación funcional se pasan funciones como parámetros para resolver problemas más complejos, y en React se sigue este mismo patrón para realizar las composiciones de los componentes, donde un componente puede estar compuesto por otros componentes.

Los componentes son piezas de código que encapsulan un comportamiento, una vista y su estado. Estas piezas tienen que ser **reutilizables** de tal forma que un componente se pueda utilizar en distintas partes de la aplicación.

Componentes



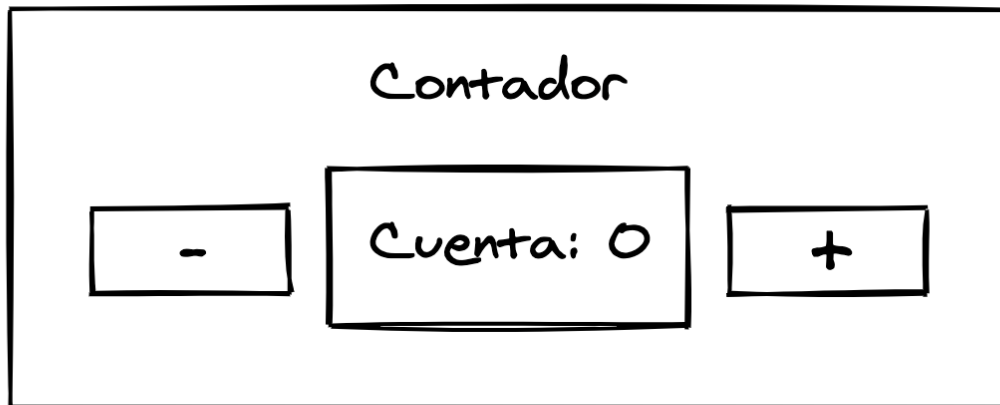
También tienen que ser autocontenidos, por lo que el comportamiento del componente queda dentro del componente haciendo que sean más fáciles de mantener.

También deberían de ser fácilmente **testeables** porque actúan como funciones puras, por lo que

para unos mismos parámetros que reciba el componente tiene que mostrar siempre la misma vista.

Al desarrollar con React, iremos creando componentes sencillos para resolver problemas pequeños, y estos componentes los iremos uniendo a otros para generar interfaces de usuario más complejas. Al final la aplicación será un conjunto de componentes que trabajarán entre ellos.

Nuevo componente





Avatar style title

Subtitle here



Title goes here

Subtitle here

Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the industry's standard dummy text ever since the 1500s, when an unknown printer took a galley of type and scrambled it to make a type specimen book.

ACTION 1

ACTION 2

Figure 1. Componente card

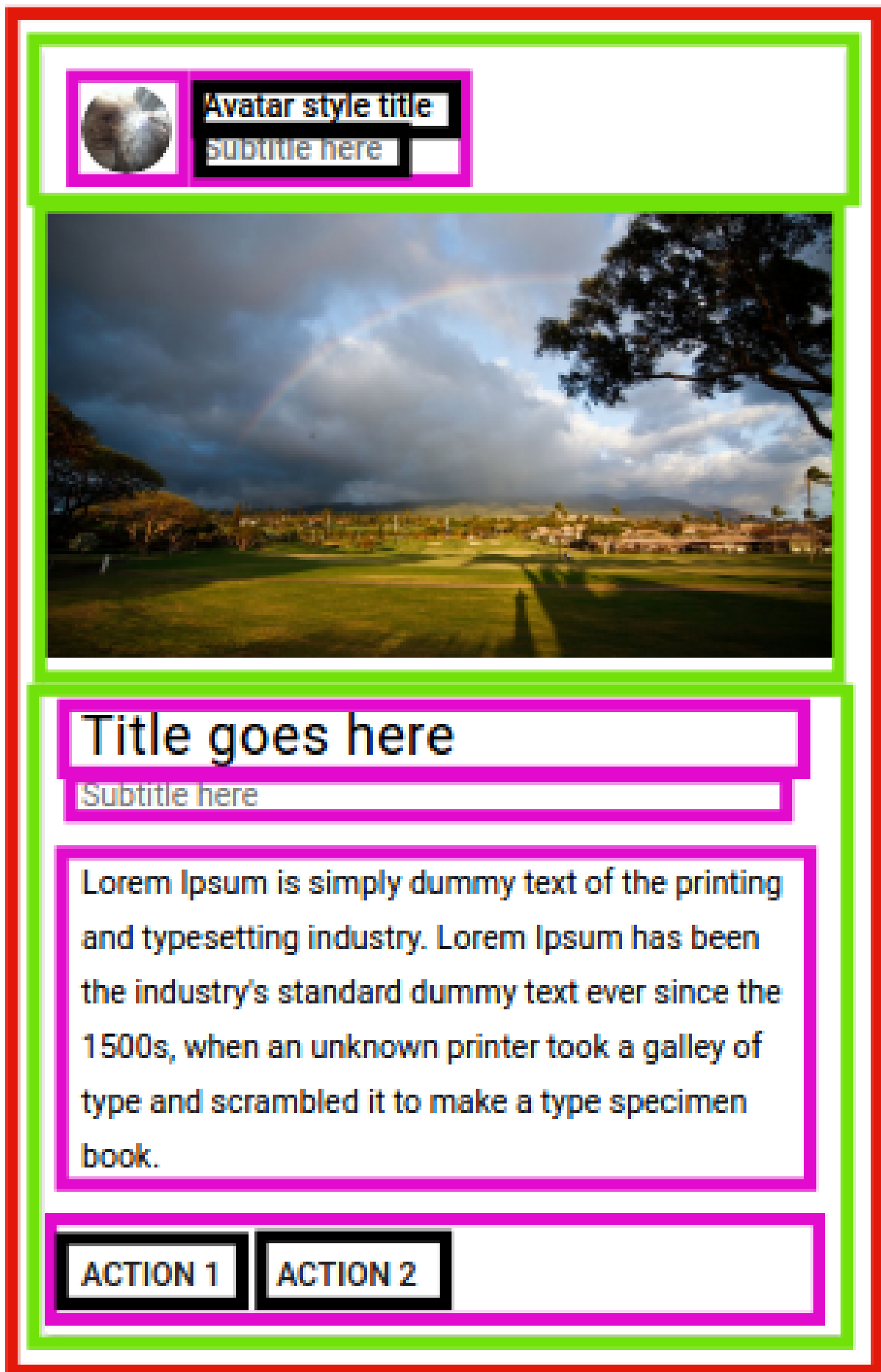


Figure 2. Composición del componente card

Chapter 4. Modelo declarativo

React sigue el paradigma de la programación declarativa, es decir, aquella donde las aplicaciones se estructuran de forma que se da importancia a describir que debería pasar en lugar de definir como tiene que pasar.

Normalmente estamos acostumbrados al estilo de la programación imperativa a la que solo le importa como conseguir el resultado esperado especificando una secuencia de instrucciones lo que requiere dedicar más tiempo a observar el código para saber que está haciendo. Mientras que con la programación declarativa se intenta que las instrucciones usadas describan lo que ese código está haciendo sin necesidad de definir un flujo de control.

Veamos un ejemplo de cada una de ellas:

Imperativa

```
var texto = "En un lugar de la Mancha, de cuyo nombre no quiero acordarme, no ha mucho tiempo que vivía un hidalgo...";
var textoNuevo = "";

for(var i=0; i < texto.length; i++) {
  if(texto[i] === " ") {
    textoNuevo += "-";
  } else {
    textoNuevo += texto[i];
  }
}

console.log(textoNuevo);
```

Declarativa

```
var texto = "En un lugar de la Mancha, de cuyo nombre no quiero acordarme, no ha mucho tiempo que vivía un hidalgo...";
var textoNuevo = texto.replace(/ /g, "-");

console.log(textoNuevo);
```

Pero ¿qué tiene esto que ver con React?, pues la forma en que se construye el DOM.

Imperativa

```
var app = document.getElementById('app');
var contenedor = document.createElement('div');
var titulo = document.createElement('h1');
contenedor.id = 'contenedor-saludo';
titulo.innerText = "Hola mundo";

contenedor.appendChild(titulo);
```

```
app.appendChild(contenedor);
```

Declarativa

```
import { createRoot } from 'react-dom/client';

const HolaMundo = () => (
  <div id="contenedor-saludo">
    <h1>Hola mundo</h1>
  </div>
)

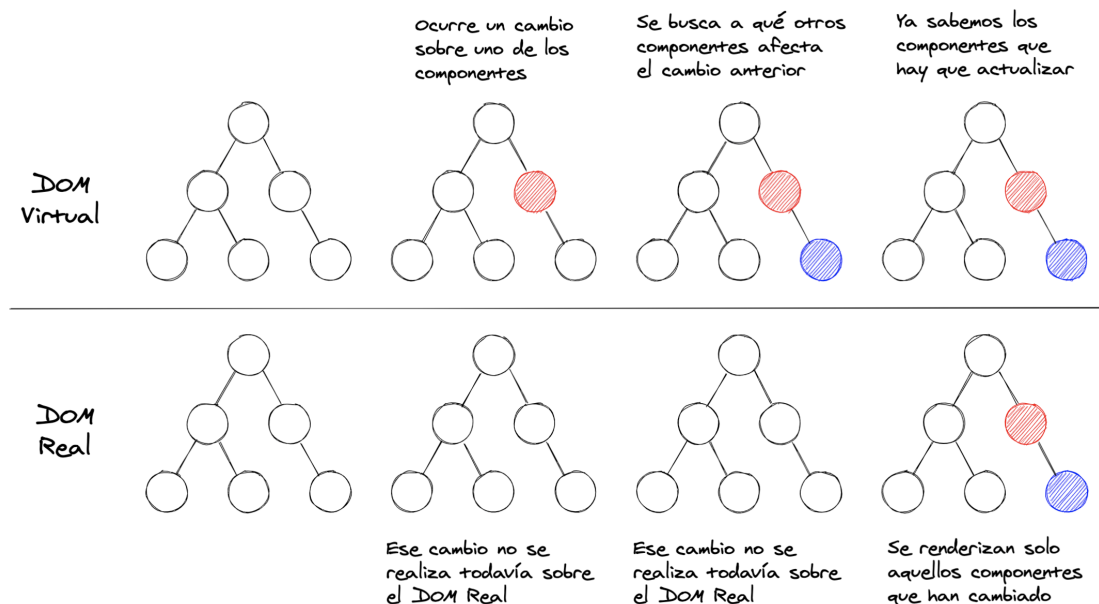
createRoot(document.getElementById('app')).render(<HolaMundo />);
```

React es declarativo, y los componentes describen los elementos del DOM que se deben de renderizar. Es la función **render** la que construye el DOM, y abstrae los detalles de como lo va a renderizar en la aplicación.

Chapter 5. Virtual DOM

Por razones de rendimiento, no es viable volver a renderizar toda la interfaz cada vez que se realiza un cambio en los datos, por lo que React usa, en memoria, una implementación propia del DOM, el **Virtual DOM**. Manipular la representación en memoria del DOM es mucho mas rápido y eficiente que hacerlo en el DOM real. Nosotros no nos tenemos que preocupar por interactuar con la API del DOM directamente, nosotros tendremos que interactuar con el Virtual DOM (con los elementos de React, que conceptualmente parecen HTML pero en realidad son objetos JavaScript) y React se encargará de realizar esos cambios en el DOM real por nosotros, de la forma más eficiente posible.

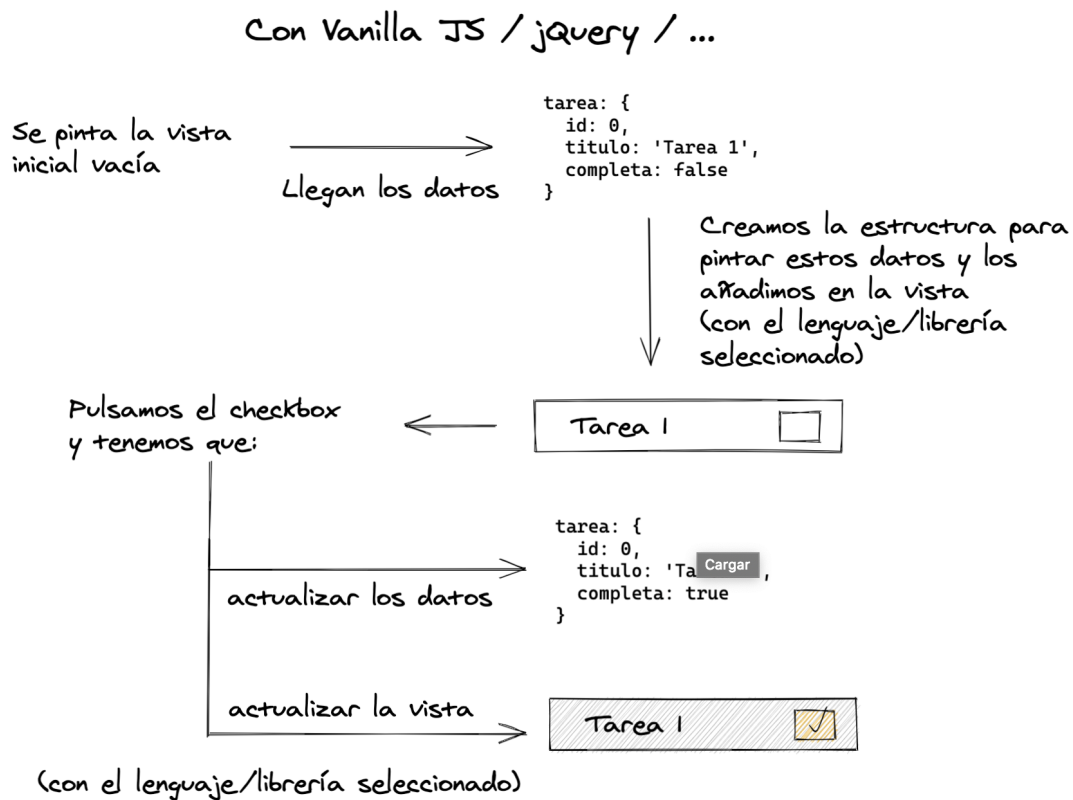
El proceso es el siguiente, cuando se produce algún cambio en el estado de la aplicación, este cambio se aplica a la representación del DOM en memoria. Después usa un algoritmo de *diff* para calcular las diferencias entre el DOM real y el que está en memoria, de donde obtiene los cambios mínimos a realizar para obtener la estructura del DOM deseada. Y de esta forma solo aplica esos cambios al DOM real sin necesidad de renderizar más elementos de los necesarios.



Chapter 6. React vs jQuery vs ...

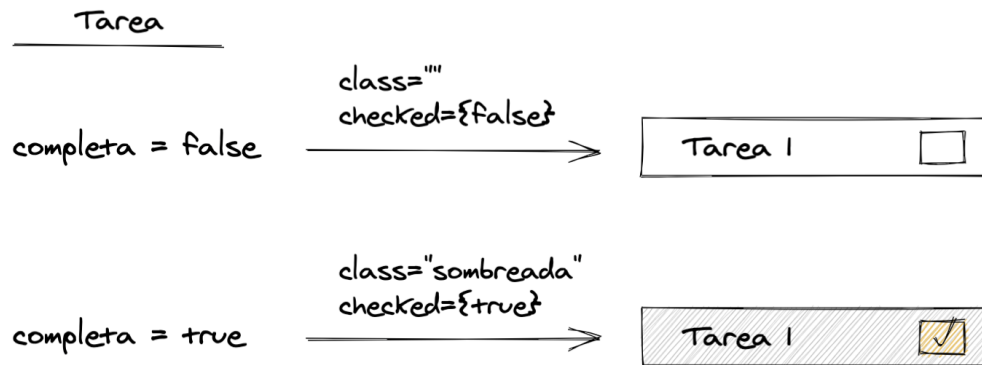
Entre utilizar React frente a utilizar otras librerías como por ejemplo jQuery, hay una gran diferencia.

En jQuery u otras somos nosotros quienes tenemos que pintar las etiquetas HTML, añadir los listeners sobre ellas para poder detectar las acciones que realiza el usuario y así poder ejecutar el código que va a realizar la modificación de los datos y las modificaciones correspondientes en la vista.



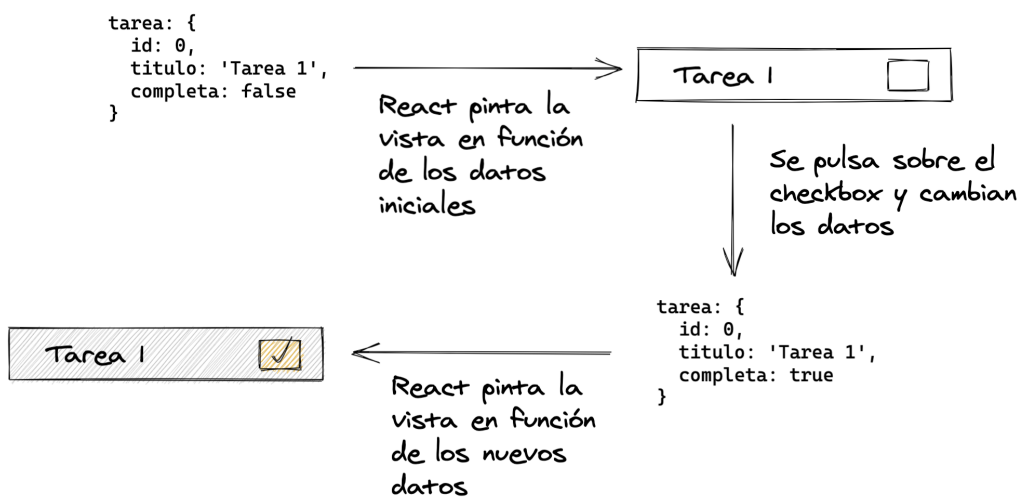
Mientras que en React, solo tenemos que indicar como tiene que pintar los datos que le llegan, y nosotros solo tendríamos que preocuparnos por indicarle como cambian los datos.

Con React



En el momento en que estos datos cambian, React los renderizará automáticamente sin que nosotros tengamos que indicarle como los tiene que volver a pintar.

Con React



Chapter 7. Create React App

Crear un proyecto de React desde cero es una tarea un poco complicada para alguien que está empezando a aprender React porque hay que instalar las dependencias necesarias, configurar algunas de estas dependencias... y por esto mismo el equipo de **Facebook** creó un proyecto llamado **create-react-app**.

Con esta herramienta podremos crear un proyecto de React en muy poco tiempo, sin necesidad de tener que preocuparnos por la configuración, ya que nos da una aplicación muy básica ya funcionando y con todo lo necesario configurado.

7.1. Lab: Create React App

En este laboratorio vamos a ver como crear un proyecto de React con la herramienta **create-react-app** sin la necesidad de tener que configurar nada.

Tenemos dos formas de crear un proyecto con esta herramienta:

- Forma 1: creamos el proyecto directamente usando **npx**:

```
$ npx create-react-app reactjs-create-react-app-lab
```

- Forma 2: instalamos la herramienta y creamos el proyecto:

```
$ npm install -g create-react-app  
$ create-react-app reactjs-create-react-app-lab
```

Una vez lanzados los comandos anteriores, se genera el proyecto de React con el nombre que le hemos dado (**reactjs-create-react-app-lab**), y en la consola se nos muestran los distintos comandos que podemos lanzar:

```
Success! Created reactjs-create-react-app-lab at /reactjs-create-react-app-lab  
Inside that directory, you can run several commands:
```

```
yarn start  
  Starts the development server.
```

```
yarn build  
  Bundles the app into static files for production.
```

```
yarn test  
  Starts the test runner.
```

```
yarn eject  
  Removes this tool and copies build dependencies, configuration files  
  and scripts into the app directory. If you do this, you can't go back!
```

```
We suggest that you begin by typing:
```

```
cd reactjs-create-react-app-lab  
yarn start
```

```
Happy hacking!
```

Ahora ya podemos levantar la aplicación lanzando uno de los siguientes comandos dentro de la carpeta del proyecto:

```
$ yarn start  
$ npm start
```

Ahora ya deberíamos de poder ver la aplicación en <http://localhost:3000/>.

7.2. Scripts del proyecto

Dentro del proyecto de create-react-app podemos lanzar una serie de comandos que vamos a ver a continuación. Estos comandos los podemos encontrar en el archivo **package.json**.

```
"scripts": {  
  "start": "react-scripts start",  
  "build": "react-scripts build",  
  "test": "react-scripts test",  
  "eject": "react-scripts eject"  
}
```

- start

Podemos lanzar el siguiente comando para levantar la aplicación en el servidor de desarrollo:

```
$ npm start
```

- build

Podemos lanzar el siguiente comando para generar el código que pondremos en producción:

```
$ npm run build
```

- test

Podemos lanzar el siguiente comando para ejecutar todos los archivos de testing con Jest:

```
$ npm test
```

- eject

Podemos lanzar el siguiente comando para sacar los archivos de configuración fuera de los módulos donde se encontraban:

```
$ npm run eject
```

7.3. Estructura del proyecto

El proyecto que nos genera la herramienta CRA tiene la siguiente estructura:

- `node_modules`: carpeta donde se encuentran todas las dependencias del proyecto.
- `public`: carpeta de distribución donde encontraremos el código final de la aplicación para llevarlo a producción.
 - `index.html`: única página HTML del proyecto, en la que se inyectará nuestra aplicación.
 - `manifest.json`: archivo que habilita el uso de nuestra aplicación como PWA.
 - `robots.txt`: archivo para indicar si la web tiene que indexarse o no.
- `src`: carpeta en la que trabajaremos nosotros para ir generando la aplicación.
 - `App.css`: archivo de estilos para el componente App.
 - `App.test.js`: archivo de testing del componente App.
 - `App.js`: componente App de la aplicación, en este caso es el componente raíz.
 - `index.css`: archivo de estilos globales.
 - `index.js`: archivo de entrada a la aplicación, encargado de indicar donde se tiene que inyectar el componente raíz dentro de la página HTML.
 - `serviceWorker.js`: archivo en el que configurar algunas funcionalidades de las PWAs como el uso de notificaciones.
 - `setupTests.js`: archivo en el que podemos añadir nuestros propios métodos de **expect** para los tests.
- `.gitignore`: archivo que le indica a Git que archivos y carpetas tiene que ignorar.
- `package.json`: archivo con la información del proyecto (versión, dependencias, scripts...).

Chapter 8. Proyecto inicial desde 0

La herramienta create-react-app (CRA) es una buena forma de empezar a trabajar con React, pero añade muchas dependencias que no necesitaremos normalmente en los proyectos, también hace que sea más difícil de modificar la configuración de los proyectos...

Podemos configurar un proyecto desde 0 utilizando las dependencias de **Babel** o **Webpack** y configurándolas a nuestro gusto para que se adapte a las necesidades de nuestro proyecto y que añada solo aquello que vamos a necesitar, permitiéndonos también modificar la configuración y entender cual es el proceso para crear aplicaciones con React.

8.1. Lab: Proyecto inicial desde 0

En este laboratorio vamos a crear un primer proyecto de React configurando desde 0 webpack y babel.

Empezamos creando una carpeta **reactjs-proyecto-inicial-lab**, entramos en ella y vamos a inicializar un proyecto de NPM.

```
$ mkdir reactjs-proyecto-inicial-lab
$ cd reactjs-proyecto-inicial-lab
$ npm init -y
```

Ahora vamos a instalar con NPM las dependencias que vamos a necesitar en nuestro proyecto. Para un proyecto de React, como mínimo necesitaremos tener las siguientes:

- React y ReactDOM
- Babel
- Webpack

De React y ReactDOM las vamos a instalar como dependencias de producción con el siguiente comando:

```
$ npm install --save react react-dom
```

Una vez instaladas estas, vamos a instalar las que necesitaremos de la parte de webpack, estas como dependencias de desarrollo:

```
$ npm install --save-dev webpack webpack-cli webpack-dev-server
```

Ahora es el turno de las dependencias de Babel, también como dependencias de desarrollo:

```
$ npm install --save-dev @babel/core @babel/preset-env @babel/preset-react @babel/plugin-transform-runtime
```

Y por último, vamos a instalar los loaders y plugins que necesitaremos utilizar con Webpack/Babel también como dependencias de desarrollo:

```
$ npm install --save-dev babel-loader css-loader style-loader html-webpack-plugin
```

Una vez que tenemos instaladas todas las dependencias que necesitaremos durante el curso, vamos a empezar a configurar webpack y a explicar para que sirven estas dependencias.

Lo primero que haremos será crear un archivo **webpack.config.js** en la raíz de la carpeta en el que vamos a empezar importando el módulo **path** de node y exportando un módulo en el que añadiremos la configuración de webpack.

/reactjs-proyecto-inicial-lab/webpack.config.js

```
const path = require('path');

module.exports = {

}
```

Webpack necesita que le indiquemos cual va a ser el punto de entrada a nuestra aplicación y cual será la salida. Con punto de entrada me refiero al archivo por el que webpack tiene que empezar a procesar el código de la aplicación, mientras que con punto de salida estaríamos indicando en que lugar va a dejar webpack los archivos finales que genera a partir del punto de entrada.

Utilizaremos el paquete path para generar la ruta a los directorios de estos dos puntos, el de entrada y el de salida. Y luego dentro de la configuración, indicaremos que archivo es el principal y que archivo tiene que generar.

/reactjs-proyecto-inicial-lab/webpack.config.js

```
const path = require('path');

const entryPath = path.join(__dirname, 'src'),
      outputPath = path.join(__dirname, 'dist');

module.exports = {
  entry: path.join(entryPath, 'index.js'),
  output: {
    path: outputPath,
    filename: 'bundle.js'
  },
}
```

Ahora vamos a añadir el modo por defecto para el que queremos que webpack procese los archivos de la aplicación, podemos utilizar **development** o **production**. Dejaremos como valor por defecto, **development**, y luego añadiremos un script de NPM para poder generar también la versión de producción.

/reactjs-proyecto-inicial-lab/webpack.config.js

```
const path = require('path');

const entryPath = path.join(__dirname, 'src'),
      outputPath = path.join(__dirname, 'dist');

module.exports = {
  mode: 'development',
  entry: path.join(entryPath, 'index.js'),
  output: {
    path: outputPath,
    filename: 'bundle.js'
  },
}
```

```
  },  
}
```

Ahora toca añadir la configuración de los **loaders** que queremos aplicar sobre el código que va a procesar webpack. Estos loaders se configuran en **module.rules** que es un array en el que pasaremos como valores objetos donde definiremos que loaders queremos utilizar y sobre que archivos queremos utilizarlos.

Empezaremos indicándole que tiene que aplicar el **babel-loader** sobre los archivos que tengan la extensión **js** o **jsx**. Este loader se encargará de transformar el código de JSX a JavaScript, y de pasar el código de JavaScript moderno a código más antiguo para que los navegadores puedan ejecutarlo sin problemas.

Cada una de las reglas en las que pondremos los loaders tienen una serie de propiedades que pondremos. Estas son:

- **test**: expresión regular para indicar a que archivos aplicar la regla.
- **exclude**: expresión regular para indicar que carpetas excluir para no aplicar las modificaciones que se indican con esta regla.
- **use**: el loader o los loaders que queremos utilizar.

/reactjs-proyecto-inicial-lab/webpack.config.js

```
const path = require('path');  
  
const entryPath = path.join(__dirname, 'src'),  
      outputPath = path.join(__dirname, 'dist');  
  
module.exports = {  
  mode: 'development',  
  entry: path.join(entryPath, 'index.js'),  
  output: {  
    path: outputPath,  
    filename: 'bundle.js'  
  },  
  module: {  
    rules: [  
      {  
        test: /\.?(js|jsx)$/,  
        exclude: /node_modules/,  
        use: 'babel-loader'  
      },  
    ],  
  },  
}
```

Una vez que tenemos la regla configurada, cuando ejecutemos webpack y se encuentre con que tiene que aplicar el **babel-loader**, webpack buscará la configuración **babel** para saber que transformaciones tiene que realizar sobre el código de JavaScript.

Vamos a crear un archivo **.babelrc** con la configuración de babel. Dentro de este archivo vamos a indicarle que utilice los presets de **@babel/preset-env** y **@babel/preset-react**, además del plugin **@babel/plugin-transform-runtime**.

- **@babel/preset-env**: conjunto de plugins que se encargan de transformar el código moderno de JavaScript a otras versiones más antiguas del lenguaje para que se pueda ejecutar en los distintos navegadores.
- **@babel/preset-react**: conjunto de plugins que se encargan de transformar el JSX en JavaScript.
 - La opción **runtime: automatic** la añadimos para no tener que importar React en los distintos componentes de la aplicación.
- **@babel/plugin-transform-runtime**: nos permitirá utilizar la sintaxis `async/await` dentro del hook `useEffect`.

/reactjs-proyecto-inicial-lab/.babelrc

```
{
  "presets": [
    "@babel/preset-env",
    [
      "@babel/preset-react",
      {
        "runtime": "automatic"
      }
    ]
  ],
  "plugins": [
    "@babel/plugin-transform-runtime"
  ]
}
```

Con esto ya tendríamos configurada la parte de babel.

Ahora toca añadir una nueva regla que se va a encargar de coger los archivos de CSS que procese webpack para añadir este CSS en la cabecera del **index.html**.

Esta nueva regla apuntará a los archivos de CSS y utilizaremos dos loaders:

- **style-loader**: coge el código de CSS y lo añade dentro de etiquetas **style** en el **head** de la página de HTML.
- **css-loader**: permite importar CSS como si fueran módulos de JavaScript, de esta forma webpack podrá procesar este código.

/reactjs-proyecto-inicial-lab/webpack.config.js

```
const path = require('path');

const entryPath = path.join(__dirname, 'src'),
      outputPath = path.join(__dirname, 'dist');
```

```

module.exports = {
  mode: 'development',
  entry: path.join(entryPath, 'index.js'),
  output: {
    path: outputPath,
    filename: 'bundle.js'
  },
  module: {
    rules: [
      {
        test: /\.js$/,
        exclude: /node_modules/,
        use: 'babel-loader'
      },
      {
        test: /\.css$/,
        exclude: /node_modules/,
        use: ['style-loader', 'css-loader']
      }
    ]
  }
}

```

Es probable que utilizemos algún archivo de video en alguno de los laboratorios que haremos durante el curso, así que vamos a añadir una regla más con la que se copiarán estos archivos a la carpeta de distribución. Esta vez no utilizaremos un loader sino una de las nuevas funcionalidades añadidas en webpack 5, los **asset modules**.

/reactjs-proyecto-inicial-lab/webpack.config.js

```

const path = require('path');

const entryPath = path.join(__dirname, 'src'),
      outputPath = path.join(__dirname, 'dist');

module.exports = {
  mode: 'development',
  entry: path.join(entryPath, 'index.js'),
  output: {
    path: outputPath,
    filename: 'bundle.js'
  },
  module: {
    rules: [
      {
        test: /\.js$/,
        exclude: /node_modules/,
        use: 'babel-loader'
      },
      {

```

```

    test: /\.css$/,
    exclude: /node_modules/,
    use: ['style-loader', 'css-loader']
  },
  {
    test: /\. (mp3|mp4)$/,
    exclude: /node_modules/,
    type: 'asset/resource'
  }
]
}
}

```

Con esto ya hemos terminado con las reglas de webpack, ahora vamos a añadir la configuración del plugin **html-webpack-plugin**.

Este plugin se encarga de crear un **index.html** en la carpeta de distribución, a partir de una plantilla dada.

Tendremos que importarlo en nuestro archivo, y añadir una nueva sección **plugins** en la configuración. En la sección de plugins, vamos a crear una instancia del plugin a la que le pasaremos un objeto de configuración indicando donde se encuentra la plantilla del archivo y algunas opciones extra.

/reactjs-proyecto-inicial-lab/webpack.config.js

```

const HtmlWebpackPlugin = require('html-webpack-plugin');
const path = require('path');

const entryPath = path.join(__dirname, 'src'),
      outputPath = path.join(__dirname, 'dist');

module.exports = {
  mode: 'development',
  entry: path.join(entryPath, 'index.js'),
  output: {
    path: outputPath,
    filename: 'bundle.js'
  },
  module: {
    rules: [
      {
        test: /\. (js|jsx)$/,
        exclude: /node_modules/,
        use: 'babel-loader'
      },
      {
        test: /\.css$/,
        exclude: /node_modules/,
        use: ['style-loader', 'css-loader']
      }
    ]
  }
}

```

```

    },
    {
      test: /\. (mp3|mp4)$ /,
      exclude: /node_modules/,
      type: 'asset/resource'
    }
  ]
},
plugins: [
  new HtmlWebpackPlugin({
    title: 'Curso de React',
    template: path.join(entryPath, 'index.html')
  })
],
}

```

Ahora toca añadir la sección **devServer** en la que indicaremos con **static**, donde se encuentra el contenido que tendría que servirse, en este caso por nuestro servidor de desarrollo que es **webpack-dev-server**

/reactjs-proyecto-inicial-lab/webpack.config.js

```

const HtmlWebpackPlugin = require('html-webpack-plugin');
const path = require('path');

const entryPath = path.join(__dirname, 'src'),
      outputPath = path.join(__dirname, 'dist');

module.exports = {
  mode: 'development',
  entry: path.join(entryPath, 'index.js'),
  output: {
    path: outputPath,
    filename: 'bundle.js'
  },
  module: {
    rules: [
      {
        test: /\. (js|jsx)$ /,
        exclude: /node_modules/,
        use: 'babel-loader'
      },
      {
        test: /\.css$/,
        exclude: /node_modules/,
        use: ['style-loader', 'css-loader']
      },
      {
        test: /\. (mp3|mp4)$ /,
        exclude: /node_modules/,

```

```

        type: 'asset/resource'
      }
    ]
  },
  plugins: [
    new HtmlWebpackPlugin({
      title: 'Curso de React',
      template: path.join(entryPath, 'index.html')
    })
  ],
  devServer: {
    static: outputPath
  },
}

```

Y para terminar con la configuración de webpack, vamos a añadir una sección más, **resolve.extensions** a la que le vamos a pasar un array de extensiones, indicando con ello que a la hora de importar cualquier archivo pueda resolverlo automáticamente con estas extensiones, sin que tengamos que ponerlas en la importación.

/reactjs-proyecto-inicial-lab/webpack.config.js

```

const HtmlWebpackPlugin = require('html-webpack-plugin');
const path = require('path');

const entryPath = path.join(__dirname, 'src'),
      outputPath = path.join(__dirname, 'dist');

module.exports = {
  mode: 'development',
  entry: path.join(entryPath, 'index.js'),
  output: {
    path: outputPath,
    filename: 'bundle.js'
  },
  module: {
    rules: [
      {
        test: /\.js$/,
        exclude: /node_modules/,
        use: 'babel-loader'
      },
      {
        test: /\.css$/,
        exclude: /node_modules/,
        use: ['style-loader', 'css-loader']
      },
      {
        test: /\.mp3|mp4$/,
        exclude: /node_modules/,

```

```

      type: 'asset/resource'
    }
  ],
},
plugins: [
  new HtmlWebpackPlugin({
    title: 'Curso de React',
    template: path.join(entryPath, 'index.html')
  })
],
devServer: {
  static: outputPath
},
resolve: {
  extensions: ['.js', '.jsx']
}
}

```

Ya hemos terminado con la configuración de webpack. Ahora tenemos que crear la estructura de carpetas y archivos en base a lo que hemos configurado aquí.

Como hemos indicado que el HTML se va a generar a partir de una plantilla **index.html**, vamos a añadirla dentro de una carpeta **src**.

Importante poner un div con un identificador (normalmente se suele poner **app** o **root**) para indicar donde se tiene que montar el componente raíz de la aplicación.

/reactjs-proyecto-inicial-lab/src/index.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title><%= HtmlWebpackPlugin.options.title %></title>
</head>
<body>
  <div id="root"></div>
</body>
</html>

```

En la configuración de webpack una de las primeras opciones que hemos añadido ha sido la del punto de entrada a la aplicación, y todavía no tenemos este archivo, así que vamos a crearlo. Crearemos dentro de **src** un archivo **index.js** en el que vamos a importar **createRoot** y lo utilizaremos para renderizar un componente **App**, que vamos a crear después, dentro del div que hay en el **index.html** con el identificador **root**.

/reactjs-proyecto-inicial-lab/src/index.js

```
import { createRoot } from 'react-dom/client';
import App from './components/App';

createRoot(document.getElementById('root')).render(<App />);
```

Ahora vamos a crear nuestro primer componente, un archivo **App.jsx** dentro de la carpeta **src/components**. En un laboratorio posterior veremos que hace el siguiente código.

/reactjs-proyecto-inicial-lab/src/components/App.jsx

```
const App = () => {
  return (
    <div>
      <h1>Hola mundo!!!</h1>
    </div>
  )
}

export default App
```

Y para terminar, vamos a dejar un archivo **style.css** dentro de la carpeta **src** que vamos a importar en el punto de entrada. Este archivo lo dejaremos creado para ver el tema de estilos con CSS en un laboratorio más adelante.

/reactjs-proyecto-inicial-lab/src/index.js

```
import { createRoot } from 'react-dom/client';
import App from './components/App';
import './style.css';

createRoot(document.getElementById('root')).render(<App />);
```

Ya tenemos todos los archivos del proyecto, pero nos falta algo importante, probar que todo lo que hemos hecho hasta ahora funcione. Para ello tenemos que levantar la aplicación con el servidor de desarrollo, o generar la carpeta de distribución con el código de la aplicación.

Estas tareas las haremos mediante scripts de NPM, por lo que vamos a añadir 3 scripts dentro del archivo **package.json**:

- **webpack serve** se encarga de levantar la aplicación en el puerto 8080 (por defecto) utilizando **webpack-dev-server**.
- **webpack** se encarga de generar el código final en una carpeta **dist** con la configuración del modo **development**.
- **webpack --mode=production** hace lo mismo que el script anterior, pero con la configuración del modo **production**.

```
{
  "name": "reactjs-proyecto-inicial-lab",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start": "webpack serve",
    "build": "webpack --mode=development",
    "build:prod": "webpack --mode=production"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "devDependencies": {
    "@babel/core": "^7.17.12",
    "@babel/preset-env": "^7.17.12",
    "@babel/preset-react": "^7.17.12",
    "babel-loader": "^8.2.5",
    "css-loader": "^6.7.1",
    "html-webpack-plugin": "^5.5.0",
    "style-loader": "^3.3.1",
    "webpack": "^5.72.1",
    "webpack-cli": "^4.9.2",
    "webpack-dev-server": "^4.9.0"
  },
  "dependencies": {
    "react": "^18.1.0",
    "react-dom": "^18.1.0"
  }
}
```

Una vez añadidos, podemos lanzar el siguiente comando y comprobar que si entramos en <http://localhost:8080/> vemos el mensaje que hemos puesto en el componente **App**:

```
$ npm start
```

Con esto ya tenemos un proyecto de react con webpack y babel configurado desde 0. A este proyecto se le pueden añadir muchas más opciones de webpack para que realice otro tipo de tareas sobre el código como:

- Extracción de archivos CSS
- Optimización de imágenes
- Separar configuración de webpack por entornos
- Activar el hot module replacement
- ...

Pero con lo que hemos configurado nos vale para continuar con el curso.

Chapter 9. ¿Qué es JSX?

El equipo de Facebook también liberaron **JSX** al mismo tiempo que React, que provee de una sintaxis para crear estructuras del DOM complejas con atributos. **JSX** es una extensión de JavaScript que nos permite crear los componentes React usando una sintaxis similar a HTML (se podría decir que es HTML incrustado en JS). La sintaxis de JSX nos permite visualizar de una forma mas rápida la estructura de nuestro componente. Para usar React no es necesario usar JSX pero lo hace más fácil. Todo este código lo tenemos que poner en el método **render** que es el método más importante de los componentes React. Lo que devuelve este método es lo que finalmente vamos a ver.

Componente Input

```
import React, { Component } from 'react';

class Input extends Component {
  const { type, maxLength, classes } = this.props;
  render() {
    return (
      <input type={ type } maxLength={ maxLength } className={ classes } />
    )
  }
}

export default Input;
```

En JSX se pueden usar etiquetas HTML o componentes propios creados con React. Por convención los componentes propios tienen que empezar por mayúscula mientras que las etiquetas HTML se escriben igual que en HTML. Las etiquetas tienen que estar balanceadas, que significa que si abrimos una etiqueta, luego la tenemos que cerrar, y en el caso de algunas etiquetas como **
** que no necesitan etiqueta de cierre se tienen que cerrar ellas mismas **
**.

Etiqueta HTML

```
<input type="text" />
```

Etiqueta JSX

```
<input type="text" />
```

Etiqueta de componente de React

```
<Input type="text" />
```

También se pueden poner atributos a los elementos HTML, y la norma es que si el nombre del atributo contiene más de una palabra, este se escribe en **camelCase**. Hay dos excepciones, los atributos **class** y **for** se escriben como **className** y **htmlFor** en JSX. Y si los atributos pertenecen a

un componente de React, estos los define el propio componente y son libres (les podemos poner el nombre que queramos). El valor de los atributos tiene que ir entre **comillas** `""` (en caso de que sea un string) o entre **llaves** `{}` (para cualquier otro valor).

Etiqueta HTML

```
<input type="text" maxLength="15" class="borde-rojo" />
```

Etiqueta JSX

```
<input type="text" maxLength={15} className="borde-rojo" />
```

Etiqueta de componente de React

```
<Input type="text" maxLength={15} />
```

Los componentes de React solo pueden renderizar un nodo raíz.

Nodo raíz JSX válido

```
render () {  
  return <h1>Hola mundo</h1>  
}
```

El siguiente caso no se puede dar:

Nodo raíz JSX inválido

```
render () {  
  return (  
    <h1>Hola mundo</h1>  
    <h1>Hola mundo de nuevo</h1>  
  )  
}
```

Se puede solucionar metiendo los dos elementos `h1` dentro de un elemento `div` y de esta forma ya se estaría devolviendo un solo nodo raíz que tiene elementos hijos.

Nodo raíz JSX válido

```
render () {  
  return (  
    <div>  
      <h1>Hola mundo</h1>  
      <h1>Hola mundo de nuevo</h1>  
    </div>  
  )  
}
```

```
}
```

Esto se debe a que cuando transforma el código JSX a JS, React solo puede crear un elemento para devolverlo.

Nodo raíz JS

```
render() {  
  return React.createElement('div', null,  
    React.createElement('h1', null, 'Hola mundo'),  
    React.createElement('h1', null, 'Hola mundo de nuevo')  
  )  
}
```

Chapter 10. Componente sin JSX

Hemos visto anteriormente un poco de JSX, la sintaxis que se utiliza para indicar que tienen que renderizar los componentes.

Al final el JSX se transforma en JavaScript, por lo que también podríamos escribir el código de nuestros componentes en JS completamente.

Entonces, ¿cómo se podría hacer esto?

Pues utilizando la función **createElement** que se importa de React.

```
import React from 'react';

const App = () => {
  return React.createElement(
    'div',
    {id: 'main'},
    React.createElement(
      'h1',
      {className: 'titulo'},
      'Hola mundo!!!'
    )
  )
}
```

En la versión 17 de React ya no es necesario importar **React**, algo obligatorio hasta ahora porque cuando se transpila el código de JSX a JS es necesario utilizar **React.createElement** y por tanto si no se importa no podemos llegar a llamar a la función de **createElement**.

Esto podemos verlo desde <https://babeljs.io/repl>.

Ahora utilizan otro módulo (**react/jsx-runtime**) para realizar esta transformación y Babel lo importará automáticamente a la hora de realizar la transpilación.

Pero si quisiéramos crear los componentes con JS en lugar de JSX, ahora el código quedaría como podemos ver a continuación:

```
import { jsx } from 'react/jsx-runtime';

const App = () => {
  return jsx(
    'div',
    {
      id: 'main',
      children: jsx(
        'h1',
        {
          className: 'titulo',

```

```
    children: 'Hola mundo!!!'  
  }  
)  
}  
)  
}
```

Como podemos ver con los ejemplos de código de arriba, crear aplicaciones web en React sin utilizar JSX se vuelve muy complejo.

Chapter 11. Componentes

Todas las interfaces de usuario están hechas de distintas piezas. Nosotros vamos a describir cada una de estas piezas como **componentes**. Los componentes nos permiten seguir el patrón de **divide y vencerás** con el que podremos romper la aplicación en distintos componentes más pequeños y reusables.

Los componentes de React se basan en los conceptos de entradas (**props**) y estados internos (**states**) que veremos un poco más adelante.

Los componentes pueden ser de dos tipos:

- De clase
- Funcionales

11.1. Componentes de clase

Los componentes de clase son clases de JavaScript que extienden de **React.Component**.

Dentro de la clase tenemos que añadir un método **render** que tiene que devolver la estructura del componente (el código JSX) para que React sepa que tiene que pintar.

```
import { Component } from 'react';

class MiComponente extends Component {
  render() {
    return (
      <h1>Hola mundo</h1>
    )
  }
}

export default MiComponente;
```

Al ser una clase, podemos añadir un constructor, el cual recibe unas propiedades desde el exterior. Más adelante veremos para que usar las propiedades. Y al estar extendiendo de **Component** tenemos que llamar al **super** pasándole estas propiedades para que las añada al objeto **this**.

```
import { Component } from 'react';

class MiComponente extends Component {
  constructor(props) {
    super(props)
  }

  render() {
    return (
      <h1>Hola mundo</h1>
    )
  }
}

export default MiComponente;
```

Dentro del constructor es donde inicializaremos el estado del componente, pero esto ya lo veremos más adelante.



Desde la aparición de los React Hooks, este tipo de componentes se están dejando de usar a favor de los componentes funcionales.

Antes no había otra opción ya que en los componentes de clase era el único sitio donde podíamos añadir un estado y los métodos del ciclo de vida de los

componentes.

11.2. Componentes funcionales

Los componentes funcionales, como su nombre indica, son funciones. Estas funciones reciben como parámetro las propiedades y tienen que devolver la estructura de JSX que le indica a React que tiene que pintar en el navegador cuando utilizamos este componente.

```
const HolaMundo = (props) => (  
  <h1>Hola mundo</h1>  
)  
  
export default HolaMundo;
```

Antiguamente estos componentes eran únicamente presentacionales, es decir, que solo se usaban para pintar datos en el navegador ya que no podían gestionar un estado ni se podía utilizar los métodos del ciclo de vida con ellos.

Esto ha cambiado con los React Hooks, ya que podemos tener todo lo que tenemos con los componentes de clase pero en una función.

Ahora estos componentes son los más utilizados ya que además de poder hacer lo mismo que con los de clase tenemos otras ventajas:

- Evitamos el uso del objeto **this** que puede dar problemas.
- Escribimos mucho menos código por lo tanto son más legibles.
- Más fáciles de entender para alguien que empieza a usar React.

Chapter 12. Expresiones

Dentro de React, usaremos `{}` para mostrar datos que tenemos en variables en nuestra aplicación, darle valores a los atributos de las etiquetas o propiedades de los componentes...

Dentro de las llaves, podemos añadir cualquier expresión de JavaScript, como una variable, una condición, llamadas a funciones...

```
<button type="button" disabled={btnDisabled}>Botón {btnDisabled ? 'deshabilitado' : 'habilitado'}</button>
```

Chapter 13. Estilos

React lo que intenta es que cada componente tenga todo lo necesario para pintarlo en el navegador, y esto incluye también los estilos. React nos permite escribir **estilos en línea** usando la sintaxis de JSX, que puede parecer un poco extraño pero tiene algunos beneficios sobre el CSS.

Por ejemplo, se evita los conflictos de especificidad, no tenemos que usar selectores, al usar JavaScript, podemos usar variables para hacerlos más dinámicos, también funciones e incluso estructuras de control de flujo para dar unos estilos u otros...

Los estilos van a ser objetos JavaScript, cuyos atributos son las propiedades de CSS y se escriben en *camelCase*.

```
const estilos = {  
  backgroundColor: '#242424',  
  width: '100%',  
  height: '50px',  
}  
  
<div style={estilos}>  
</div>
```

Por supuesto, estos estilos son locales al componente, y podemos aplicar estilos que se encuentran definidos en un archivo externo asignando las clases (className) correctas al componente.

En este caso tenemos que tener en cuenta que el archivo de CSS se tiene que importar en el **index.html**, o los tiene que gestionar Webpack con los loaders adecuados para ello:

- css-loader
- style-loader
- sass-loader
- postcss-loader
- ...

Chapter 14. Propiedades

Trabajar con componentes nos permite tener piezas de código que podemos reutilizar en nuestra aplicación. Pero necesitamos conocer otro concepto más para poder hacer que estos componentes se puedan reutilizar a un mayor nivel, pudiendo configurarlos para que reciban ciertos valores que se utilizarán internamente en ellos.

Estos valores que van a recibir desde el exterior son las **propiedades**, unos atributos que podemos poner en los componentes (al igual que los atributos de las etiquetas HTML) y que nos permiten pasar valores desde componentes que están por encima del componente que las va a recibir.

```
<MiComponente prop1="Un texto" prop2={3} prop3 />
```

Los valores de las **propiedades** no se pueden modificar. Entonces si necesitamos cambiar el valor de una propiedad que ha recibido nuestro componente, no podremos hacerlo. Lo que si podemos hacer es enviarle el nuevo valor a la propiedad para que el componente se actualice con dicho valor.

Los componentes de clase las reciben como parámetro del constructor y usaremos **this.props** para acceder a las propiedades.

```
class MiComponente extends Component {
  constructor(props) {
    super(props);
    // ...
  }

  render() {
    const { prop1, prop2, prop3 } = this.props;
    return (...)
  }
}
```

Mientras que en los componentes funcionales las recibiremos como parámetros de la función.

```
const MiComponente = (props) => {
  const { prop1, prop2, prop3 } = props;

  return (...)
}
```

Cada vez que se produce un cambio en una propiedad, se vuelven a renderizar los componentes a los que afecta dicho cambio.

14.1. Valores por defecto

React nos permite darles valores por defecto a las propiedades que reciben los componentes de tal forma que no sea necesario pasarselas siempre.

Para añadir los valores por defecto, hay que desestructurar las propiedades y asignarle estos valores.

```
const MiComponente = ({ miNum = 0, miTexto = 'Hola mundo!' }) => {  
  // Código aquí  
}
```

A la hora de pintar los componentes se utilizarán estas propiedades si no se las pasamos. En caso de pasarle estas propiedades, las nuevas sobrescribirán a las que habíamos definido como valores iniciales.



Antiguamente se podían añadir los valores por defecto usando **defaultProps**, pero esto se deprecó en React 17, y ya no funciona.

```
MiComponente.defaultProps = {  
  miNum: 0,  
  miTexto: 'Hola mundo!'  
}
```

14.2. PropTypes (validación de propiedades)

Las propiedades que reciben los componentes se pueden validar para asegurarnos de que nuestros componentes reciben los tipos de datos esperados.



Antiguamente se podían validar los tipos de las propiedades con la librería **prop-types**, pero esto se deprecó, y ya no funciona. **Ahora habría que utilizar TypeScript para las validaciones de tipos.**

Algunas de las validaciones que podemos utilizar son:

Validador	Descripción
PropTypes.isRequired	La propiedad tiene que ser <i>obligatoria</i>
PropTypes.string	La propiedad tiene que ser un <i>string</i>
PropTypes.number	La propiedad tiene que ser un <i>número</i>
PropTypes.bool	La propiedad tiene que ser un <i>booleano</i>
PropTypes.object	La propiedad tiene que ser un <i>objeto</i>
PropTypes.array	La propiedad tiene que ser un <i>array</i>
PropTypes.func	La propiedad tiene que ser una <i>función</i>
PropTypes.oneOfType([PropTypes.number, PropTypes.string])	La propiedad tiene que ser de uno de esos tipos
PropTypes.arrayOf(PropTypes.string)	La propiedad tiene que ser un <i>array de string</i>
PropTypes.objectOf(PropTypes.number)	La propiedad tiene que ser un <i>objeto cuyos atributos sean números</i>
PropTypes.shape({nombre: PropTypes.string, edad: PropTypes.number})	La propiedad tiene que ser un <i>objeto que tiene que tener un atributo nombre de tipo string y un atributo edad de tipo número</i>
PropTypes.oneOf(['Gato', 'Perro', 'Canario'])	La propiedad tiene que ser <i>uno de los valores del enumerado</i>



Cuando una validación no se cumple, se muestra un **warning** por la consola, y la aplicación seguirá funcionando aunque esto no quiere decir que funcione como se espera ya que puede ser que el tipo de dato no sea el esperado.

Para utilizar las validaciones necesitamos la siguiente dependencia:

```
$ npm install --save prop-types
```

Estas validaciones se asignan a la propiedad **propTypes** del componente:

```
import PropTypes from 'prop-types';
```

```
// ...
```

```
MiComponente.propTypes = {  
  miPropiedad: PropTypes.string.isRequired  
}
```

Las propTypes también nos sirven de documentación, ya que podremos ver en todo momento que propiedades puede recibir un componente, si son obligatorias u opcionales, el tipo...

14.3. Validación de propiedades personalizadas

A parte de las validaciones que vienen predefinidas, podemos crear nuestras propias validaciones asignando una función a la propiedad. Cuando no se cumple la validación la función tiene que devolver un **Error** con el mensaje a mostrar.

```
MiComponente.propTypes = {  
  numero: function(props, propName, componentName) {  
    if (props[propName] < 0) {  
      return new Error('El número tiene que ser mayor a 0');  
    }  
  }  
}
```

14.4. Lab: Propiedades

En este laboratorio vamos a ver como pasar a un mismo componente distintas propiedades para poder reutilizarlo y que nos muestre la misma estructura pero con distintos datos.

Vamos a copiar la plantilla del proyecto que se ha realizado en el laboratorio **reactjs-proyecto-inicial** y vamos a cambiarle el nombre del proyecto a **reactjs-propiedades-lab**.



Es posible que sea necesario volver a instalar las dependencias del proyecto para que funcione correctamente. Para ello solo tenemos que lanzar el comando **npm install** dentro de la carpeta del proyecto.

Ahora vamos a levantar el servidor de desarrollo con el comando:

```
$ npm start
```

Una vez que tenemos el proyecto levantado, vamos a crear un componente funcional **Sugus** dentro de la carpeta **components**.

Dentro de este componente vamos a poner la estructura inicial de este en el que nos encontraremos con un **div** que hará de envoltorio de los sugus, y un **párrafo** para poner el sabor de los sugus.

/reactjs-propiedades-lab/src/components/Sugus.jsx

```
const Sugus = () => {  
  // Hay que cambiarlo por el color de cada sugus  
  const color = 'white'  
  return (  
    <div>  
      <p>Aquí va el sabor</p>  
    </div>  
  )  
}  
  
export default Sugus
```

También le añadiremos unos estilos desde JavaScript, aplicándolos al atributo **style**.

/reactjs-propiedades-lab/src/components/Sugus.jsx

```
const styles = {  
  envoltorio: {  
    border: '1px solid black',  
    width: '100px',  
    height: '100px',  
    borderRadius: '5px',  
    color: 'white',  
    position: 'relative',  
    margin: '10px',  
    overflow: 'hidden',  
  },  
}
```

```

letras: {
  textAlign: 'center',
  transformOrigin: 'center center',
  transform: 'rotate(-45deg)',
  position: 'absolute',
  top: '25px',
  left: '30px',
  textShadow: '60px 0px 0px, -60px 0px 0px, -25px 30px 0px, 25px -30px 0px, 30px 30px 0px, -30px -30px 0px, 0px 60px
0px, 0px -60px 0px',
}
}

const Sugus = () => {
  // Hay que cambiarlo por el color de cada sugus
  const color = 'white'
  return (
    <div style={{...styles.envoltorio, backgroundColor: color}}>
      <p style={styles.letras}>Aquí va el sabor</p>
    </div>
  )
}

export default Sugus

```

Una vez que tenemos el componente sugus, vamos a ir al componente **App** para utilizar este.

Dentro de este otro componente, vamos a añadir la etiqueta de nuestro componente **Sugus** cinco veces, una por cada sugus que existe.

/reactjs-propiedades-lab/src/components/App.jsx

```

import Sugus from "./Sugus"

const App = () => {
  return (
    <div>
      <Sugus />
      <Sugus />
      <Sugus />
      <Sugus />
      <Sugus />
    </div>
  )
}

export default App

```

Y ahora que tenemos el componente varias veces, vamos a añadir sus propiedades para cada uno de ellos. Las propiedades se añaden como atributos de las etiquetas HTML, por lo que vamos a añadirle a cada etiqueta **Sugus**, un atributo **sabor** y otro **color**, con los valores que queremos que se pinten estos.

/reactjs-propiedades-lab/src/components/App.jsx

```

import Sugus from "./Sugus"

```

```
const App = () => {
  return (
    <div>
      <Sugus sabor="limón" color="#FDE23A" />
      <Sugus sabor="naranja" color="#F28E40" />
      <Sugus sabor="piña" color="#227BBE" />
      <Sugus sabor="cereza" color="#AD3B52" />
      <Sugus sabor="fresa" color="#EA464C" />
    </div>
  )
}

export default App
```

Una vez añadidas las propiedades, vamos a ver como las recibimos en el componente **Sugus**.

Como parámetro de la función del componente **Sugus** recibimos un objeto **props**, dentro del cual encontraremos dos claves con sus dos valores, el **color** y el **sabor**. Por tanto, allí donde debemos poner estos dos valores, vamos a asignar el valor de las propiedades.

/reactjs-propiedades-lab/src/components/Sugus.jsx

```
const styles = {
  envoltorio: {
    border: '1px solid black',
    width: '100px',
    height: '100px',
    borderRadius: '5px',
    color: 'white',
    position: 'relative',
    margin: '10px',
    overflow: 'hidden',
  },
  letras: {
    textAlign: 'center',
    transformOrigin: 'center center',
    transform: 'rotate(-45deg)',
    position: 'absolute',
    top: '25px',
    left: '30px',
    textShadow: '60px 0px 0px, -60px 0px 0px, -25px 30px 0px, 25px -30px 0px, 30px 30px 0px, -30px -30px 0px, 0px 60px 0px, 0px -60px 0px',
  }
}

const Sugus = (props) => {
  return (
    <div style={{...styles.envoltorio, backgroundColor: props.color}}>
      <p style={styles.letras}>{props.sabor}</p>
    </div>
  )
}

export default Sugus
```

Y con esto, hemos conseguido pintar los 5 tipos de sugus que existen utilizando un único

componente y pasandole los datos necesarios para pintarse como debe desde el exterior a través del uso de las **propiedades**.

Chapter 15. Renderizado condicional

En nuestras aplicaciones de React nos vamos a encontrar con algunos casos en los que tendremos que pintar un componente u otro dependiendo de alguna condición.

Por ejemplo, cuando estamos logueados veremos el botón de Logout, pero cuando no lo estemos tendríamos que ver el botón de Login. Entonces, tendremos que usar el renderizado condicional para indicar cual de los dos botones se pintan.

En React esto lo podemos conseguir hacer con instrucciones de JavaScript ya que no tenemos directivas al igual que ocurre en otros frameworks como Vue o Angular que se encarguen de ello.

Así que la idea es utilizar una instrucción **if-else** de JavaScript para devolver estos dos componentes o etiquetas.

```
if (logueado) {  
  return <button>Logout</button>  
} else {  
  return <button>Login</button>  
}
```

Lo de arriba serviría, pero es algo que no está bien visto dentro de React, por lo que tenemos que buscar alguna otra instrucción que permita hacer lo mismo. El operador ternario, **condicion ? loQueSeEjecutaSiEsTrue : loQueSeEjecutaSiEsFalse**.

```
return (  
  <div>  
    {logueado ? <button>Logout</button> : <button>Login</button>}  
  </div>  
)
```

El operador ternario sería la opción correcta para pintar un componente u otro en función de una condición.

Otras veces solo queremos pintar un elemento si cumplimos una condición, pero en el caso de que no se cumpla la condición no debería de pintarse nada.

```
return (  
  <div>  
    {condicion ? <p>El componente</p> : null}  
  </div>  
)
```

El código anterior sirve, porque se pinta un párrafo, o no se pinta nada, pero es más común utilizar el operador **and** para evitar tener que poner ese **null** del operador ternario.

```
return (  
  <div>  
    {condicion && <p>El componente</p>}  
  </div>  
)
```

Chapter 16. Listas

Cuando queremos iterar una lista de datos y generar un componente por cada item que se encuentra dentro de la lista, tenemos que utilizar el método **map** de los arrays.



Al método `map` se le pasa como parámetro una función de callback como `(item, index) => {...}`, donde el primer parámetro será cada elemento del array y el `index` la posición en la que se encuentra. Esta función de callback tiene que devolver por cada item un componente o etiqueta de HTML.

```
items.map(item => <li>{item}</li>)
```


16.1. Atributo key

A la hora de generar listas de componentes o etiquetas con React hay que tener en cuenta otro concepto más, el atributo **key**.

Si hemos generado una lista podremos ver en la consola un warning que dice **Warning: Each child in a list should have a unique "key" prop..**

Aquí nos indican que hay que ponerle a cada componente que generamos en la lista un atributo **key** y dicho valor tiene que ser único dentro de la lista.

```
items.map((item, pos) => <li key={pos}>{item.texto}</li>)
```



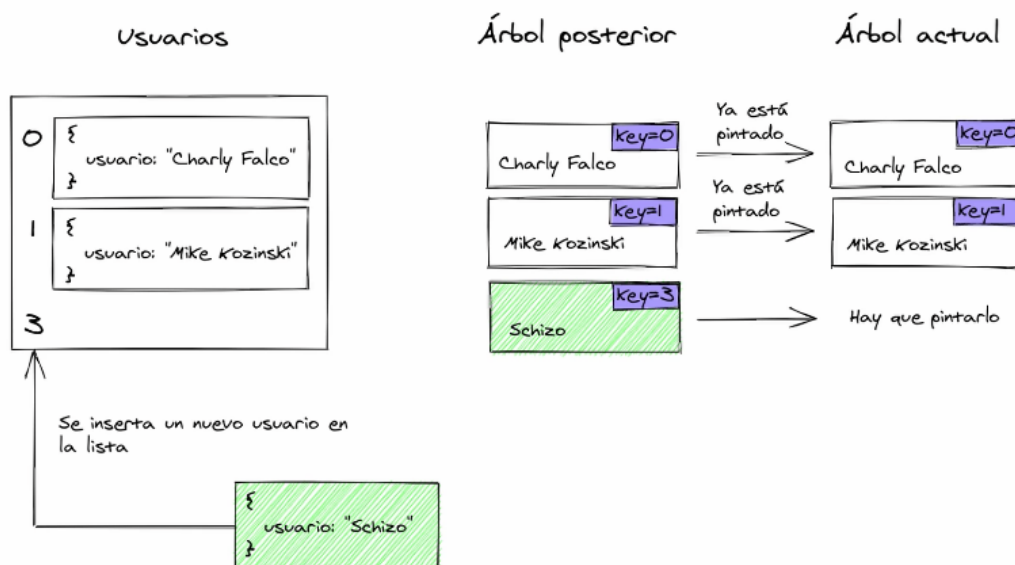
En el código de encima estamos usando la posición de los elementos como valor del atributo **key**, y esto es algo que soluciona el warning pero que no está bien del todo.

En React nos recomiendan que no utilicemos la posición como key, por tanto lo mejor sería utilizar un identificador único (que normalmente suelen tener todos los objetos).

```
items.map(item => <li key={item.id}>{item.texto}</li>)
```

Tenemos que utilizar este atributo para ayudar a React a que cualquier cambio sobre la lista se realice de la forma más eficiente posible.

Cada vez que se modifique la lista, React va a ir comparando los valores de los atributos **key** para saber si ese elemento ya estaba pintado o si es el que hay que pintar, o si hay que eliminar un elemento de esta, evitando tener que volver a pintar la lista entera desde 0 cada vez que cambia.



Podemos comprobar en los siguientes enlaces los problemas que pueden ocasionarnos no poner

bien las keys:

- Usando la posición: <https://es.reactjs.org/redirect-to-codepen/reconciliation/index-used-as-key>
- Usando un identificador único: <https://es.reactjs.org/redirect-to-codepen/reconciliation/no-index-used-as-key>

16.2. Lab: Lista de noticias

En este laboratorio vamos a ver como mostrar una lista de noticias con los datos que se cargan de un archivo JSON.

Vamos a copiar la plantilla del proyecto que se ha realizado en el laboratorio **reactjs-proyecto-inicial-lab** y vamos a cambiarle el nombre del proyecto a **reactjs-listas-lab**.



Es posible que sea necesario volver a instalar las dependencias del proyecto para que funcione correctamente. Para ello solo tenemos que lanzar el comando **npm install** dentro de la carpeta del proyecto.

Ahora vamos a levantar el servidor de desarrollo con el comando:

```
$ npm start
```

Empezamos por crear el archivo con los datos de las noticias a mostrar en **src/data/noticias.json**.

/reactjs-listas-lab/src/data/noticias.json

```
[
  {
    "id": "8s63jd3las3",
    "titulo": "Historiadores descubren que el caballo blanco de Santiago era blanco",
    "contenido": "Rem ab, animi ea pariatum praesentium at omnis obcaecati officia ipsum aspernatur ratione doloribus debitis nisi eligendi illum architecto voluptates amet. Quibusdam voluptatibus, doloremque ipsa officia vitae quasi reiciendis quod?"
  },
  {
    "id": "36klsd38fdj",
    "titulo": "El caballo del ajedrez se independiza y crea su propio juego de mesa de equitación",
    "contenido": "Necessitatibus, sed nostrum fugit consectetur aliquam quaerat repellat eaque quo rerum esse culpa sequi exercitationem magni non ea officiis aut? Perspiciatis, aspernatur? Delectus ducimus non veritatis inventore quod laboriosam tempora?"
  },
  {
    "id": "hs2lsk8523i",
    "titulo": "La orquesta del buque encallado en el Canal de Suez se niega a dejar de tocar",
    "contenido": "Consectetur et consequatur sint. Quibusdam, nihil quasi modi dolorum, eaque doloribus vero id distinctio tenetur perspiciatis ducimus consequuntur aut iusto facilis itaque eum sequi dolores aspernatur explicabo voluptate laudantium minus."
  },
  {
    "id": "l29d64f-1s",
    "titulo": "El Rover Perseverance atropella y pisa la única forma de vida que había en Marte",
    "contenido": "Aliquid cupiditate omnis repellendus expedita laborum illo, corrupti voluptates sed, ducimus rerum eligendi? Adipisci voluptatum expedita eos quis beatae illum asperiores nemo omnis ratione? Blanditiis, deleniti? Rem alias labore itaque."
  },
  {
    "id": "ux6-23e7c6",
    "titulo": "España pondrá un sello a los turistas franceses para que puedan volver a entrar",
    "contenido": "Aperiam commodi nemo beatae amet quibusdam? Sint quas ex dicta. Consequuntur quo pariatum corporis unde asperiores nesciunt reprehenderit cum, impedit, esse et omnis hic iste placeat voluptatum quisquam recusandae eius."
  }
]
```

Ahora vamos a crear un componente **Noticia** dentro de la carpeta de **components** con el que vamos a mostrar los datos de cada noticia.

Este componente recibirá como propiedad un objeto **noticia** con los datos de una noticia que vamos a mostrar.

/reactjs-listas-lab/src/components/Noticia.jsx

```
const Noticia = ({noticia}) => {  
  return (  
    <div>  
      <h2>{noticia.titulo}</h2>  
      <p>{noticia.contenido}</p>  
    </div>  
  )  
}  
  
export default Noticia
```

Ahora nos vamos a ir al componente **App** donde empezaremos por cargar los datos de las noticias del JSON, que será una simple importación del archivo.

/reactjs-listas-lab/src/components/App.jsx

```
import noticias from '../data/noticias.json'  
  
const App = () => {  
  
  return (  
    <div>  
  
    </div>  
  )  
}  
  
export default App
```

Una vez que tenemos los datos, usaremos el método **map** sobre el array de noticias con el que vamos a crear un array de componentes **Noticia**.

/reactjs-listas-lab/src/components/App.jsx

```
import noticias from '../data/noticias.json'  
import Noticia from './Noticia'  
  
const App = () => {  
  
  return (  
    <div>  
      {noticias.map(noticia => <Noticia />)}  
    </div>  
  )  
}
```

```

    </div>
  )
}

export default App

```

Ahora le vamos a pasar a cada componente Noticia la propiedad **noticia** con el objeto noticia que obtenemos de iterar sobre el array, y además añadiremos la propiedad **key** cuyo valor será el identificador de cada una de las noticias.

/reactjs-listas-lab/src/components/App.jsx

```

import noticias from '../data/noticias.json'
import Noticia from './Noticia'

const App = () => {

  return (
    <div>
      {noticias.map(noticia => <Noticia key={noticia.id} noticia={noticia} />)}
    </div>
  )
}

export default App

```

Y con esto ya deberíamos de poder ver nuestras noticias en la aplicación.

Chapter 17. Eventos

React no usa los eventos nativos, sino que utilizaremos los **eventos sintéticos** que han implementado. Estos eventos son un wrapper sobre los eventos nativos y se han encargado de normalizar los eventos para que funcionen en todos los navegadores de la misma forma.

Estos eventos tienen la misma API que los nativos, por lo que podremos acceder a las mismas propiedades a las que accederíamos en los eventos nativos.

Para poder usar estos eventos tenemos que ponerlos en las etiquetas con forma de **camelCase**.

Además React se encarga de eliminar automáticamente los **EventListeners** una vez que se desmonta el componente.

17.1. Tipos de eventos

Los eventos disponibles son los siguientes:

Table 1. Eventos táctiles

onTouchStart	onTouchMove	onTouchEnd	onTouchCancel
--------------	-------------	------------	---------------

Table 2. Eventos de ratón

onClick	onDoubleClick	onMouseDown
onMouseUp	onMouseOver	onMouseOut
onMouseEnter	onMouseLeave	onMouseMove
onContextMenu		

Table 3. Eventos de arrastrar y soltar

onDrag	onDragEnter	onDragLeave
onDragExit	onDragStart	onDragEnd
onDragOver	onDrop	

Table 4. Eventos de teclado

onKeyDown	onKeyUp	onKeyPress
-----------	---------	------------

Table 5. Eventos de foco y formularios

onFocus	onBlur	onChange
onInput	onSubmit	

Table 6. Otros eventos

onCopy	onCut	onPaste
onScroll	onWheel	

Podemos encontrar más eventos disponibles en <https://es.reactjs.org/docs/events.html#supported-events>.

17.2. Lab: Eventos

En este laboratorio vamos a ver como detectar el evento click para hacer que nuestro navegador cante la intro de la serie de dibujos de Batman, pero mal cantada.

Vamos a copiar la plantilla del proyecto que se ha realizado en el laboratorio **reactjs-proyecto-inicial-lab** y vamos a cambiarle el nombre del proyecto a **reactjs-eventos-lab**.



Es posible que sea necesario volver a instalar las dependencias del proyecto para que funcione correctamente. Para ello solo tenemos que lanzar el comando **npm install** dentro de la carpeta del proyecto.

Ahora vamos a levantar el servidor de desarrollo con el comando:

```
$ npm start
```

Lo primero que vamos a hacer es crear un botón y la función que se encargará de hacer que el navegador hable.

/reactjs-eventos-lab/src/components/App.jsx

```
const App = () => {
  const handleClick = () => {

  }

  return (
    <div>
      <button type="button">Que suene la intro</button>
    </div>
  )
}

export default App
```

Ahora vamos a rellenar la función de **handleClick**.

Empezaremos añadiendo el código que genera el texto que queremos que diga el navegador. Este texto se lo pasaremos al constructor de **SpeechSynthesisUtterance**.

Con la instancia generada vamos a cambiar el valor de **rate** para que hable un poco más lento de lo que por defecto hace.

Y por último llamaremos a **window.speechSynthesis.speak** para pasarle la configuración anterior y que finalmente nos cante mal la intro.

/reactjs-eventos-lab/src/components/App.jsx

```
const App = () => {
```



```

const handleClick = () => {
  const textoIntro = new Array(16).join(1-'wat') + ' Batman!';
  const configSpeech = new SpeechSynthesisUtterance(textoIntro);
  configSpeech.rate = 0.8;
  window.speechSynthesis.speak(configSpeech);
}

return (
  <div>
    <button type="button">Que suene la intro</button>
  </div>
)
}

export default App

```

Finalmente, si pulsamos el botón, vemos que no hace nada, y esto se debe a que nos falta poner el evento **onClick** y asignarle la función que queremos que se ejecute.

/reactjs-eventos-lab/src/components/App.jsx

```

const App = () => {
  const handleClick = () => {
    const textoIntro = new Array(16).join(1-'wat') + ' Batman!';
    const configSpeech = new SpeechSynthesisUtterance(textoIntro);
    configSpeech.rate = 0.8;
    window.speechSynthesis.speak(configSpeech);
  }

  return (
    <div>
      <button type="button" onClick={handleClick}>Que suene la intro</button>
    </div>
  )
}

export default App

```

Ahora ya debería de escucharse la intro al pulsar el botón.

Chapter 18. Estados

Como hemos dicho antes, las propiedades no cambian, y necesitamos un mecanismo por el que podamos volver a renderizar los componentes con una propiedades nuevas. Este mecanismo son los **estados**, y una vez que el estado cambia vuelve a lanzar el render para mostrar esos cambios.

El estado es **mutable** sólo por el propio componente donde se ha definido, ni el componente padre ni los componentes hijos podrán modificar su valor.

Podemos enviarle datos que tenemos en el estado de un componente a los componentes hijos en forma de props.

En React los estados son opcionales, y podemos diferenciar los componentes en componentes que no tienen estado y aquellos que si lo tienen.



Una buena práctica es mantener la mayor parte de los componentes sin estado.

El estado de un componente de clase se inicializa en el **constructor** del componente y es un objeto de JavaScript en el que le vamos a dar unos valores por defecto. El método **constructor()** recibe como parámetro las **props** que le envía el componente padre. Dentro del constructor usamos **super()** para inicializar la instancia del componente y Component le va a añadir funcionalidad para poder manejar los cambios de estado.

```
class MiComponente extends Component {
  constructor(props) {
    super(props);
    this.state = {
      clave: 'valor'
    }
  }

  render() {
    return (
      <div>Contador: {this.state.clave}</div>
    )
  }
}
```

Para cambiar el estado hay que usar el método **setState()** que recibe un objeto con los atributos que se van a modificar, y una vez que se ha realizado la modificación del estado, lanza un nuevo render para actualizar la interfaz de usuario.

```
this.setState({
  clave: 'nuevoValor'
})
```



Para ver como añadir el estado en los componentes funcionales tenemos que ir al

tema de **React Hooks**, concretamente al hook de **useState**.

18.1. Lab: Estado

En este laboratorio vamos a ver como utilizar el estado de los componentes con un panel de teclas con el que hay que encontrar el código secreto pulsando sobre ellas. Tenemos que tener en cuenta lo siguiente:

- Al pulsar sobre las teclas **del 0 al 9** se añade el número pulsado al final del código actual.
- Al pulsar sobre la tecla **CLD** borramos el código y lo dejamos vacío.
- Al pulsar sobre la tecla **DEL** borramos el último número que contiene el código.
- No se pueden introducir más de 4 números.
- Cuando acertamos el código secreto se muestra en el display el mensaje **CODE OK**.

Vamos a copiar la plantilla del proyecto que se ha realizado en el laboratorio **reactjs-proyecto-inicial-lab** y vamos a cambiarle el nombre del proyecto a **reactjs-estado-lab**.



Es posible que sea necesario volver a instalar las dependencias del proyecto para que funcione correctamente. Para ello solo tenemos que lanzar el comando **npm install** dentro de la carpeta del proyecto.

Ahora vamos a levantar el servidor de desarrollo con el comando:

```
$ npm start
```

Empezaremos creando el siguiente archivo de estilos para que nuestro teclado no quede demasiado cutre.

/reactjs-estado-lab/src/style.css

```
.panel-codigo-secreto {  
  width: 150px;  
  background-color: #242424;  
}  
  
.display {  
  height: 50px;  
  font-size: 2rem;  
  font-style: italic;  
  color: #34e89e;  
  text-align: right;  
}  
  
.fila-teclas {  
  display: flex;  
}  
  
.tecla {  
  background-color: transparent;
```

```

color: #34e89e;
border: 1px solid #34e89e;
width: 100%;
height: 50px;
cursor: pointer;
}

.tecla:hover {
  opacity: 0.6;
}

```

Ahora vamos a crear el componente **PanelCodigoSecreto.jsx** en la carpeta de **components**, y pondremos el código de JSX que pinta el panel. Este componente será un componente de clase.

/reactjs-estado-lab/src/components/PanelCodigoSecreto.jsx

```

import React, { Component } from 'react'

class PanelCodigoSecreto extends Component {
  render() {
    return (
      <div className="panel-codigo-secreto">
        <div className="display">
          0000
        </div>
        <div className="teclas">
          <div className="fila-teclas">
            <button type="button" className="tecla">1</button>
            <button type="button" className="tecla">2</button>
            <button type="button" className="tecla">3</button>
          </div>
          <div className="fila-teclas">
            <button type="button" className="tecla">4</button>
            <button type="button" className="tecla">5</button>
            <button type="button" className="tecla">6</button>
          </div>
          <div className="fila-teclas">
            <button type="button" className="tecla">7</button>
            <button type="button" className="tecla">8</button>
            <button type="button" className="tecla">9</button>
          </div>
          <div className="fila-teclas">
            <button type="button" className="tecla">CLD</button>
            <button type="button" className="tecla">0</button>
            <button type="button" className="tecla">DEL</button>
          </div>
        </div>
      </div>
    )
  }
}

```

```
}  
  
export default PanelCodigoSecreto;
```

Y antes de añadirle la lógica al componente vamos a mostrarlo en el componente **App.jsx**.

/reactjs-estado-lab/src/components/App.jsx

```
import PanelCodigoSecreto from './PanelCodigoSecreto';  
  
const App = () => {  
  return (  
    <div>  
      <PanelCodigoSecreto />  
    </div>  
  )  
}  
  
export default App
```

Si nos fijamos, los estilos no se están aplicando, esto se debe a que no los hemos importado en nuestra aplicación. Tenemos que realizar la importación en el **index.js**.

/reactjs-estado-lab/src/index.js

```
import { createRoot } from 'react-dom/client';  
import App from './components/App';  
import './style.css';  
  
createRoot(document.getElementById('root')).render(<App />);
```

Ahora ya deberían de estar aplicándose los estilos correctamente.



En un tema posterior veremos porque hay que hacerlo así y que otras formas de trabajar con los estilos tenemos en React.

Ahora vamos a empezar a añadirle la lógica que nos piden al componente **PanelCodigoSecreto**.

Empezaremos por añadirle el estado que necesita. Utilizaremos dentro del estado la clave **codigoActual** para ir almacenando el código que está escribiendo el usuario. Además también vamos a guardar en el estado el código secreto que hay que adivinar, dentro de la clave **codigoSecreto**.

Como estamos con un componente de clase, el estado lo tenemos que inicializar en el constructor del componente.

/reactjs-estado-lab/src/components/PanelCodigoSecreto.jsx

```
import React, { Component } from 'react'
```

```

class PanelCodigoSecreto extends Component {
  constructor(props) {
    super(props);
    this.state = {
      codigoSecreto: '3038',
      codigoActual: ''
    }
  }

  render() {
    return (
      <div className="panel-codigo-secreto">
        <div className="display">
          0000
        </div>
        <div className="teclas">
          <div className="fila-teclas">
            <button type="button" className="tecla">1</button>
            <button type="button" className="tecla">2</button>
            <button type="button" className="tecla">3</button>
          </div>
          <div className="fila-teclas">
            <button type="button" className="tecla">4</button>
            <button type="button" className="tecla">5</button>
            <button type="button" className="tecla">6</button>
          </div>
          <div className="fila-teclas">
            <button type="button" className="tecla">7</button>
            <button type="button" className="tecla">8</button>
            <button type="button" className="tecla">9</button>
          </div>
          <div className="fila-teclas">
            <button type="button" className="tecla">CLD</button>
            <button type="button" className="tecla">0</button>
            <button type="button" className="tecla">DEL</button>
          </div>
        </div>
      </div>
    )
  }
}

export default PanelCodigoSecreto;

```

Ahora vamos a sustituir el código que se está mostrando por defecto (0000) por el código actual que se va almacenando en el estado.

```
import React, { Component } from 'react'

class PanelCodigoSecreto extends Component {
  constructor(props) {
    super(props);
    this.state = {
      codigoSecreto: '3038',
      codigoActual: ''
    }
  }

  render() {
    return (
      <div className="panel-codigo-secreto">
        <div className="display">
          {this.state.codigoActual}
        </div>
        <div className="teclas">
          <div className="fila-teclas">
            <button type="button" className="tecla">1</button>
            <button type="button" className="tecla">2</button>
            <button type="button" className="tecla">3</button>
          </div>
          <div className="fila-teclas">
            <button type="button" className="tecla">4</button>
            <button type="button" className="tecla">5</button>
            <button type="button" className="tecla">6</button>
          </div>
          <div className="fila-teclas">
            <button type="button" className="tecla">7</button>
            <button type="button" className="tecla">8</button>
            <button type="button" className="tecla">9</button>
          </div>
          <div className="fila-teclas">
            <button type="button" className="tecla">CLD</button>
            <button type="button" className="tecla">0</button>
            <button type="button" className="tecla">DEL</button>
          </div>
        </div>
      </div>
    )
  }
}

export default PanelCodigoSecreto;
```

El siguiente paso es añadir el evento **onClick** en el bloque de teclas y crear la función a la que se va a llamar desde el evento.


```
import React, { Component } from 'react'

class PanelCodigoSecreto extends Component {
  constructor(props) {
    super(props);
    this.state = {
      codigoSecreto: '3038',
      codigoActual: ''
    }
  }

  handleClick(event) {

  }

  render() {
    return (
      <div className="panel-codigo-secreto">
        <div className="display">
          {this.state.codigoActual}
        </div>
        <div className="teclas" onClick={this.handleClick}>
          <div className="fila-teclas">
            <button type="button" className="tecla">1</button>
            <button type="button" className="tecla">2</button>
            <button type="button" className="tecla">3</button>
          </div>
          <div className="fila-teclas">
            <button type="button" className="tecla">4</button>
            <button type="button" className="tecla">5</button>
            <button type="button" className="tecla">6</button>
          </div>
          <div className="fila-teclas">
            <button type="button" className="tecla">7</button>
            <button type="button" className="tecla">8</button>
            <button type="button" className="tecla">9</button>
          </div>
          <div className="fila-teclas">
            <button type="button" className="tecla">CLD</button>
            <button type="button" className="tecla">0</button>
            <button type="button" className="tecla">DEL</button>
          </div>
        </div>
      </div>
    )
  }
}
```

```
export default PanelCodigoSecreto;
```

Dentro de la función del **handleClick** vamos a empezar sacando el texto de la tecla sobre la que se ha pulsado accediendo al **event.target.textContent**.

También vamos a extraer del estado los dos valores que estamos almacenando.

/reactjs-estado-lab/src/components/PanelCodigoSecreto.jsx

```
import React, { Component } from 'react'

class PanelCodigoSecreto extends Component {
  constructor(props) {
    super(props);
    this.state = {
      codigoSecreto: '3038',
      codigoActual: ''
    }
  }

  handleClick(event) {
    const teclaPulsada = event.target.textContent;
    const { codigoActual, codigoSecreto } = this.state;

  }

  render() {
    return (
      <div className="panel-codigo-secreto">
        <div className="display">
          {this.state.codigoActual}
        </div>
        <div className="teclas" onClick={this.handleClick}>
          <div className="fila-teclas">
            <button type="button" className="tecla">1</button>
            <button type="button" className="tecla">2</button>
            <button type="button" className="tecla">3</button>
          </div>
          <div className="fila-teclas">
            <button type="button" className="tecla">4</button>
            <button type="button" className="tecla">5</button>
            <button type="button" className="tecla">6</button>
          </div>
          <div className="fila-teclas">
            <button type="button" className="tecla">7</button>
            <button type="button" className="tecla">8</button>
            <button type="button" className="tecla">9</button>
          </div>
          <div className="fila-teclas">
            <button type="button" className="tecla">CLD</button>
            <button type="button" className="tecla">0</button>
          </div>
        </div>
      </div>
    );
  }
}
```

```

        <button type="button" className="tecla">DEL</button>
      </div>
    </div>
  </div>
)
}
}

export default PanelCodigoSecreto;

```

Después de añadir el código anterior veremos que nos da un error en la consola y nos dice que no se puede acceder a **state** de **undefined**. Esto se debe a que el objeto **this** es undefined en lugar de ser la instancia de la clase en la que se encuentra este método.

Podemos solucionarlo bindeando en el constructor el **this** a esta función.

/reactjs-estado-lab/src/components/PanelCodigoSecreto.jsx

```

import React, { Component } from 'react'

class PanelCodigoSecreto extends Component {
  constructor(props) {
    super(props);
    this.state = {
      codigoSecreto: '3038',
      codigoActual: ''
    }
    this.handleClick = this.handleClick.bind(this);
  }

  handleClick(event) {
    const teclaPulsada = event.target.textContent;
    const { codigoActual, codigoSecreto } = this.state;

  }

  render() {
    return (
      <div className="panel-codigo-secreto">
        <div className="display">
          {this.state.codigoActual}
        </div>
        <div className="teclas" onClick={this.handleClick}>
          <div className="fila-teclas">
            <button type="button" className="tecla">1</button>
            <button type="button" className="tecla">2</button>
            <button type="button" className="tecla">3</button>
          </div>
          <div className="fila-teclas">
            <button type="button" className="tecla">4</button>

```

```

        <button type="button" className="tecla">5</button>
        <button type="button" className="tecla">6</button>
    </div>
    <div className="fila-teclas">
        <button type="button" className="tecla">7</button>
        <button type="button" className="tecla">8</button>
        <button type="button" className="tecla">9</button>
    </div>
    <div className="fila-teclas">
        <button type="button" className="tecla">CLD</button>
        <button type="button" className="tecla">0</button>
        <button type="button" className="tecla">DEL</button>
    </div>
</div>
</div>
)
}
}

export default PanelCodigoSecreto;

```

Solucionado el error anterior, vamos a poner los 3 casos de teclas que nos podemos encontrar, **CLD**, **DEL** o una **tecla numérica**.

/reactjs-estado-lab/src/components/PanelCodigoSecreto.jsx

```

import React, { Component } from 'react'

class PanelCodigoSecreto extends Component {
  constructor(props) {
    super(props);
    this.state = {
      codigoSecreto: '3038',
      codigoActual: ''
    }
    this.handleClick = this.handleClick.bind(this);
  }

  handleClick(event) {
    const teclaPulsada = event.target.textContent;
    const { codigoActual, codigoSecreto } = this.state;

    if (teclaPulsada === 'CLD') {

    } else if (teclaPulsada === 'DEL') {

    } else {
      if (codigoActual.length < 4) {

      }
    }
  }
}

```

```

    }
  }

  render() {
    return (
      <div className="panel-codigo-secreto">
        <div className="display">
          {this.state.codigoActual}
        </div>
        <div className="teclas" onClick={this.handleClick}>
          <div className="fila-teclas">
            <button type="button" className="tecla">1</button>
            <button type="button" className="tecla">2</button>
            <button type="button" className="tecla">3</button>
          </div>
          <div className="fila-teclas">
            <button type="button" className="tecla">4</button>
            <button type="button" className="tecla">5</button>
            <button type="button" className="tecla">6</button>
          </div>
          <div className="fila-teclas">
            <button type="button" className="tecla">7</button>
            <button type="button" className="tecla">8</button>
            <button type="button" className="tecla">9</button>
          </div>
          <div className="fila-teclas">
            <button type="button" className="tecla">CLD</button>
            <button type="button" className="tecla">0</button>
            <button type="button" className="tecla">DEL</button>
          </div>
        </div>
      </div>
    )
  }
}

export default PanelCodigoSecreto;

```

Ahora vamos a crear una variable **nuevoCodigoActual** en la que guardaremos el siguiente valor del código.

- Cuando se pulsa **CLD** lo dejamos vacío.
- Cuando se pulsa **DEL** vamos a quitar el último carácter utilizando la función **slice** de los strings.
- Para el resto de casos (cuando pulsamos los números), vamos a comprobar primero si tenemos 4 números y si es así no hacemos nada. Pero si tenemos menos de 4 entonces vamos a añadir el número pulsado al final del código actual.

```
import React, { Component } from 'react'

class PanelCodigoSecreto extends Component {
  constructor(props) {
    super(props);
    this.state = {
      codigoSecreto: '3038',
      codigoActual: ''
    }
    this.handleClick = this.handleClick.bind(this);
  }

  handleClick(event) {
    const teclaPulsada = event.target.textContent;
    const { codigoActual, codigoSecreto } = this.state;

    let nuevoCodigoActual = codigoActual;
    if (teclaPulsada === 'CLD') {
      nuevoCodigoActual = ''
    } else if (teclaPulsada === 'DEL') {
      nuevoCodigoActual = codigoActual.slice(0, codigoActual.length-1);
    } else {
      if (codigoActual.length < 4) {
        nuevoCodigoActual = codigoActual + teclaPulsada;
      }
    }
  }

  render() {
    return (
      <div className="panel-codigo-secreto">
        <div className="display">
          {this.state.codigoActual}
        </div>
        <div className="teclas" onClick={this.handleClick}>
          <div className="fila-teclas">
            <button type="button" className="tecla">1</button>
            <button type="button" className="tecla">2</button>
            <button type="button" className="tecla">3</button>
          </div>
          <div className="fila-teclas">
            <button type="button" className="tecla">4</button>
            <button type="button" className="tecla">5</button>
            <button type="button" className="tecla">6</button>
          </div>
          <div className="fila-teclas">
            <button type="button" className="tecla">7</button>
            <button type="button" className="tecla">8</button>
            <button type="button" className="tecla">9</button>
          </div>
        </div>
      </div>
    )
  }
}
```

```

        </div>
        <div className="fila-teclas">
          <button type="button" className="tecla">CLD</button>
          <button type="button" className="tecla">0</button>
          <button type="button" className="tecla">DEL</button>
        </div>
      </div>
    </div>
  )
}
}

export default PanelCodigoSecreto;

```

Ahora vamos a cambiar el estado usando la función **this.setState** y pasándole el objeto con el nuevo código.

/reactjs-estado-lab/src/components/PanelCodigoSecreto.jsx

```

import React, { Component } from 'react'

class PanelCodigoSecreto extends Component {
  constructor(props) {
    super(props);
    this.state = {
      codigoSecreto: '3038',
      codigoActual: ''
    }
    this.handleClick = this.handleClick.bind(this);
  }

  handleClick(event) {
    const teclaPulsada = event.target.textContent;
    const { codigoActual, codigoSecreto } = this.state;

    let nuevoCodigoActual = codigoActual;
    if (teclaPulsada === 'CLD') {
      nuevoCodigoActual = ''
    } else if (teclaPulsada === 'DEL') {
      nuevoCodigoActual = codigoActual.slice(0, codigoActual.length-1);
    } else {
      if (codigoActual.length < 4) {
        nuevoCodigoActual = codigoActual + teclaPulsada;
      }
    }

    this.setState({
      codigoActual: nuevoCodigoActual
    });
  }
}

```

```

render() {
  return (
    <div className="panel-codigo-secreto">
      <div className="display">
        {this.state.codigoActual}
      </div>
      <div className="teclas" onClick={this.handleClick}>
        <div className="fila-teclas">
          <button type="button" className="tecla">1</button>
          <button type="button" className="tecla">2</button>
          <button type="button" className="tecla">3</button>
        </div>
        <div className="fila-teclas">
          <button type="button" className="tecla">4</button>
          <button type="button" className="tecla">5</button>
          <button type="button" className="tecla">6</button>
        </div>
        <div className="fila-teclas">
          <button type="button" className="tecla">7</button>
          <button type="button" className="tecla">8</button>
          <button type="button" className="tecla">9</button>
        </div>
        <div className="fila-teclas">
          <button type="button" className="tecla">CLD</button>
          <button type="button" className="tecla">0</button>
          <button type="button" className="tecla">DEL</button>
        </div>
      </div>
    </div>
  )
}
}

export default PanelCodigoSecreto;

```

Ya deberíamos de poder escribir el código, pero nos falta una última cosa. Tenemos que mostrar el mensaje **CODE OK** cuando escribimos el código correcto, por lo que vamos a añadir en el **else** una instrucción **if** para comprobar si el código actual es el código secreto, y en caso de serlo cambiar el valor de **nuevoCodigoActual**.

/reactjs-estado-lab/src/components/PanelCodigoSecreto.jsx

```

import React, { Component } from 'react'

class PanelCodigoSecreto extends Component {
  constructor(props) {
    super(props);
    this.state = {
      codigoSecreto: '3038',

```



```

        codigoActual: ''
    }
    this.handleClick = this.handleClick.bind(this);
}

handleClick(event) {
    const teclaPulsada = event.target.textContent;
    const { codigoActual, codigoSecreto } = this.state;

    let nuevoCodigoActual = codigoActual;
    if (teclaPulsada === 'CLD') {
        nuevoCodigoActual = ''
    } else if (teclaPulsada === 'DEL') {
        nuevoCodigoActual = codigoActual.slice(0, codigoActual.length-1);
    } else {
        if (codigoActual.length < 4) {
            nuevoCodigoActual = codigoActual + teclaPulsada;
            if (nuevoCodigoActual === codigoSecreto) {
                nuevoCodigoActual = 'CODE OK';
            }
        }
    }

    this.setState({
        codigoActual: nuevoCodigoActual
    });
}

render() {
    return (
        <div className="panel-codigo-secreto">
            <div className="display">
                {this.state.codigoActual}
            </div>
            <div className="teclas" onClick={this.handleClick}>
                <div className="fila-teclas">
                    <button type="button" className="tecla">1</button>
                    <button type="button" className="tecla">2</button>
                    <button type="button" className="tecla">3</button>
                </div>
                <div className="fila-teclas">
                    <button type="button" className="tecla">4</button>
                    <button type="button" className="tecla">5</button>
                    <button type="button" className="tecla">6</button>
                </div>
                <div className="fila-teclas">
                    <button type="button" className="tecla">7</button>
                    <button type="button" className="tecla">8</button>
                    <button type="button" className="tecla">9</button>
                </div>
                <div className="fila-teclas">

```

```
        <button type="button" className="tecla">CLD</button>
        <button type="button" className="tecla">0</button>
        <button type="button" className="tecla">DEL</button>
      </div>
    </div>
  </div>
)
}
}

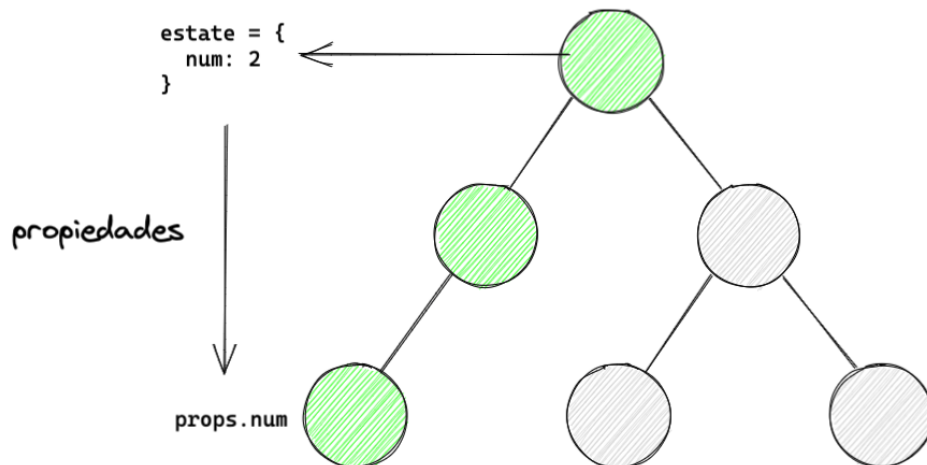
export default PanelCodigoSecreto;
```

Ya debería de funcionar el panel de teclas como se indicaba al principio.

Chapter 19. Flujo de datos

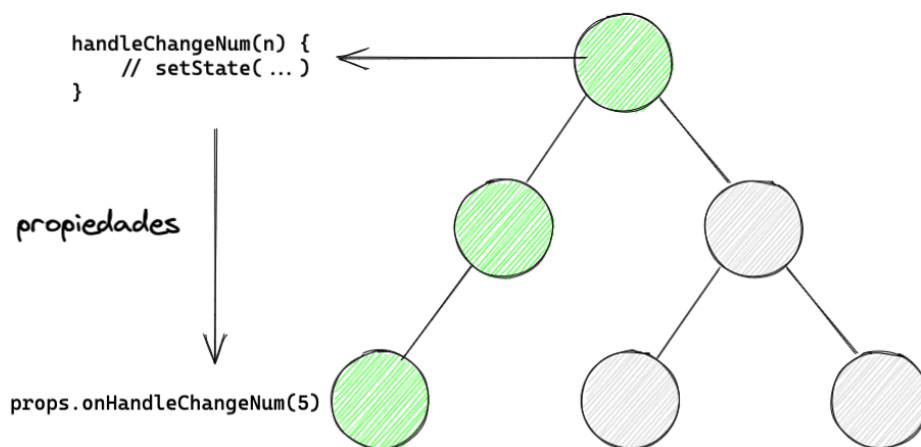
En las aplicaciones de React, los datos que van recibiendo los componentes (las propiedades) siempre viajan desde arriba hacia abajo, es decir, de componentes padres a componentes hijos.

Estos datos nunca irán en dirección contraria.



Cuando se necesita cambiar el valor de una propiedad que proviene del estado de un componente definido varios niveles por encima, necesitamos llamar al método **setState** de dicho componente. Pero este método no es accesible directamente desde los componentes inferiores, por lo que pasaremos en forma de propiedades la referencia a las funciones encargadas de cambiar el estado.

Desde el componente inferior que necesite realizar el cambio de estado ejecutará la referencia de la función que ha recibido como propiedad.



Chapter 20. Ciclo de vida

Todos los componentes siguen un **ciclo de vida**, lo que permite ejecutar unos métodos automáticamente en ciertas ocasiones o cada vez que ocurre algún evento importante sobre el componente como cuando se actualiza o se va a destruir.

Estos métodos se pueden dividir en 3 grupos:

- Construcción
- Actualización
- Destrucción

20.1. constructor

El **constructor** no es un método del ciclo de vida, sino que es algo propio de las clases de JavaScript.

Este método solo es necesario si:

- Tenemos que inicializar el estado
- Tenemos que enlazar handlers a la instancia del componente

```
constructor(props) {  
  super(props);  
  
  this.state = {...};  
  this.handleClick = this.handleClick.bind(this);  
}
```



Evita añadir al estado directamente el valor de las propiedades. No tiene sentido, ya que puedes acceder a ellas con **this.props**.

Otra cosa es cuando el estado se deriva del valor de estas. Para ello usaremos el método **getDerivedStateFromProps**.

20.2. getDerivedStateFromProps

El método **static getDerivedStateFromProps** es un método que se va a ejecutar justo antes de ejecutar el método de **render** de los componentes. Se ejecuta tanto al montar el componente como al actualizarlo.

En este método recibimos las propiedades y el estado que vamos a tener después de renderizar el componente por lo que podremos compararlas con las propiedades y estado actuales del componente para calcular si el estado se tiene que modificar o no.

Si la función devuelve un null no se actualiza nada, pero si devolvemos un objeto, este actualizará el estado del componente.

Es un método del ciclo de vida avanzado y que no se suele utilizar mucho, pero tendremos que usarlo cuando **el valor del estado depende del valor de alguna de las propiedades que recibe el componente**.

```
static getDerivedStateFromProps(props, state) {  
  return {  
    texto: props.number % 2 === 0 ? 'es par' : 'es impar'  
  }  
}
```

20.3. componentDidMount

El **componentDidMount** es el método que se ejecuta después de montar (insertar el componente en el árbol) el componente utilizaremos para tareas de inicialización:

- Peticiones HTTP para obtener datos de la red
- Inicilizar observables y timers
- Crear elementos del DOM con otras librerías (jQuery)

```
componentDidMount() {  
  fetch('https://miapp.com/api/datos')  
    .then(resp => resp.json())  
    .then(datos => {  
      this.setState({datos});  
    })  
}
```

20.4. shouldComponentUpdate

El método **shouldComponentUpdate** es un método que se ejecuta justo antes del render.

Este método nos permite indicarle a React si hace falta volver a renderizar el componente o no. Es un método que se utiliza para **mejorar el rendimiento** de las aplicaciones ahorrando renderizados innecesarios de algunos componentes.

Para indicarle a React si tiene que volver a pintar el componente o no, este método tiene que devolver un valor booleano:

- true: se renderiza
- false: no se renderiza

Para saber si es necesario renderizar el componente vamos a recibir las siguientes propiedades y el siguiente estado que podremos utilizar para comparar con los valores actuales (antes del render).

```
shouldComponentUpdate(nextProps, nextState) {  
  return nextProps.valor !== this.props.valor ? true : false;  
}
```



Si se devuelve false, al no actualizar el componente tampoco se ejecutarán los métodos de **render** y **componentDidUpdate**.

20.5. getSnapshotBeforeUpdate

El método **getSnapshotBeforeUpdate** se ejecuta después del método **render**, pero antes de que los cambios se realicen sobre el DOM real. Este es uno de los métodos del ciclo de vida menos utilizados.

Este método recibe las propiedades y el estado anteriores y con el podemos devolver datos que llegarán al siguiente método del ciclo de vida **componentDidUpdate**.

Este método se usa cuando necesitamos información de la vista o del componente justo antes del cambio para compararla con la misma información después del cambio y saber si hay que realizar alguna modificación.

Un caso donde se podría utilizar es cuando tenemos un chat y escribimos un mensaje. Podríamos calcular si es necesario hacer un scroll automático para que el mensaje se muestre en caso de que no entre en la pantalla.

```
getSnapshotBeforeUpdate(prevProps, prevState) {  
  if (prevProps.mensajes.length < this.props.mensajes.length) {  
    const listaMensajes = this.mensajesRef.current;  
    return listaMensajes.scrollHeight - listaMensajes.scrollTop;  
  }  
  return null;  
}
```

20.6. componentDidUpdate

El **componentDidUpdate** es un método que se ejecuta justo después de que el componente se actualice.

Este método se puede utilizar para volver a pedir unos datos del exterior (una API) o para actualizar elementos del DOM gestionados por otras librerías.

Este método recibe las propiedades anteriores para poder compararlas con las actuales y saber por ejemplo si es necesario volver a pedir unos datos. Quizás el dato por el cual pediríamos los datos sigue siendo el mismo, no haría falta realizar la petición de nuevo.

Recibe un parámetro más, el **snapshot** que se lo manda el método **getSnapshotBeforeUpdate** con la info necesaria para modificar algo en el DOM (por ejemplo el scroll).

```
componentDidUpdate(prevProps, snapshot) {  
  if (prevProps.textoFiltro !== this.props.textoFiltro) {  
    // Pedimos los datos filtrados por el nuevo texto  
  }  
}
```



Cuidado con modificar dentro de este método el estado sin añadir una condición, ya que entraremos en un bucle infinito de renderizados.

20.7. componentWillUnmount

El método **componentWillUnmount** se ejecuta justo antes de eliminar el componente del DOM.

Este método se suele utilizar para limpiar operaciones que se han inicializado anteriormente y que de no eliminarlas seguirían ejecutandose o gastando recursos.

Algunos ejemplos en los que se puede utilizar este método:

- Limpiar un timer (setInterval)
- Desuscribirse de observables
- Eliminar elementos HTML manejados por otras librerías (jQuery)

```
componentWillUnmount() {  
  observable.unsubscribe();  
}
```

20.8. Lab: Ciclo de vida

En este laboratorio vamos a ver como utilizar algunos de los métodos del ciclo de vida de los componentes.

Vamos a copiar la plantilla del proyecto que se ha realizado en el laboratorio **reactjs-proyecto-inicial-lab** y vamos a cambiarle el nombre del proyecto a **reactjs-ciclo-de-vida-lab**.



Es posible que sea necesario volver a instalar las dependencias del proyecto para que funcione correctamente. Para ello solo tenemos que lanzar el comando **npm install** dentro de la carpeta del proyecto.

Ahora vamos a levantar el servidor de desarrollo con el comando:

```
$ npm start
```

Empezaremos creando el componente **CajaColor** en el que vamos a utilizar un par de métodos del ciclo de vida de los componentes.

Este componente va a ser de clase porque queremos utilizar los métodos del ciclo de vida. También va a necesitar un estado en el que vamos a indicar de que color pintar la caja, que inicialmente vamos a dejar como **null**.

/reactjs-ciclo-de-vida-lab/src/components/CajaColor.jsx

```
import { Component } from 'react'

class CajaColor extends Component {
  constructor(props) {
    super(props);
    this.state = {
      color: null
    }
  }

  render() {
    console.log('Se ha vuelto a pintar...');
    return (
      <div style={{
        width: '100px',
        height: '100px',
        backgroundColor: this.state.color,
      }}></div>
    )
  }
}

export default CajaColor;
```

El color de la caja va a depender de un valor numérico que va a recibir como propiedad desde el componente **App**. La idea es que si el número es positivo, la caja se pinte en verde, mientras que si es negativo se pintará en rojo.

Así que el primer método que vamos a utilizar dentro de este componente es el **getDerivedStateFromProps** ya que como hemos dicho, el estado depende de un valor que se recibe como propiedad.

En este método recibimos como parámetros las propiedades y el estado. Accederemos a la propiedad del **num** y haremos que se devuelva un objeto con la parte del estado que queremos inicializar en base al valor de la propiedad.

/reactjs-ciclo-de-vida-lab/src/components/CajaColor.jsx

```
import { Component } from 'react'

class CajaColor extends Component {
  constructor(props) {
    super(props);
    this.state = {
      color: null
    }
  }

  static getDerivedStateFromProps(props, state) {
    return {
      color: props.num < 0 ? 'red' : 'green'
    }
  }

  render() {
    console.log('Se ha vuelto a pintar...');
    return (
      <div style={{
        width: '100px',
        height: '100px',
        backgroundColor: this.state.color,
      }}></div>
    )
  }
}

export default CajaColor;
```

Ahora vamos al componente **App** en el que crearemos un contador para cambiar un valor numérico que pasaremos como propiedad a este componente.

/reactjs-ciclo-de-vida-lab/src/components/App.jsx

```
import { useState } from "react"
import CajaColor from "../CajaColor"
```

```

const App = () => {
  const [cantidadTotal, setCantidadTotal] = useState(0);

  return (
    <div>
      <div>
        <label htmlFor="cantidadTotal">Introduce la cantidad total:</label>
        <input type="number" name="cantidadTotal" id="cantidadTotal" value={cantidadTotal} onChange={(e) =>
setCantidadTotal(e.target.value)} />
      </div>
      <CajaColor num={cantidadTotal} />
    </div>
  )
}

export default App

```

Si vamos al navegador a probar estos componentes, veremos que la caja se pinta de verde o rojo dependiendo del valor que introduzcamos en el campo de texto.

Pero tenemos un problema y es que si el número pasa de un valor positivo a otro positivo o de uno negativo a otro negativo, el componente se vuelve a renderizar sin mostrar ningún cambio significativo.

Vamos a solucionar este problema con el método **shouldComponentUpdate** el cual recibe las siguientes propiedades y el siguiente estado como parámetros.

Usaremos el siguiente estado para compararlo con el actual. En caso de que sean distintos devolveremos un true para indicarle a React que tiene que renderizar el componente, pero en caso de que sean iguales devolveremos un false porque el color no ha cambiado y por tanto no hace falta volver a renderizar el componente.

/reactjs-ciclo-de-vida-lab/src/components/CajaColor.jsx

```

import { Component } from 'react'

class CajaColor extends Component {
  constructor(props) {
    super(props);
    this.state = {
      color: null
    }
  }

  static getDerivedStateFromProps(props, state) {
    return {
      color: props.num < 0 ? 'red' : 'green'
    }
  }

  shouldComponentUpdate(nextProps, nextState) {
    return this.state.color !== nextState.color;
  }
}

```

```

render() {
  console.log('Se ha vuelto a pintar...');
  return (
    <div style={{
      width: '100px',
      height: '100px',
      backgroundColor: this.state.color,
    }}></div>
  )
}
}

export default CajaColor;

```

Ahora solo deberíamos de ver por consola el mensaje de **Se ha vuelto a pintar...** cuando pasamos de un número negativo a uno positivo o viceversa.

Para ver otros dos métodos del ciclo de vida, estos son de los más utilizados en React, vamos a crear otro componente **HoraActual**.

/reactjs-ciclo-de-vida-lab/src/components/HoraActual.jsx

```

import { Component } from 'react'

class HoraActual extends Component {
  constructor(props) {
    super(props);
  }

  render() {
    return (
      <div>

      </div>
    )
  }
}

export default HoraActual

```

Dentro del componente vamos a pintar la hora actual y para ello vamos a añadirla en el estado del componente.

En el método render llamaremos al método **toLocaleTimeString** de la fecha para mostrar solo la hora.

/reactjs-ciclo-de-vida-lab/src/components/HoraActual.jsx

```

import { Component } from 'react'

```

```

class HoraActual extends Component {
  constructor(props) {
    super(props);
    this.state = {
      fechaActual: new Date(),
    }
  }

  render() {
    return (
      <div>
        <p>{this.state.fechaActual.toLocaleTimeString()}</p>
      </div>
    )
  }
}

export default HoraActual

```

Ahora tenemos que conseguir que cada segundo que pase, la fecha se vaya actualizando para que se muestre la hora actual, y aquí es donde entra el primer método del ciclo de vida que vamos a usar.

Añadimos el **componentDidMount** en el que crearemos un timer que se ejecutará cada segundo y que irá modificando la fecha del estado.

/reactjs-ciclo-de-vida-lab/src/components/HoraActual.jsx

```

import { Component } from 'react'

class HoraActual extends Component {
  constructor(props) {
    super(props);
    this.state = {
      fechaActual: new Date(),
    }
  }

  componentDidMount() {
    const intervalId = setInterval(() => {
      console.log('Pidiendo la hora actual...')
      this.setState({
        fechaActual: new Date()
      })
    }, 1000);
  }

  render() {
    return (

```



```

    <div>
      <p>{this.state.fechaActual.toLocaleTimeString()}</p>
    </div>
  )
}
}

export default HoraActual

```

Ya deberíamos de ver en la aplicación la hora actualizándose cada segundo que pasa. Parece que ya tenemos lo que queríamos, pero no está hecho del todo bien. ¿Qué pasaría si borramos el componente y lo volvemos a crear?

Vamos a verlo. Para ello, en el componente **App**, vamos a añadir un botón para crear o eliminar este componente. Este botón modificará el estado del componente, un booleano que indica si se muestra o no la hora.

/reactjs-ciclo-de-vida-lab/src/components/App.jsx

```

import { useState } from 'react'
import CajaColor from './CajaColor'
import HoraActual from './HoraActual'

const App = () => {
  const [cantidadTotal, setCantidadTotal] = useState(0);
  const [mostrarHoraActual, setMostrarHoraActual] = useState(true);

  return (
    <div>
      <div>
        <label htmlFor="cantidadTotal">Introduce la cantidad total:</label>
        <input type="number" name="cantidadTotal" id="cantidadTotal" value={cantidadTotal} onChange={(e) =>
setCantidadTotal(e.target.value)} />
      </div>
      <CajaColor num={cantidadTotal} />
      <button type="button" onClick={() => setMostrarHoraActual(!mostrarHoraActual)}>Toggle HoraActual</button>
      <HoraActual />
    </div>
  )
}

export default App

```

Ahora vamos a utilizar el renderizado condicional de React para mostrar HoraActual cuando sea necesario.

/reactjs-ciclo-de-vida-lab/src/components/App.jsx

```

import { useState } from 'react'
import CajaColor from './CajaColor'
import HoraActual from './HoraActual'

const App = () => {
  const [cantidadTotal, setCantidadTotal] = useState(0);
  const [mostrarHoraActual, setMostrarHoraActual] = useState(true);

  return (

```

```

<div>
  <div>
    <label htmlFor="cantidadTotal">Introduce la cantidad total:</label>
    <input type="number" name="cantidadTotal" id="cantidadTotal" value={cantidadTotal} onChange={(e) =>
setCantidadTotal(e.target.value)} />
  </div>
  <CajaColor num={cantidadTotal} />
  <button type="button" onClick={() => setMostrarHoraActual(!mostrarHoraActual)}>Toggle HoraActual</button>
  {mostrarHoraActual && <HoraActual />}
</div>
)
}

export default App

```

Si pulsamos varias veces el botón, el componente se va a crear y se va a eliminar varias veces. El problema con ello es que si miramos la consola del navegador, seguramente veamos que nos muestra un warning, además de que el mensaje que estamos pintando dentro del **setInterval** se muestra muchas más veces de las que debería de mostrarse.

Esto se debe a que estamos acumulando timers cada vez que volvemos a crear el componente de HoraActual, pero no los estamos eliminando al desmontar el componente.

Para solucionarlo vamos a utilizar el método **componentWillUnmount** en el que llamaremos a la función de **clearInterval** para eliminar el timer que se ha creado.

El método anterior necesita recibir como parámetro el identificador del intervalo, por lo que tendremos que guardarlo también en el estado.

/reactjs-ciclo-de-vida-lab/src/components/HoraActual.jsx

```

import { Component } from 'react'

class HoraActual extends Component {
  constructor(props) {
    super(props);
    this.state = {
      fechaActual: new Date(),
      intervalId: null
    }
  }

  componentDidMount() {
    const intervalId = setInterval(() => {
      console.log('Pidiendo la hora actual...')
      this.setState({
        fechaActual: new Date()
      })
    }, 1000);

    this.setState({
      intervalId
    })
  }
}

```

```
componentWillUnmount() {  
  clearInterval(this.state.intervalId);  
}  
  
render() {  
  return (  
    <div>  
      <p>{this.state.fechaActual.toLocaleTimeString()}</p>  
    </div>  
  )  
}  
}  
  
export default HoraActual
```

Ahora ya podemos decir que este componente está funcionando correctamente y si pulsamos varias veces el botón que hemos añadido antes podemos ver la diferencia, ahora solo tendremos un solo timer ejecutándose en la aplicación.

Chapter 21. Referencias

Las **referencias** de React nos permiten acceder a los elementos del DOM. Equivaldrían a utilizar métodos como `getElementById` o `querySelector` del objeto `window.document`, pero como aquí estamos trabajando con un Virtual DOM, tenemos que evitar esos métodos a toda costa ya que pueden llegar a darnos algún problema.

Con las referencias, al poder acceder a los elementos HTML que se renderizan con los componentes, podremos acceder y modificar los atributos de estos.

Antes de ponernos a crear referencias en nuestros componentes, para poder acceder a ellos, tenemos que pensar en que si una etiqueta HTML necesita ser cambiada, lo más recomendable sería cambiar el valor del estado para que vuelva a renderizarse con el nuevo valor. Solo deberíamos de usar las referencias cuando no tenemos otra forma de modificar dichos elementos.

Para crear una referencia usaremos el método `React.createRef()` que asignaremos a una propiedad de la clase dentro del constructor.

```
constructor(props) {  
  super(props);  
  this.miReferencia = React.createRef();  
}
```

Una vez hecho esto, tendremos que asociarla a un elemento de nuestro componente dándole esta propiedad como valor mediante el atributo **ref**.

```
<input type="text" ref={this.miReferencia} />
```

Una vez que tenemos una referencia asociada a uno de los elementos HTML de la vista, podemos acceder a través de la propiedad que tiene como valor la referencia, y a través de **current** podremos acceder a todos los atributos de dicho elemento HTML.

```
console.log(this.miReferencia.current.value);
```

21.1. Lab: Referencias

En este laboratorio vamos a crear los controles de reproducción de la etiqueta Audio utilizando las referencias de React. Podremos realizar las siguientes acciones:

- Reproducir el sonido
- Pausar el sonido
- Cambiar el volumen

Vamos a copiar la plantilla del proyecto que se ha realizado en el laboratorio **reactjs-proyecto-inicial-lab** y vamos a cambiarle el nombre del proyecto a **reactjs-refs-lab**.



Es posible que sea necesario volver a instalar las dependencias del proyecto para que funcione correctamente. Para ello solo tenemos que lanzar el comando **npm install** dentro de la carpeta del proyecto.

Ahora vamos a levantar el servidor de desarrollo con el comando:

```
$ npm start
```

Empezamos añadiendo en una carpeta **src/assets** el sonido que vamos a utilizar en el laboratorio, yo usaré **sonido-piolin.mp3**, pero podéis usar cualquier otro archivo de sonido que tengáis a mano.

Vamos a empezar por crear el código HTML dentro del App para añadir los tres elementos con los que controlaremos el audio.

/reactjs-refs-lab/src/components/App.jsx

```
import React, { Component } from 'react'

export default class App extends Component {
  constructor(props) {
    super(props);
  }

  render() {
    return (
      <div>
        <audio></audio>
        <button type="button">Play</button>
        <button type="button">Pause</button>
        <input type="range" min="0" max="100" />
      </div>
    )
  }
}
```

Ahora vamos a añadir las funciones y el estado que necesitaremos para manejar los controles. Necesitaremos en el estado el valor del volumen, y luego tres funciones, una por cada control que vamos a añadir.

/reactjs-refs-lab/src/components/App.jsx

```
import React, { Component } from 'react'

export default class App extends Component {
  constructor(props) {
    super(props);
    this.state = {
      volumen: 100
    }
    this.handlePlay = this.handlePlay.bind(this);
    this.handlePause = this.handlePause.bind(this);
    this.handleChangeVolume = this.handleChangeVolume.bind(this);
  }

  handlePlay() {

  }

  handlePause() {

  }

  handleChangeVolume() {

  }

  render() {
    return (
      <div>
        <audio></audio>
        <button type="button">Play</button>
        <button type="button">Pause</button>
        <input type="range" min="0" max="100" />
      </div>
    )
  }
}
```

El siguiente paso es crear la referencia y asignarla al atributo ref de la etiqueta **audio**.

/reactjs-refs-lab/src/components/App.jsx

```
import React, { Component } from 'react'

export default class App extends Component {
```

```

constructor(props) {
  super(props);
  this.state = {
    volumen: 100
  }
  this.handlePlay = this.handlePlay.bind(this);
  this.handlePause = this.handlePause.bind(this);
  this.handleChangeVolume = this.handleChangeVolume.bind(this);
  this.refAudio = React.createRef(new Audio());
}

handlePlay() {

}

handlePause() {

}

handleChangeVolume() {

}

render() {
  return (
    <div>
      <audio ref={this.refAudio}></audio>
      <button type="button">Play</button>
      <button type="button">Pause</button>
      <input type="range" min="0" max="100" />
    </div>
  )
}
}

```

También tenemos que añadir el **src** con el archivo de sonido. Para que Webpack genere los assets correctamente en el proceso de construcción de la aplicación, tenemos que importar el archivo (al igual que vimos con los archivos de estilos). El objeto que obtenemos al importar el sonido será el valor que le vamos a dar al atributo **src** del audio.

/reactjs-refs-lab/src/components/App.jsx

```

import React, { Component } from 'react'
import sonidoPiolin from '../assets/sonido-piolin.mp3';

export default class App extends Component {
  constructor(props) {
    super(props);
    this.state = {
      volumen: 100
    }
  }
}

```

```

    }
    this.handlePlay = this.handlePlay.bind(this);
    this.handlePause = this.handlePause.bind(this);
    this.handleChangeVolume = this.handleChangeVolume.bind(this);
    this.refAudio = React.createRef(new Audio());
  }

  handlePlay() {

  }

  handlePause() {

  }

  handleChangeVolume() {

  }

  render() {
    return (
      <div>
        <audio src={sonidoPiolin} ref={this.refAudio}></audio>
        <button type="button">Play</button>
        <button type="button">Pause</button>
        <input type="range" min="0" max="100" />
      </div>
    )
  }
}

```

Ahora ya podemos dedicarnos a añadir la lógica de los botones e inputs que habíamos añadido al principio.

Vamos a empezar por el botón de **Play**. Sobre dicho botón añadiremos el evento **click** para llamar a la función **handlePlay** en la que mediante la referencia que tenemos vamos a llamar a la función **play** de las etiquetas media.

/reactjs-refs-lab/src/components/App.jsx

```

import React, { Component } from 'react'
import sonidoPiolin from '../assets/sonido-piolin.mp3';

export default class App extends Component {
  constructor(props) {
    super(props);
    this.state = {
      volumen: 100
    }
    this.handlePlay = this.handlePlay.bind(this);
  }
}

```



```

    this.handlePause = this.handlePause.bind(this);
    this.handleChangeVolume = this.handleChangeVolume.bind(this);
    this.refAudio = React.createRef(new Audio());
  }

  handlePlay() {
    this.refAudio.current.play();
  }

  handlePause() {

  }

  handleChangeVolume(e) {

  }

  render() {
    return (
      <div>
        <audio src={sonidoPiolin} ref={this.refAudio}></audio>
        <button type="button" onClick={this.handlePlay}>Play</button>
        <button type="button">Pause</button>
        <input type="range" min="0" max="100" />
      </div>
    )
  }
}

```

El siguiente paso es hacer lo mismo con el botón de **Pause** llamando en este caso a la función **pause**.

/reactjs-refs-lab/src/components/App.jsx

```

import React, { Component } from 'react'
import sonidoPiolin from '../assets/sonido-piolin.mp3';

export default class App extends Component {
  constructor(props) {
    super(props);
    this.state = {
      volumen: 100
    }
    this.handlePlay = this.handlePlay.bind(this);
    this.handlePause = this.handlePause.bind(this);
    this.handleChangeVolume = this.handleChangeVolume.bind(this);
    this.refAudio = React.createRef(new Audio());
  }

  handlePlay() {

```

```

    this.refAudio.current.play();
  }

  handlePause() {
    this.refAudio.current.pause();
  }

  handleChangeVolume(e) {

  }

  render() {
    return (
      <div>
        <audio src={sonidoPiolin} ref={this.refAudio}></audio>
        <button type="button" onClick={this.handlePlay}>Play</button>
        <button type="button" onClick={this.handlePause}>Pause</button>
        <input type="range" min="0" max="100" />
      </div>
    )
  }
}

```

Con esto ya deberíamos de poder hacer que suene el sonido y pausarlo pulsando sobre esos dos botones.

Vamos a por el volumen. En el **input** con el que vamos a controlar el volumen del sonido le vamos a dar como valor aquel que tenemos guardado en el estado, y añadiremos el evento **change** para llamar a la función que controlará el volumen cada vez que se cambie el valor del input.

/reactjs-refs-lab/src/components/App.jsx

```

import React, { Component } from 'react'
import sonidoPiolin from '../assets/sonido-piolin.mp3';

export default class App extends Component {
  constructor(props) {
    super(props);
    this.state = {
      volumen: 100
    }
    this.handlePlay = this.handlePlay.bind(this);
    this.handlePause = this.handlePause.bind(this);
    this.handleChangeVolume = this.handleChangeVolume.bind(this);
    this.refAudio = React.createRef(new Audio());
  }

  handlePlay() {
    this.refAudio.current.play();
  }

  handlePause() {
    this.refAudio.current.pause();
  }
}

```

```

handleChangeVolume(e) {

}

render() {
  return (
    <div>
      <audio src={sonidoPiolin} ref={this.refAudio}></audio>
      <button type="button" onClick={this.handlePlay}>Play</button>
      <button type="button" onClick={this.handlePause}>Pause</button>
      <input type="range" min="0" max="100" value={this.state.volumen} onChange={this.handleChangeVolume} />
    </div>
  )
}

```

En la función de **handleChangeVolume** tenemos el evento que ha ocurrido del cual vamos a obtener el valor del input que setearemos como nuevo valor del estado.

Al mismo tiempo cambiaremos el valor del volumen del audio mediante la referencia. Para ello le asignamos al atributo **volume** un valor entre 0 y 1, por lo que el valor que hemos recibido del input tendremos que dividirlo entre 100 para asignarle un valor que se encuentre entre los parámetros que acepta.

/reactjs-refs-lab/src/components/App.jsx

```

import React, { Component } from 'react'
import sonidoPiolin from '../assets/sonido-piolin.mp3';

export default class App extends Component {
  constructor(props) {
    super(props);
    this.state = {
      volumen: 100
    }
    this.handlePlay = this.handlePlay.bind(this);
    this.handlePause = this.handlePause.bind(this);
    this.handleChangeVolume = this.handleChangeVolume.bind(this);
    this.refAudio = React.createRef(new Audio());
  }

  handlePlay() {
    this.refAudio.current.play();
  }

  handlePause() {
    this.refAudio.current.pause();
  }

  handleChangeVolume(e) {
    const volumen = e.target.value;
    this.refAudio.current.volume = volumen / 100;
    this.setState({
      volumen
    })
  }

  render() {

```

```
return (  
  <div>  
    <audio src={sonidoPiolin} ref={this.refAudio}></audio>  
    <button type="button" onClick={this.handlePlay}>Play</button>  
    <button type="button" onClick={this.handlePause}>Pause</button>  
    <input type="range" min="0" max="100" value={this.state.volumen} onChange={this.handleChangeVolume} />  
  </div>  
)  
}
```

Y con esto ya podemos utilizar nuestro panel de control para manejar los archivos media como audio y video que queramos.

Chapter 22. Propiedad children

A veces nos encontramos con algunos componentes en los que su contenido no es algo fijo, sino que podemos cambiarlo dependiendo de donde queramos utilizarlo como por ejemplo:

- Un carrousel de componentes (dentro podemos tener imagenes, cards...)
- Un componente colapsable (dentro podemos tener texto simple, listas...)

El contenido de estos componentes se les pasa entre las etiquetas de apertura y cierre de los componentes contenedores.

```
<MiBoton>Texto del botón</MiBoton>
```

El valor que tenemos entre las etiquetas de nuestros componentes le llegan a estos dentro de la propiedad **children**.

```
<button type="button">{props.children}</button>
```

22.1. Lab: Propiedad children

En este laboratorio vamos a ver como utilizar la propiedad children en un componente que se puede colapsar.

Vamos a copiar la plantilla del proyecto que se ha realizado en el laboratorio **reactjs-proyecto-inicial-lab** y vamos a cambiarle el nombre del proyecto a **reactjs-children-lab**.



Es posible que sea necesario volver a instalar las dependencias del proyecto para que funcione correctamente. Para ello solo tenemos que lanzar el comando **npm install** dentro de la carpeta del proyecto.

Ahora vamos a levantar el servidor de desarrollo con el comando:

```
$ npm start
```

Empezaremos creando el componente **Acordeon** que se va a componer de una cabecera que siempre se va a mostrar y el contenido que es aquello que queremos ocultar o expandir según pulsemos sobre la cabecera.

/reactjs-children-lab/src/components/Acordeon.jsx

```
const Acordeon = () => {  
  return (  
    <div>  
      <div>  
        <h3>Aquí va el título</h3>  
      </div>  
      <div>  
        Aquí va el contenido  
      </div>  
    </div>  
  )  
}  
  
export default Acordeon
```

Tanto el título como el contenido del acordeón vendrán desde los componentes superiores, por tanto los vamos a recibir como propiedades.

Como el contenido no tiene porque mostrarse siempre con los mismos elementos de HTML, sino que una vez podemos querer mostrar una lista, mientras que otra solo un párrafo de información, vamos a pintar aquí aquello que pasen entre las etiquetas de apertura y cierre del componente. Es decir, usaremos la propiedad children para indicar cual es el contenido del Acordeon.

Otra propiedad que vamos a recibir es la que indica si el contenido tiene que aparecer expandido o no al crear el componente.

```
const Acordeon = ({ children, titulo, cerrado = true }) => {
  return (
    <div>
      <div>
        <h3>{titulo}</h3>
      </div>
      <div>
        {children}
      </div>
    </div>
  )
}

export default Acordeon
```

Ahora vamos a añadir un estado para indicarle al componente si tiene que estar colapsado o expandido.

```
import { useState } from 'react';

const Acordeon = ({ children, titulo, cerrado = true }) => {
  const [estaCerrado, setEstaCerrado] = useState(null);

  return (
    <div>
      <div>
        <h3>{titulo}</h3>
      </div>
      <div>
        {children}
      </div>
    </div>
  )
}

export default Acordeon
```

Como podemos indicarle desde la propiedad **cerrado** el estado inicial y la propiedad puede cambiar desde el exterior, vamos a utilizar el hook de **useEffect** para inicializar y sincronizar el estado con esa propiedad.

```
import { useState, useEffect } from 'react';

const Acordeon = ({ children, titulo, cerrado = true }) => {
```

```

const [estaCerrado, setEstaCerrado] = useState(null);

useEffect(() => {
  setEstaCerrado(cerrado);
}, [cerrado]);

return (
  <div>
    <div>
      <h3>{titulo}</h3>
    </div>
    <div>
      {children}
    </div>
  </div>
)
}

export default Acordeon

```

También tenemos que añadir el evento **click** sobre la cabecera para poder expandir y colapsar el contenido.

/reactjs-children-lab/src/components/Acordeon.jsx

```

import { useState, useEffect } from 'react';

const Acordeon = ({ children, titulo, cerrado = true }) => {
  const [estaCerrado, setEstaCerrado] = useState(null);

  useEffect(() => {
    setEstaCerrado(cerrado);
  }, [cerrado]);

  return (
    <div>
      <div onClick={() => setEstaCerrado(!estaCerrado)}>
        <h3>{titulo}</h3>
      </div>
      <div>
        {children}
      </div>
    </div>
  )
}

export default Acordeon

```

Para que se vea un poco mejor el Acordeon vamos a crear un archivo de estilo en el que aprovecharemos para añadir las clases de **abierto** y **cerrado** que se encargan de modificar la

altura del contenido para que se colapse o se expanda.

/reactjs-children-lab/src/style.css

```
.acordeon {
  border: 1px solid black;
  border-radius: 5px;
}

.acordeon-heading {
  text-align: center;
  border-bottom: 2px solid black;
  cursor: pointer;
}

.acordeon-content {
  overflow: hidden;
}

.acordeon-content.cerrado {
  height: 0;
}

.acordeon-content.abierto {
  height: auto;
  padding: 20px;
}
```

Ahora importamos el archivo de estilos y le añadimos a nuestro componente las clases anteriores en sus elementos correspondientes.

/reactjs-children-lab/src/components/Acordeon.jsx

```
import { useState, useEffect } from 'react';
import './Acordeon.css';

const Acordeon = ({ children, titulo, cerrado = true }) => {
  const [estaCerrado, setEstaCerrado] = useState(null);

  useEffect(() => {
    setEstaCerrado(cerrado);
  }, [cerrado]);

  return (
    <div className="acordeon">
      <div className="acordeon-heading" onClick={() => setEstaCerrado(!estaCerrado)}>
        <h3>{titulo}</h3>
      </div>
      <div className={'acordeon-content ' + (estaCerrado ? 'cerrado' : 'abierto')}>
        {children}
      </div>
    </div>
  );
}
```

```

    </div>
  )
}

export default Acordeon

```

Con esto ya tenemos nuestro Acordeon finalizado.

Vamos a utilizarlo en el componente App para mostrar varias cosas que se puedan colapsar.

/reactjs-children-lab/src/components/App.jsx

```

import Acordeon from './Acordeon';

const App = () => {
  return (
    <div>
      <Acordeon titulo="Una lista de productos">
        <ul>
          <li>Albahaca</li>
          <li>Queso parmesano</li>
          <li>Pechuga de pollo</li>
          <li>Tomates</li>
        </ul>
      </Acordeon>
      <Acordeon titulo="Librería JS" cerrado={false}>
        <div>
          <h4>¿Que es React?</h4>
          
          <p>React es una librería de JS que permite pintar interfaces de usuario...</p>
        </div>
      </Acordeon>
    </div>
  )
}

export default App

```

Chapter 23. Fragments

En React no podemos devolver dos elementos HTML o componentes si no están envueltos por otro elemento.

```
import { useState } from 'react';

const App = () => {
  const [productos, setProductos] = useState(['Tripode para teléfono', 'Gafas de sol cuadradas', 'Sombrero de pescador', 'Tira LED RGB de 5m']);

  const listaProductos = productos.map((producto, pos) => (
    // ERROR
    <p>{producto}</p>
    {pos !== productos.length - 1 && <hr/>}
  ))

  return (
    <div>
      {listaProductos}
    </div>
  )
}

export default App
```

El problema de esto es que el código puede quedar sucio si nos encontramos elementos envueltos por otros que no necesitamos. Al final acabamos con muchas etiquetas **div** innecesarias.

Los **Fragments** son la solución a este problema que se añaden en la versión 16 de React.

Lo que nos permiten hacer es envolver las etiquetas con una etiqueta especial (**<React.Fragment>**), y que a la hora de renderizar estos elementos no se mostrará en el DOM.

```
import { useState, Fragment } from 'react';

const App = () => {
  const [productos, setProductos] = useState(['Tripode para teléfono', 'Gafas de sol cuadradas', 'Sombrero de pescador', 'Tira LED RGB de 5m']);

  const listaProductos = productos.map((producto, pos) => (
    <Fragment key={pos}>
      <p>{producto}</p>
      {pos !== productos.length - 1 && <hr/>}
    </Fragment>
  ))

  return (
    <div>
      {listaProductos}
    </div>
  )
}

export default App
```

En lugar de utilizar la etiqueta de React **Fragment**, podemos utilizar una etiqueta vacía siempre que esta transformación esté contemplada por nuestro transpilador de JSX a JS, es decir, Babel.

```
import { useState } from 'react';

const App = () => {
  const [productos, setProductos] = useState(['Tripode para teléfono', 'Gafas de sol cuadradas', 'Sombrero de pescador', 'Tira LED RGB de 5m']);

  const listaProductos = productos.map((producto, pos) => (
    <>
      <p>{producto}</p>
      {pos !== productos.length - 1 && <hr/>}
    </>
  ))

  return (
    <div>
      {listaProductos}
    </div>
  )
}

export default App
```

En la configuración de Babel se le indica con la opción de **"runtime": "automatic"**.

/.babelrc

```
[
  "@babel/preset-react",
  {
    "runtime": "automatic"
  }
]
```

Chapter 24. Context API

Cuando las aplicaciones van creciendo se vuelve muy difícil enviar datos desde los componentes superiores a los componentes inferiores, además datos que son globales a toda la aplicación tendríamos que pasarlos como propiedades por todo el árbol de componentes incluso por muchos de ellos que seguramente no necesiten dichos datos.

Esto se puede solucionar fácilmente con alguna librería de gestión de estados como Redux, pero en React implementaron algo que nos va a permitir compartir estos datos solo entre aquellos componentes que los necesiten. Esto es el **Context API**.

Primero hay que crear un contexto que almacene los datos a compartir con otros componentes usando la función **React.createContext**.

```
export const MiContexto = React.createContext('Aquí van los datos');
```

Después tenemos dos componentes:

- El **Provider** que es el que se va a encargar de proveer los datos. Este componente tiene que envolver los componentes por los cuales se encuentre alguno que necesite los datos guardados en el contexto.

```
<MiContexto.Provider>  
  <ComponenteA />  
</MiContexto.Provider>
```

- El **Consumer** que es el componente que recibe los datos y se los pasa al componente que los quiere utilizar. Recibe los datos como children en una función.

```
<MiContexto.Consumer>  
  {datos => (  
    <p>Datos: {datos}</p>  
  )}  
</MiContexto.Consumer>
```

Algunos ejemplos para los que podríamos utilizar esta funcionalidad es para el tema de color de la aplicación, el idioma en el que queremos mostrarla, saber si un usuario está logueado o no...

Podemos crear varios contextos o juntar datos dentro de un mismo contexto, eso depende de como queramos organizar nuestro código.

Si los datos que se van a compartir a través de la Context API son dinámicos, es decir, que pueden cambiar con las acciones de los usuarios, entonces tendremos que pasar estos datos como valor de la propiedad **value** del provider, de tal forma que cada vez que estos sufran un cambio, los consumidores puedan recibir la actualización automáticamente.

```
<MiContexto.Provider value={datos}>  
  <ComponenteA />  
</MiContexto.Provider>
```

24.1. Lab: Context API

En este laboratorio vamos a ver como podríamos hacer que nuestra aplicación cambie entre el dark mode y el light mode utilizando la Context API para compartir entre los componentes el tema que se tiene que aplicar.

Vamos a copiar la plantilla del proyecto que se ha realizado en el laboratorio **reactjs-proyecto-inicial-lab** y vamos a cambiarle el nombre del proyecto a **reactjs-context-api-lab**.



Es posible que sea necesario volver a instalar las dependencias del proyecto para que funcione correctamente. Para ello solo tenemos que lanzar el comando **npm install** dentro de la carpeta del proyecto.

Ahora vamos a levantar el servidor de desarrollo con el comando:

```
$ npm start
```

Empezaremos añadiendo en el componente raíz de la aplicación el estado y un botón para cambiar entre ambos modos.

/reactjs-context-api-lab/src/components/App.jsx

```
import { useState } from 'react';

const App = () => {
  const [darkMode, setDarkMode] = useState(true);

  return (
    <div>
      <button type="button" onClick={() => setDarkMode(!darkMode)}>Activado {darkMode ? '☑' : '☐'}</button>
    </div>
  )
}

export default App
```

Ahora vamos a crear un objeto de estilos que se aplicarán dependiendo de la opción seleccionada.

/reactjs-context-api-lab/src/components/App.jsx

```
import { useState } from 'react';

const App = () => {
  const [darkMode, setDarkMode] = useState(true);
  const stylesThemeMode = darkMode ? {
    backgroundColor: 'black',
    color: 'white'
  } : {
    backgroundColor: 'white',
    color: 'black'
  }
}
```

```

return (
  <div>
    <button type="button" onClick={() => setDarkMode(!darkMode)}>Activado {darkMode ? '☑' : '☐'}</button>
  </div>
)
}

export default App

```

Una vez tenemos la lógica necesaria para cambiar entre el dark mode y el light mode, vamos a crear el contexto **ThemeContext** con el que pasaremos el objeto con los estilos a los componentes que lo necesiten.

Para ello importamos **React** y utilizamos la función de **createContext**. Para luego poder utilizar un consumidor en otros componentes, tenemos que exportar el contexto.

/reactjs-context-api-lab/src/components/App.jsx

```

import React, { useState } from 'react';

export const ThemeContext = React.createContext();

const App = () => {
  const [darkMode, setDarkMode] = useState(true);
  const stylesThemeMode = darkMode ? {
    backgroundColor: 'black',
    color: 'white'
  } : {
    backgroundColor: 'white',
    color: 'black'
  }

  return (
    <div>
      <button type="button" onClick={() => setDarkMode(!darkMode)}>Activado {darkMode ? '☑' : '☐'}</button>
    </div>
  )
}

export default App

```

Ahora vamos a crear el componente **CmpConsumidor** que de momento vamos a dejar de la siguiente forma.

/reactjs-context-api-lab/src/components/CmpConsumidor.jsx

```

import { ThemeContext } from './App';

const CmpConsumidor = () => {
  return (
    <div>
      <p>Este componente consumirá el valor del context</p>
    </div>
  )
}

```



```
}

export default CmpConsumidor;
```

Vamos a importar el nuevo componente en **App** y lo envolvemos con el **Provider** del contexto que hemos creado, pasándole el objeto de estilos mediante el provider.

/reactjs-context-api-lab/src/components/App.jsx

```
import React, { useState } from 'react';
import CmpConsumidor from './CmpConsumidor';

export const ThemeContext = React.createContext();

const App = () => {
  const [darkMode, setDarkMode] = useState(true);
  const stylesThemeMode = darkMode ? {
    backgroundColor: 'black',
    color: 'white'
  } : {
    backgroundColor: 'white',
    color: 'black'
  }

  return (
    <div>
      <button type="button" onClick={() => setDarkMode(!darkMode)}>Activado {darkMode ? '☑' : '☐'}</button>

      <ThemeContext.Provider value={stylesThemeMode}>
        <CmpConsumidor />
      </ThemeContext.Provider>
    </div>
  )
}

export default App
```

Al envolver con el Provider el componente CmpConsumidor, tanto este como cualquiera que se renderice dentro de él (sus descendientes) podrán pedir el valor del contexto.

Por último, vamos a añadir el **Consumer** dentro del componente **CmpConsumidor** para recibir el objeto con los estilos y así aplicarlos sobre las etiquetas de HTML que va a pintar.

Como **children** del componente Consumer pasaremos una función que va a devolver los elementos que queremos pintar. En esta función recibimos los estilos como parámetro y ya podríamos utilizarlos.

/reactjs-context-api-lab/src/components/CmpConsumidor.jsx

```
import { ThemeContext } from './App';

const CmpConsumidor = () => {
  return (
    <ThemeContext.Consumer>
```

```
    {(styles) => (  
      <div style={styles}>  
        <p>Este componente consumirá el valor del context</p>  
      </div>  
    )}  
  </ThemeContext.Consumer>  
)  
}  
  
export default CmpConsumidor;
```

Chapter 25. Higher Order Components (HOC)

Algunos componentes pueden tener una funcionalidad repetida, y los **Higher Order Components** nos permiten evitar tener que repetirla en todos esos componentes creando un componente que les añadirá dicha funcionalidad.

Son **funciones** que reciben un componente como parámetro, que es al que se le va a añadir la funcionalidad que queremos reutilizar, y que tiene que devolver un nuevo componente (el que le pasamos como parámetro envuelto en otro que añade la funcionalidad) que es el que utilizaremos.



Es una buena práctica hacer que el nombre de los higher order components empiecen con **with**.

```
const withFuncX = WrappedCmp => {  
  return class CmpConFuncX extends Component {  
    render() {  
      return (  
        <div>  
          // ...  
        </div>  
      );  
    }  
  }  
}  
  
export default withFuncX;
```

A la hora de utilizarlo tenemos que llamar a la función pasándole un componente y nos devolverá el componente a pintar.

```
const ComponenteAPintar = withFuncX(UnComponenteCualquiera);  
  
return (  
  <ComponenteAPintar />  
);
```

25.1. Lab: Añadir estilos con un HOC

En este laboratorio vamos a ver como crear un higher order component para añadir unos estilos sobre los componentes y darles así el efecto de sombreado cuando pasamos el ratón por encima de ellos.

Vamos a copiar la plantilla del proyecto que se ha realizado en el laboratorio **reactjs-proyecto-inicial-lab** y vamos a cambiarle el nombre del proyecto a **reactjs-higher-order-components-hover-lab**.



Es posible que sea necesario volver a instalar las dependencias del proyecto para que funcione correctamente. Para ello solo tenemos que lanzar el comando **npm install** dentro de la carpeta del proyecto.

Ahora vamos a levantar el servidor de desarrollo con el comando:

```
$ npm start
```

Vamos a empezar por crear un componente **Boton** dentro de la carpeta de **components** al que después le aplicaremos los estilos con el HOC.

Este componente recibirá en las props el texto y el método que se ejecutará cuando se pulse sobre el. Además, vamos a añadirle unos estilos en línea con JavaScript para que quede más curioso el botón.

/reactjs-higher-order-components-hover-lab/src/components/Boton.jsx

```
const styles = {
  padding: '8px 12px',
  border: '1px solid black',
  borderRadius: '5px',
  backgroundColor: 'white'
}

const Boton = ({children, handleClick}) => {
  return (
    <button
      type="button"
      onClick={handleClick}
      style={styles}
    >
      {children}
    </button>
  )
}

export default Boton;
```

Una vez tenemos el botón ya podemos añadirlo en el componente **App** para mostrarlo.

/reactjs-higher-order-components-hover-lab/src/components/App.jsx

```
import Boton from './Boton';

const App = () => {
  return (
    <div>
      <Boton handleClick={() => alert('Hola mundo!!!')}>Saludar al mundo</Boton>
    </div>
  )
}

export default App
```

Ahora vamos a crear el HOC **withHover** en un archivo de JavaScript en una carpeta **src/hoc**.

Como ya sabemos, un HOC es una función a la que se le va a pasar el componente que queremos envolver y añadirle una funcionalidad que no tiene. Por tanto empezamos creando esta función y haciendo que esta devuelva el nuevo componente que queremos generar.

/reactjs-higher-order-components-hover-lab/src/hoc/withHover.js

```
const withHover = (WrappedCmp) => {
  return (props) => {
    return ()
  }
}

export default withHover;
```

Ahora dentro del **return** vamos a poner el componente que recibimos como parámetro y lo envolveremos con una etiqueta **div** a la que le vamos a aplicar unos estilos.

/reactjs-higher-order-components-hover-lab/src/hoc/withHover.js

```
const withHover = (WrappedCmp) => {
  return (props) => {
    return (
      <div>
        <WrappedCmp />
      </div>
    )
  }
}

export default withHover;
```

Estos estilos dependerán de si el ratón está encima del botón o si está fuera de este. Por tanto, vamos a necesitar un estado para controlar los estilos a aplicar.

Importamos el hook de **useState** y creamos un estado del tipo booleano para indicar si el ratón está encima o no.

/reactjs-higher-order-components-hover-lab/src/hoc/withHover.js

```
import { useState } from 'react';

const withHover = (WrappedCmp) => {
  return (props) => {
    const [isMouseOn, setIsMouseOn] = useState(false);

    return (
      <div>
        <WrappedCmp />
      </div>
    )
  }
}

export default withHover;
```

Ahora vamos a añadir dos eventos **onMouseEnter** y **onMouseLeave** para cambiar este estado de valor.

/reactjs-higher-order-components-hover-lab/src/hoc/withHover.js

```
import { useState } from 'react';

const withHover = (WrappedCmp) => {
  return (props) => {
    const [isMouseOn, setIsMouseOn] = useState(false);

    return (
      <div
        onMouseEnter={() => setIsMouseOn(true)}
        onMouseLeave={() => setIsMouseOn(false)}
      >
        <WrappedCmp />
      </div>
    )
  }
}

export default withHover;
```

Para ir terminando con el HOC, tenemos que crear los estilos, que es un objeto donde pondremos la propiedad **opacity** a 1 cuando el ratón está fuera del botón, y a 0.4 cuando está sobre el. También

vamos a hacer que se ajuste al contenido de la etiqueta con la propiedad **width: 'max-content'**.

Solo tenemos que asignarle el objeto con los estilos a la propiedad **style** del **div**.

/reactjs-higher-order-components-hover-lab/src/hoc/withHover.js

```
import { useState } from 'react';

const withHover = (WrappedCmp) => {
  return (props) => {
    const [isMouseOn, setIsMouseOn] = useState(false);

    const styles = {
      opacity: isMouseOn ? '0.4' : '1',
      width: 'max-content'
    }

    return (
      <div
        style={styles}
        onMouseEnter={() => setIsMouseOn(true)}
        onMouseLeave={() => setIsMouseOn(false)}
      >
        <WrappedCmp />
      </div>
    )
  }
}

export default withHover;
```

Ahora que ya tenemos el HOC creado, lo importamos en el componente App, donde vamos a crear un componente **BotonWithHover** que se obtiene de envolver el componente Boton con el HOC.

/reactjs-higher-order-components-hover-lab/src/components/App.jsx

```
import withHover from '../hoc/withHover';
import Boton from './Boton';

const App = () => {
  const BotonWithHover = withHover(Boton);

  return (
    <div>
      <Boton handleClick={() => alert('Hola mundo!!!')}>Saludar al mundo</Boton>
    </div>
  )
}

export default App
```

Una vez que tenemos el nuevo componente, ya podemos pintarlo en nuestra aplicación pasándole las propiedades que necesita.

/reactjs-higher-order-components-hover-lab/src/components/App.jsx

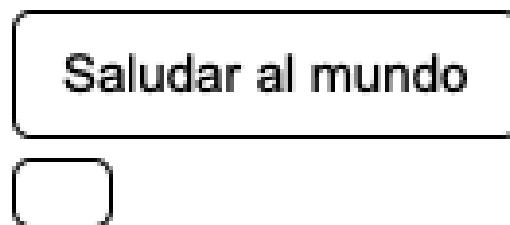
```
import withHover from '../hoc/withHover';
import Boton from './Boton';

const App = () => {
  const BotonWithHover = withHover(Boton);

  return (
    <div>
      <Boton handleClick={() => alert('Hola mundo!!!')}>Saludar al mundo</Boton>
      <BotonWithHover handleClick={() => alert('Bienvenid@s...')}>Dar bienvenida</BotonWithHover>
    </div>
  )
}

export default App
```

Si abrimos la aplicación en el navegador, <http://localhost:8080> veremos algo como lo siguiente.



En la imagen podemos ver que el botón sobre el que hemos usado el HOC no muestra el texto que le pasamos y si pulsamos sobre el tampoco ejecuta la función correspondiente. Esto se debe a que las propiedades le están llegando al componente que estamos creando con el HOC, pero este no se las está pasando al componente Boton.

Para hacérselas llegar al componente Boton y que de esta forma se muestre el texto del botón y se ejecute la función al pulsar sobre el, le vamos a pasar al **WrappedCmp** todas las propiedades.

/reactjs-higher-order-components-hover-lab/src/hoc/withHover.js

```
import { useState } from 'react';

const withHover = (WrappedCmp) => {
  return (props) => {
```



```

const [isMouseOn, setIsMouseOn] = useState(false);

const styles = {
  opacity: isMouseOn ? '0.4' : '1',
  width: 'max-content'
}

return (
  <div
    style={styles}
    onMouseEnter={() => setIsMouseOn(true)}
    onMouseLeave={() => setIsMouseOn(false)}
  >
    <WrappedCmp {...props} />
  </div>
)
}
}

export default withHover;

```

Ahora ya debería de verse todo correctamente, y si pasamos el ratón por encima del segundo botón debería de aplicarse los estilos que añade sobre el nuestro HOC.

Saludar al mundo

Dar bienvenida

25.2. Lab: Cargar datos con un HOC

En este laboratorio vamos a ver como crear un higher order component que se encargará de pedir unos datos para inyectarlos en un componente, y mostrar un spinner mientras se piden.

Vamos a copiar la plantilla del proyecto que se ha realizado en el laboratorio **reactjs-proyecto-inicial-lab** y vamos a cambiarle el nombre del proyecto a **reactjs-higher-order-components-cargar-datos-lab**.



Es posible que sea necesario volver a instalar las dependencias del proyecto para que funcione correctamente. Para ello solo tenemos que lanzar el comando **npm install** dentro de la carpeta del proyecto.

Ahora vamos a levantar el servidor de desarrollo con el comando:

```
$ npm start
```

Empezaremos creando la siguiente estructura de archivos que después iremos rellenando:

- src/hoc/withData.js
- src/components/InfoUsuario.jsx

Vamos a empezar a rellenar el hoc **withData** con la función que recibe el componente y vamos a hacer que retorne el nuevo componente que renderiza el **WrappedCmp** con las propiedades que se le pasan.

/reactjs-higher-order-components-cargar-datos-lab/src/hoc/withData.js

```
import { Component } from 'react';

const withData = (WrappedCmp) => {
  return class extends Component {
    render() {
      return (
        <>
          <WrappedCmp {...this.props} />
        </>
      )
    }
  }
}

export default withData;
```

La idea es que desde este hoc realicemos una petición GET a una URL para obtener los datos que se le van a pasar a nuestro componente. Entonces vamos a hacer que nuestro hoc reciba también la URL.

Una vez que tenemos la URL, tenemos que hacer la petición, pero los datos recibidos tendremos que almacenarlos en algún sitio para poder pasárselos al componente. Así que crearemos un estado **data** que inicializamos a null hasta que los datos se hayan obtenido.

/reactjs-higher-order-components-cargar-datos-lab/src/hoc/withData.js

```
import { Component } from 'react';

const withData = (WrappedCmp, url) => {
  return class extends Component {
    constructor(props) {
      super(props);
      this.state = {
        data: null
      }
    }

    render() {
      return (
        <>
          <WrappedCmp {...this.props} />
        </>
      )
    }
  }
}

export default withData;
```

Ahora vamos a realizar la petición GET usando el método fetch en el método del ciclo de vida **componentDidMount**, y una vez que hayamos obtenido los datos cambiaremos el estado.

/reactjs-higher-order-components-cargar-datos-lab/src/hoc/withData.js

```
import { Component } from 'react';

const withData = (WrappedCmp, url) => {
  return class extends Component {
    constructor(props) {
      super(props);
      this.state = {
        data: null
      }
    }

    async componentDidMount() {
      const response = await fetch(url);
      const datos = await response.json();
      this.setState({
        data: datos
      })
    }
  }
}
```

```

    })
  }

  render() {
    return (
      <>
        <WrappedCmp {...this.props} />
      </>
    )
  }
}

export default withData;

```

Una vez que tenemos los datos ya se los podemos pasar a nuestro componente **WrappedCmp** de la misma forma en que le pasamos cualquier valor, en forma de propiedad.

/reactjs-higher-order-components-cargar-datos-lab/src/hoc/withData.js

```

import { Component } from 'react';

const withData = (WrappedCmp, url) => {
  return class extends Component {
    constructor(props) {
      super(props);
      this.state = {
        data: null
      }
    }

    async componentDidMount() {
      const response = await fetch(url);
      const datos = await response.json();
      this.setState({
        data: datos
      })
    }

    render() {
      return (
        <>
          <WrappedCmp {...this.props} data={this.state.data} />
        </>
      )
    }
  }
}

export default withData;

```

Ahora que tenemos el hoc, podemos mejorarlo un poco más para hacer que no se pinte el componente que espera los datos hasta que se hayan obtenido estos. Mientras se obtienen vamos a mostrar un mensaje de **loading**.

Para hacer esto vamos a añadir en el estado otro atributo **cargandoDatos** que vamos a inicializar a **true**, y mientras este valor sea true, se mostrará el mensaje de loading.

/reactjs-higher-order-components-cargar-datos-lab/src/hoc/withData.js

```
import { Component } from 'react';

const withData = (WrappedCmp, url) => {
  return class extends Component {
    constructor(props) {
      super(props);
      this.state = {
        data: null,
        cargandoDatos: true
      }
    }

    async componentDidMount() {
      const response = await fetch(url);
      const datos = await response.json();
      this.setState({
        data: datos
      })
    }

    render() {
      return (
        <>
          {this.state.cargandoDatos ? <p>Loading...</p> : <WrappedCmp {...this.props} data={this.state.data} />}
        </>
      )
    }
  }
}

export default withData;
```

De momento se queda siempre mostrando el mensaje de loading, y es porque cuando obtenemos los datos tenemos que cambiar el valor de **cargandoDatos** a false. Vamos a poner el cambio de estado dentro de un setTimeout para ver que el mensaje se muestra.

/reactjs-higher-order-components-cargar-datos-lab/src/hoc/withData.js

```
import { Component } from 'react';

const withData = (WrappedCmp, url) => {
  return class extends Component {
    constructor(props) {
      super(props);
      this.state = {
        data: null,
        cargandoDatos: true
      }
    }
  }
}
```

```

    async componentDidMount() {
      const response = await fetch(url);
      const datos = await response.json();
      setTimeout(() => {
        this.setState({
          data: datos,
          cargandoDatos: false
        })
      }, 1400)
    }

    render() {
      return (
        <>
          {this.state.cargandoDatos ? <p>Loading...</p> : <WrappedCmp {...this.props} data={this.state.data} />}
        </>
      )
    }
  }
}

export default withData;

```

Por último para dejar este componente más completo, vamos a hacer que el loading se le pueda configurar mandandoselo como parámetro del hoc de tal forma que podamos mostrar mensajes o spinners que se adapten mejor a nuestra aplicación.

/reactjs-higher-order-components-cargar-datos-lab/src/hoc/withData.js

```

import { Component } from 'react';

const withData = (WrappedCmp, url, Loader = <p>Loading...</p>) => {
  return class extends Component {
    constructor(props) {
      super(props);
      this.state = {
        data: null,
        cargandoDatos: true
      }
    }

    async componentDidMount() {
      const response = await fetch(url);
      const datos = await response.json();
      setTimeout(() => {
        this.setState({
          data: datos,
          cargandoDatos: false
        })
      }, 1400)
    }

    render() {
      return (
        <>
          {this.state.cargandoDatos ? Loader : <WrappedCmp {...this.props} data={this.state.data} />}
        </>
      )
    }
  }
}

```

```

    )
  }
}
}

export default withData;

```

Ahora que tenemos el hoc completo, vamos a crear el componente que recibirá los datos y los pintará.

/reactjs-higher-order-components-cargar-datos-lab/src/components/InfoUsuario.jsx

```

const InfoUsuario = ({ data }) => {
  const { name, email, picture } = data.results[0];
  return (
    <div>
      <h2>{name.first} {name.last}</h2>
      <img src={picture.medium} alt={`Foto de ${name.first}`} />
      <p>✉: {email}</p>
    </div>
  )
}

export default InfoUsuario

```

Y ahora en el componente **App** vamos a utilizar nuestro hoc **withData** para inyectar en el componente **InfoUsuario** los datos que obtenemos de la API de <https://randomuser.me/api/>.

/reactjs-higher-order-components-cargar-datos-lab/src/components/App.jsx

```

import withData from "../hoc/withData"
import InfoUsuario from './InfoUsuario';

const App = () => {
  const InfoUsuarioWithData = withData(InfoUsuario, 'https://randomuser.me/api/');

  return (
    <div>
      <InfoUsuarioWithData />
    </div>
  )
}

export default App

```

Con esto ya deberíamos de ver la información aleatoria de un usuario con el loader mientras se cargan los datos.

Para hacer un poco más molón el ejemplo, vamos a utilizar algún spinner más currado. Para ello utilizaremos la dependencia de **spinners-react** que podemos encontrar en

<https://adexin.github.io/spinners/>.

Instalamos la dependencia en nuestro proyecto lanzando el comando:

```
$ npm install --save spinners-react
```

Una vez instalada, buscamos el spinner que más nos guste y lo importamos dentro de nuestro componente principal, y se lo vamos a pasar a nuestro HOC.

/reactjs-higher-order-components-cargar-datos-lab/src/components/App.jsx

```
import withData from "../hoc/withData"
import InfoUsuario from './InfoUsuario';

const App = () => {
  const InfoUsuarioWithData = withData(
    InfoUsuario,
    'https://randomuser.me/api/',
    <SpinnerDotted size={50} thickness={100} speed={100} color="#36ad47" />
  );

  return (
    <div>
      <InfoUsuarioWithData />
    </div>
  )
}

export default App
```


Chapter 26. Suspense (lazy loading)

En aplicaciones que son muy grandes, descargarnos toda la aplicación la primera vez que se entra puede tardar más tiempo del que debería tardar. Para solucionar este problema tenemos la **carga perezosa** o **lazy loading**.

El lazy loading nos ayuda a decrementar el tiempo de arranque de la aplicación. La primera vez que entramos en la aplicación, no necesitamos cargar todo. Solo cargaremos aquello que el usuario espera ver.

El resto de la aplicación, aquellas partes en las que no se suele entrar solo se cargarán una vez que el usuario entre en ellas. De esta forma nuestra aplicación se va a ir descargando a trozos según el usuario los vaya necesitando evitándole esperas superiores a las necesarias.

En React podemos realizar el lazy loading con **React.Suspense** y **React.lazy**.

El primero es un componente que envuelve a los componentes que todavía no se han renderizado porque se están descargando, y con el que haremos que se muestre un componente mientras se descargan y renderizan.

El segundo es una función que se encarga de cargar de forma diferida el componente que mostraremos dentro de **Suspense**.

```
const ComponentePerezoso = React.lazy(() => import('./ComponentePerezoso'));

function MiComponente() {
  return (
    <React.Suspense fallback={<p>Loading...</p>}>
      <div>
        <ComponentePerezoso />
      </div>
    </React.Suspense>
  );
}
```

26.1. Lab: Suspense

En este laboratorio vamos a ver como cargar un componente únicamente cuando se pide renderizarlo y no en el primer renderizado con el resto de la aplicación como venimos haciendo hasta ahora.

Vamos a copiar la plantilla del proyecto que se ha realizado en el laboratorio **reactjs-proyecto-inicial-lab** y vamos a cambiarle el nombre del proyecto a **reactjs-suspense-lab**.



Es posible que sea necesario volver a instalar las dependencias del proyecto para que funcione correctamente. Para ello solo tenemos que lanzar el comando **npm install** dentro de la carpeta del proyecto.

Ahora vamos a levantar el servidor de desarrollo con el comando:

```
$ npm start
```

Empezaremos creando los dos componentes que mostraremos en nuestra aplicación.

/reactjs-suspense-lab/src/components/Inicio.jsx

```
const Inicio = () => {  
  return (  
    <div>  
      <h1>Inicio</h1>  
    </div>  
  )  
}  
  
export default Inicio
```

/reactjs-suspense-lab/src/components/Admin.jsx

```
const Admin = () => {  
  return (  
    <div>  
      <h1>Panel de administración</h1>  
    </div>  
  )  
}  
  
export default Admin
```

Ahora vamos a ir al componente raíz, el App, para mostrar un componente u otro dependiendo de una condición.

```
import { useState } from 'react';
import Inicio from './Inicio';
import Admin from './Admin';

const App = () => {
  const [esAdmin, setEsAdmin] = useState(false);

  return (
    <div>
      <button type="button" onClick={() => setEsAdmin(!esAdmin)}>Toggle admin</button>
      {esAdmin ? <Admin /> : <Inicio />}
    </div>
  )
}

export default App
```

Tal cual está el código ahora mismo todo el código se va a descargar la primera vez que entremos a la aplicación. Ahora toca modificar este componente para hacer que el componente **Admin** solo se descargue cuando se vaya a mostrar, por lo que vamos a importar **Suspense** y **lazy** que utilizaremos para ello.

```
import { Suspense, lazy, useState } from 'react';
import Inicio from './Inicio';
import Admin from './Admin';

const App = () => {
  const [esAdmin, setEsAdmin] = useState(false);

  return (
    <div>
      <button type="button" onClick={() => setEsAdmin(!esAdmin)}>Toggle admin</button>
      {esAdmin ? <Admin /> : <Inicio />}
    </div>
  )
}

export default App
```

El siguiente paso es envolver el componente de Admin entre las etiquetas de **Suspense** y añadir sobre esta la propiedad **fallback** con el componente o elemento a mostrar mientras se descarga el componente de forma diferida.

Dentro del componente Suspense meteremos el componente de Admin.

```
import { Suspense, lazy, useState } from 'react';
import Inicio from './Inicio';
import Admin from './Admin';

const App = () => {
  const [esAdmin, setEsAdmin] = useState(false);

  return (
    <div>
      <button type="button" onClick={() => setEsAdmin(!esAdmin)}>Toggle admin</button>
      {
        esAdmin
        ?
        <Suspense fallback={<p>Loading...</p>}>
          <Admin />
        </Suspense>
        :
        <Inicio />
      }
    </div>
  )
}

export default App
```

Por último, usaremos la función **lazy** para importar dentro de ella el componente de **Admin** para que solo se descargue su código cuando se ejecute la función que le pasamos como parámetro.

```
import { Suspense, lazy, useState } from 'react';
import Inicio from './Inicio';

const Admin = lazy(() => import('./Admin'));

const App = () => {
  const [esAdmin, setEsAdmin] = useState(false);

  return (
    <div>
      <button type="button" onClick={() => setEsAdmin(!esAdmin)}>Toggle admin</button>
      {
        esAdmin
        ?
        <Suspense fallback={<p>Loading...</p>}>
          <Admin />
        </Suspense>
        :
      }
    </div>
  )
}
```

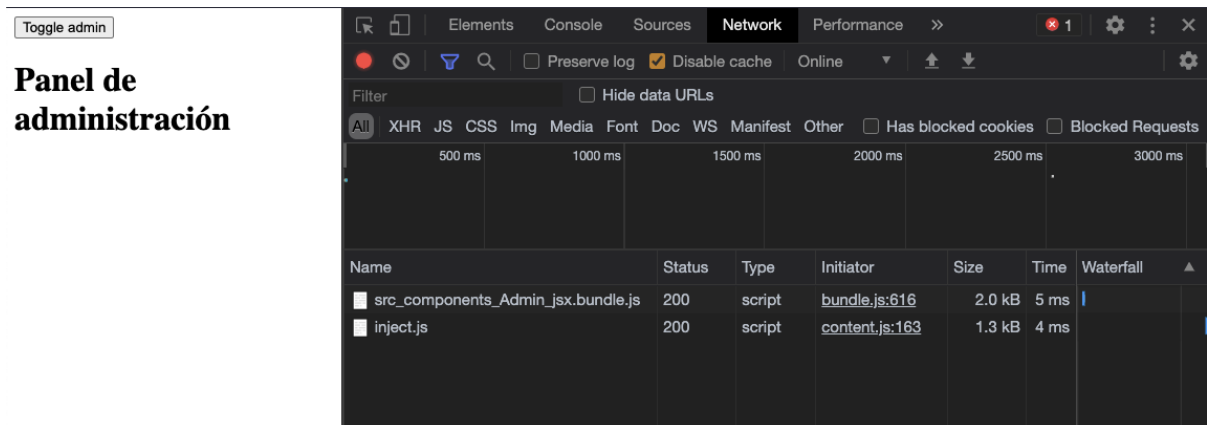
```

    <Inicio />
  }
</div>
)
}

export default App

```

Ahora podemos comprobar que cuando entramos en la página se muestra el componente Inicio, y al pulsar el botón para mostrar el componente de Admin podremos ver que se descarga su código desde la pestaña de **Developer tools > Network**.



Chapter 27. Portals

Los **Portals** son una de las características de React que nos permite renderizar componentes fuera del árbol de componentes de nuestra aplicación, entendiendo árbol de componentes, todos aquellos componentes que cuelgan desde el componente raíz (normalmente el que llamamos **App** y que se inyecta en el archivo de **index.html**). Esto es especialmente útil cuando necesitamos que ciertos elementos (como modales) se rendericen en una ubicación diferente del árbol DOM.

Para crear un portal en React utilizamos el método **ReactDOM.createPortal()** que recibe dos parámetros:

- Aquello que queremos renderizar
- El elemento del DOM donde queremos renderizarlo

```
ReactDOM.createPortal(contenido, contenedor)
```

27.1. Lab: Portals

En este laboratorio vamos a ver cómo usar React Portals para mostrar un modal en un elemento HTML externo al contenedor donde se muestra la aplicación de React.

Vamos a copiar la plantilla del proyecto que se ha realizado en el laboratorio **reactjs-proyecto-inicial** y vamos a cambiarle el nombre del proyecto a **reactjs-portals-lab**.



Es posible que sea necesario volver a instalar las dependencias del proyecto para que funcione correctamente. Para ello solo tenemos que lanzar el comando **npm install** dentro de la carpeta del proyecto.

Ahora vamos a levantar el servidor de desarrollo con el comando:

```
$ npm start
```

Primero, definimos una etiqueta para el portal del modal en el archivo **public/index.html**:

/public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title><%= htmlWebpackPlugin.options.title %></title>
</head>
<body>
  <div id="root"></div>

  <div id="modal"></div>
</body>
</html>
```

Ahora creamos el componente Modal usando React Portal:

/src/components/Modal.jsx

```
import ReactDOM from 'react-dom';
import './Modal.css'

const Modal = ({ onClose, children }) => {
  return ReactDOM.createPortal(
    <div class="backdrop">
      <div class="modal">
        <button onClick={onClose}>X</button>
        <hr />
        {children}
      </div>
    </div>,
    document.getElementById('modal')
```

```

    </div>
  </div>,
  document.getElementById('modal')
)
}

export default Modal

```

Creamos los estilos para el modal:

/src/components/Modal.css

```

.backdrop {
  position: fixed;
  width: 100%;
  height: 100%;
  background-color: rgba(0, 0, 0, 0.5);
  top: 0;
  left: 0;
  display: flex;
  justify-content: center;
  align-items: center;
}

.modal {
  position: relative;
  width: 70vw;
  height: 60vh;
  background-color: white;
}

```

Finalmente, implementamos el componente App:

/src/components/App.jsx

```

import React, { useState } from "react"
import Modal from "../Modal"

const App = () => {
  const [isClosed, setIsClosed] = useState(true)

  return (
    <div>
      <button type="button" onClick={() => setIsClosed(false)}>Abrir modal</button>

      {!isClosed && <Modal onClose={() => setIsClosed(true)}>
        Hola mundo!
      </Modal>}
    </div>
  )
}

```



```
}
```

```
export default App
```

Chapter 28. React Hooks

En la versión 16.8 de React, se han añadido los **React Hooks**, que son funciones que permiten usar características de React en los componentes funcionales que antes no se podían usar, como por ejemplo el estado.

Además, otra ventaja de estas funciones es que nos van a permitir **reutilizar lógica entre los distintos componentes**, algo que antes conseguimos con los **Higher Order Components** cuyo principal problema era que si inspeccionábamos el árbol de componentes, este se volvía demasiado grande y por tanto difícil de seguir.

Usando los componentes funcionales, ya no vamos a tener que preocuparnos por **el objeto this** que tantos problemas puede traer sobre todo a aquellas personas que todavía no se manejan bien con JavaScript.

Ahora en lugar de tener la lógica repartida en distintos métodos del ciclo de vida de los componentes, podemos agruparla en uno de los hooks que veremos.

28.1. Reglas de los React Hooks

Para usar los hooks correctamente, tenemos que tener en cuenta las siguientes reglas:

- Hay que declarar los hooks al inicio de la función del componente.
- No se pueden usar los hooks dentro de componentes de clases. Solo podemos llamar a los hooks desde los componentes funcionales y los custom hooks.
- No debemos llamar a estas funciones dentro de bucles, condicionales o funciones anidadas porque React se basa en el orden en que llamamos a los hooks para guardar el estado del componente. Si el orden de ejecución cambia, React no será capaz de preservar el estado.

28.2. useState

El hook `useState` nos permite añadir estado dentro de nuestros componentes funcionales.

Se trata de una función que recibe como parámetro el valor inicial del estado, y que devuelve un array de dos posiciones, donde la primera es el valor actual del estado, y la segunda es una función para cambiar dicho estado.

```
const [estadoActual, cambiarEstado] = useState(estadoInicial)
```

No hace falta meter todo el estado del componente en un `useState`, sino que podemos usar esta función las veces que lo necesitemos.

```
const [estadoActual1, cambiarEstado1] = useState(estadoInicial1)  
const [estadoActual2, cambiarEstado2] = useState(estadoInicial2)
```

28.2.1. Lab: useState

En este laboratorio vamos a ver como crear un componente contador y mantener su estado con el hook **useState**.

Vamos a copiar la plantilla del proyecto que se ha realizado en el laboratorio **reactjs-proyecto-inicial** y vamos a cambiarle el nombre del proyecto a **reactjs-hooks-usestate-lab**.



Es posible que sea necesario volver a instalar las dependencias del proyecto para que funcione correctamente. Para ello solo tenemos que lanzar el comando **npm install** dentro de la carpeta del proyecto.

Ahora vamos a levantar el servidor de desarrollo con el comando:

```
$ npm start
```

Una vez que tenemos el proyecto levantado, vamos a crear un componente **Contador** funcional dentro de la carpeta **components**.

/reactjs-hooks-usestate-lab/src/components/Contador.jsx

```
import React from 'react'

const Contador = () => {
  return (
    <div>
      <button type="button">-</button>
      <span>Cuenta: </span>
      <button type="button">+</button>
    </div>
  )
}

export default Contador;
```

Necesitamos añadir un estado en este componente para llevar la cuenta, y aquí es donde vamos a usar el hook **useState** que primero vamos a importar desde **react**.

/reactjs-hooks-usestate-lab/src/components/Contador.jsx

```
import React, { useState } from 'react'

const Contador = () => {
  return (
    <div>
      <button type="button">-</button>
      <span>Cuenta: </span>
      <button type="button">+</button>
    </div>
  )
}
```

```

    )
  }

  export default Contador;

```

Una vez importado, vamos a llamar a la función **useState** pasandole como parámetro el valor inicial que queremos usar para el estado, en este caso le pasaremos 0 como valor inicial de la cuenta.

Además, esta función devuelve un array de dos posiciones, donde la primera es el valor actual y la segunda una función para cambiar el estado.

/reactjs-hooks-usestate-lab/src/components/Contador.jsx

```

import React, { useState } from 'react'

const Contador = () => {
  const [cuenta, setCuenta] = useState(0);

  return (
    <div>
      <button type="button">-</button>
      <span>Cuenta: </span>
      <button type="button">+</button>
    </div>
  )
}

export default Contador;

```

El siguiente paso es mostrar el valor actual de la cuenta en el **span**, como venimos haciendo durante el curso.

/reactjs-hooks-usestate-lab/src/components/Contador.jsx

```

import React, { useState } from 'react'

const Contador = () => {
  const [cuenta, setCuenta] = useState(0);

  return (
    <div>
      <button type="button">-</button>
      <span>Cuenta: {cuenta}</span>
      <button type="button">+</button>
    </div>
  )
}

```

```
export default Contador;
```

Antes de continuar con este componente, vamos a comprobar que de momento ya se está mostrando la cuenta, y para ello, vamos a añadir el componente de **Contador**, dentro del **App**.

/reactjs-hooks-usestate-lab/src/components/App.jsx

```
import React, { Component } from 'react'
import Contador from './Contador';

class App extends Component {
  render() {
    return (
      <div>
        <Contador />
      </div>
    )
  }
}

export default App
```

Ahora ya deberíamos de poder ver el contador en nuestro navegador en <http://localhost:8080>.

Lo siguiente es añadir la funcionalidad a los botones para modificar el estado de la cuenta. Tenemos que usar el método **setCuenta** y pasarle como parámetro el nuevo valor de la cuenta.

/reactjs-hooks-usestate-lab/src/components/Contador.jsx

```
import React, { useState } from 'react'

const Contador = () => {
  const [cuenta, setCuenta] = useState(0);

  return (
    <div>
      <button type="button" onClick={() => setCuenta(cuenta-1)}>-</button>
      <span>Cuenta: {cuenta}</span>
      <button type="button" onClick={() => setCuenta(cuenta+1)}>+</button>
    </div>
  )
}

export default Contador;
```

Ahora ya deberíamos de poder modificar la cuenta sin problemas.

28.3. useEffect

El hook **useEffect** es el hook que viene a sustituir a algunos de los métodos del ciclo de vida de los componentes, en concreto viene a sustituir a:

- `componentDidMount`
- `componentDidUpdate`
- `componentWillUnmount`

Este hook, **se ejecuta justo después de cada render**, es decir, cuando se termina de ejecutar

Este hook recibe como parámetro una función que es la que se va a ejecutar cada vez que se renderice el componente, es decir, como si se estuvieran ejecutando **`componentDidMount`** y **`componentDidUpdate`**.

Dentro de esta función será donde pondremos las suscripciones, timers, peticiones AJAX...

```
useEffect(() => {  
  // ...  
})
```

Pero hay que tener cuidado con este hook, ya que puede provocar problemas de memoria porque la función se estará ejecutando cada vez que el componente se renderice.

Esto lo solucionamos pasándole un segundo parámetro al hook. Tenemos las siguientes posibilidades:

- Un array vacío: solo se ejecuta en el primer renderizado.
- Un array con el nombre de propiedades: solo se ejecuta si cambia alguna de estas propiedades.
- No le pasamos nada: se ejecuta cada vez que se renderiza el componente.

```
useEffect(() => {  
  // ...  
}, [texto])
```

Y por último, para realizar las tareas de limpieza en el componente, como desuscribirnos de eventos o eliminar los timers, aquellas que haríamos en dentro del **`componentWillUnmount`**, hay que devolver una función dentro de la función que le hemos pasado al hook.

```
useEffect(() => {  
  // ...  
  return () => {  
    // ...  
  }  
}, [texto])
```

Una ventaja de este hook frente a los métodos del ciclo de vida de los componentes con clase es que podemos usarlo varias veces separandolo por efectos de tal forma que el código queda más limpio sin tener que llenar la función de condicionales que controlen si hay que ejecutar cierto código o no.

```
useEffect(() => {  
  // Efecto 1  
}, [])  
  
useEffect(() => {  
  // Efecto 2  
}, [nombre])
```


28.3.1. Lab: useEffect

En este laboratorio vamos a ver como crear un buscador de cocktails en el que usaremos el hook de **useEffect** para realizar las peticiones.

Vamos a copiar la plantilla del proyecto que se ha realizado en el laboratorio **reactjs-proyecto-inicial** y vamos a cambiarle el nombre del proyecto a **reactjs-hooks-useeffect-lab**.



Es posible que sea necesario volver a instalar las dependencias del proyecto para que funcione correctamente. Para ello solo tenemos que lanzar el comando **npm install** dentro de la carpeta del proyecto.

Ahora vamos a levantar el servidor de desarrollo con el comando:

```
$ npm start
```

Una vez que tenemos el proyecto, vamos a crear un componente **Autocomplete** dentro de la carpeta **components**.

En este componente vamos a empezar añadiendo un input que se va a encargar de guardar el texto por el que vamos a buscar los cocktails. Y también añadiremos la lista de los cocktails que de momento estará vacía.

/reactjs-hooks-useeffect-lab/src/components/Autocomplete.jsx

```
import React, { useState } from 'react'

const Autocomplete = () => {
  const [nombre, setNombre] = useState('');
  const [cocktails, setCocktails] = useState([]);

  const listaCocktails = cocktails.map(c => <li key={c.idDrink}>{c.strDrink}</li>)

  return (
    <div>
      <input type="text" value={nombre} onChange={e => setNombre(e.target.value)} />
      <ul>
        {listaCocktails}
      </ul>
    </div>
  )
}

export default Autocomplete
```

Una vez que tenemos el estado del componente y ya estamos mostrando los elementos, vamos a poner este componente dentro del componente **App** del proyecto.

```
import React, { Component } from 'react'
import Autocomplete from './Autocomplete'

class App extends Component {
  render() {
    return (
      <div>
        <Autocomplete />
      </div>
    )
  }
}

export default App
```

El siguiente paso es realizar la petición de búsqueda a la API de cocktails que podemos encontrar en <https://www.thecocktaildb.com/api.php>.

Para ello, vamos a empezar añadiendo el hook del **useEffect** en el que añadiremos una petición **fetch** a la API en la que pasaremos como parámetro de búsqueda el nombre que estamos escribiendo en el campo de texto. Esta petición solo la haremos si este nombre tiene más de 3 caracteres.

Esta petición devuelve una promesa que nos da la respuesta de la que primero tendremos que extraer los datos y después tendremos que sacar el array de bebidas, el cual guardaremos en el estado que habíamos creado.

```
import React, { useState, useEffect } from 'react'

const Autocomplete = () => {
  const [nombre, setNombre] = useState('');
  const [cocktails, setCocktails] = useState([]);

  useEffect(() => {
    if (nombre.length > 3) {
      fetch(`https://www.thecocktaildb.com/api/json/v1/1/search.php?s=${nombre}`)
        .then(resp => resp.json())
        .then(({drinks}) => {
          setCocktails(() => drinks || [])
        })
    }
  })

  const listaCocktails = cocktails.map(c => <li key={c.idDrink}>{c.strDrink}</li>)

  return (
```

```

    <div>
      <input type="text" value={nombre} onChange={e => setNombre(e.target.value)} />
      <ul>
        {listaCocktails}
      </ul>
    </div>
  )
}

export default Autocomplete

```

Ahora ya deberíamos de poder ver la lista con los cocktails que coinciden con el parámetro de búsqueda que hemos puesto en el campo de texto.

Pero ahora mismo tenemos un problema que no estamos viendo, y es que no le hemos pasado una lista de dependencias al hook, por lo que cuando cambie cualquier dato del componente, este se ejecutará, aunque dicho cambio no le afecte a la petición. Esto lo podemos solucionar añadiendo el **nombre** como dependencia, de tal forma que solo se va a ejecutar la petición a la API cuando se vaya cambiando el nombre.

/reactjs-hooks-useeffect-lab/src/components/Autocomplete.jsx

```

import React, { useState, useEffect } from 'react'

const Autocomplete = () => {
  const [nombre, setNombre] = useState('');
  const [cocktails, setCocktails] = useState([]);

  useEffect(() => {
    if (nombre.length > 3) {
      fetch(`https://www.thecocktaildb.com/api/json/v1/1/search.php?s=${nombre}`)
        .then(resp => resp.json())
        .then(({drinks}) => {
          setCocktails(() => drinks || [])
        })
    }
  }, [nombre])

  const listaCocktails = cocktails.map(c => <li key={c.idDrink}>{c.strDrink}</li>)

  return (
    <div>
      <input type="text" value={nombre} onChange={e => setNombre(e.target.value)} />
      <ul>
        {listaCocktails}
      </ul>
    </div>
  )
}

```

export default Autocomplete

El siguiente paso será seleccionar uno de los cocktails que se nos muestran para autocompletar el input con el.

Ahora vamos a añadir una función que va a recibir el cocktail sobre el que pulsemos y se va a setear el nombre de este en el estado.

/reactjs-hooks-useeffect-lab/src/components/Autocomplete.jsx

```
import React, { useState, useEffect } from 'react'

const Autocomplete = () => {
  const [nombre, setNombre] = useState('');
  const [cocktails, setCocktails] = useState([]);

  useEffect(() => {
    if (nombre.length > 3) {
      fetch(`https://www.thecocktaildb.com/api/json/v1/1/search.php?s=${nombre}`)
        .then(resp => resp.json())
        .then(({drinks}) => {
          setCocktails(() => drinks || [])
        })
    }
  }, [nombre])

  const selectCocktail = (cocktailSeleccionado) => {
    setNombre(cocktailSeleccionado.strDrink)
  }

  const listaCocktails = cocktails.map(c => <li onClick={() => selectCocktail(c)} key={c.idDrink}>{c.strDrink}</li>)

  return (
    <div>
      <input type="text" value={nombre} onChange={e => setNombre(e.target.value)} />
      <ul>
        {listaCocktails}
      </ul>
    </div>
  )
}

export default Autocomplete
```

Y con esto ya tendríamos nuestro componente que autocompleta un campo con la opción seleccionada de aquellas que se nos muestran al realizar la búsqueda.

28.4. useContext

El hook **useContext** nos permite acceder al contexto creado con React para pasar valores directamente de un componente a otro cuando hay demasiados niveles entre medias de estos componentes.

Este hook es una función a la que le vamos a pasar como parámetro el contexto que hemos tenido que haber creado con **React.createContext**, y nos devolverá el valor que se ha dado inicialmente al contexto o el que se ha pasado en la propiedad **value** del **Provider** del contexto creado.

```
const Ctx = React.createContext(1);  
  
const num = useContext(Ctx)
```

28.4.1. Lab: useContext

En este laboratorio vamos a ver como usar el hook **useContext** para traducir los textos de la aplicación.

Vamos a copiar la plantilla del proyecto que se ha realizado en el laboratorio **reactjs-proyecto-inicial** y vamos a cambiarle el nombre del proyecto a **reactjs-hooks-usecontext-lab**.



Es posible que sea necesario volver a instalar las dependencias del proyecto para que funcione correctamente. Para ello solo tenemos que lanzar el comando **npm install** dentro de la carpeta del proyecto.

Ahora vamos a levantar el servidor de desarrollo con el comando:

```
$ npm start
```

Lo primero que vamos a hacer es crear el componente **Context**, en el que vamos a crear un contexto de React (que exportaremos) y añadiremos un objeto con las traducciones de nuestra aplicación en distintos idiomas.

/reactjs-hooks-usecontext-lab/src/components/Context.jsx

```
import React from 'react'

export const LangCtx = React.createContext(null)

const traducciones = {
  es: {
    bienvenida: 'Bienvenido a mi Startup'
  },
  en: {
    bienvenida: 'Welcome to my Startup'
  },
  fr: {
    bienvenida: 'Bienvenue dans ma Startup'
  }
}

const Context = () => {

  return (
    <div>

    </div>
  )
}

export default Context
```

Ahora vamos a añadir este componente dentro del componente raíz de la aplicación.

/reactjs-hooks-usecontext-lab/src/components/App.jsx

```
import React, { Component } from 'react'
import Context from './Context'

class App extends Component {
  render() {
    return (
      <div>
        <Context />
      </div>
    )
  }
}

export default App
```

Ahora vamos a crearnos un componente **SelectLang** que tendrá un desplegable para poder mostrar los distintos idiomas y seleccionar en cual queremos que se muestre nuestra aplicación. Este componente recibirá como propiedades los distintos lenguajes, el lenguaje seleccionado y la función que cambia de lenguaje.

/reactjs-hooks-usecontext-lab/src/components/SelectLang.jsx

```
import React from 'react';

const SelectLang = ({langs, onChangeLang, selectedLang}) => {
  const optionsLang = langs.map((l, pos) => <option key={pos} value={l}>{l.toUpperCase()}</option>)

  return (
    <select onChange={(e) => onChangeLang(e.target.value)} value={selectedLang}>
      {optionsLang}
    </select>
  )
}

export default SelectLang
```

Una vez que tenemos el componente, vamos a añadirlo en el componente **Context**, donde añadiremos el estado que guarda el lenguaje seleccionado.

/reactjs-hooks-usecontext-lab/src/components/Context.jsx

```
import React, { useState } from 'react'
import SelectLang from './SelectLang'

export const LangCtx = React.createContext(null)

const traducciones = {
  es: {
```

```

    bienvenida: 'Bienvenido a mi Startup'
  },
  en: {
    bienvenida: 'Welcome to my Startup'
  },
  fr: {
    bienvenida: 'Bienvenue dans ma Startup'
  }
}

const Context = () => {
  const [lang, setLang] = useState('es')

  return (
    <div>
      <SelectLang langs={Object.keys(traducciones)} onChangeLang={setLang} selectedLang={lang} />
    </div>
  )
}

export default Context

```

Ahora ya deberíamos de poder ver el desplegable y cambiar el valor del lenguaje.

Lo siguiente es crear el componente **Header** en el que usaremos el hook del **useContext** para pasarle las traducciones correspondientes al lenguaje seleccionado y así mostrar el título en distintos idiomas.

En este componente importaremos el **LangCtx** que hemos creado anteriormente para pasárselo al hook y que nos devuelva el valor que nos provee el contexto, para así poder mostrarlo.

/reactjs-hooks-usecontext-lab/src/components/Header.jsx

```

import React, { useContext } from 'react'
import { LangCtx } from './Context'

const Header = () => {
  const { bienvenida } = useContext(LangCtx)

  return (
    <h1>{bienvenida}</h1>
  )
}

export default Header

```

Por último, nos faltaría envolver este componente con el **Provider** y pasarle el valor a través de esta etiqueta desde el componenten **Context**.

/reactjs-hooks-usecontext-lab/src/components/Context.jsx

```

import React, { useState } from 'react'

```



```

import SelectLang from './SelectLang'
import Header from './Header'

export const LangCtx = React.createContext(null)

const traducciones = {
  es: {
    bienvenida: 'Bienvenido a mi Startup'
  },
  en: {
    bienvenida: 'Welcome to my Startup'
  },
  fr: {
    bienvenida: 'Bienvenue dans ma Startup'
  }
}

const Context = () => {
  const [lang, setLang] = useState('es')

  return (
    <div>
      <SelectLang langs={Object.keys(traducciones)} onChangeLang={setLang} selectedLang={lang} />
      <LangCtx.Provider value={traducciones[lang]}>
        <Header />
      </LangCtx.Provider>
    </div>
  )
}

export default Context

```

Y con lo que acabamos de hacer ya debería de cambiar el texto del header al lenguaje que hayamos seleccionado en el desplegable.

28.5. useMemo

El hook **useMemo** va a permitirnos memorizar un valor que se usará en los renderizados de tal forma que solo se va a volver a calcular cuando alguna de las dependencias que se le han pasado como segundo parámetro haya sufrido cambios. Como primer parámetro recibe la función que calcula el valor y lo devuelve, y el hook nos devuelve dicho valor, memorizado o calculado de nuevo.

```
const valorCalculado = useMemo(() => {  
  // val = ...  
  return val;  
}, [dep1])
```

Este hook sería equivalente a usar el método del ciclo de vida **shouldComponentUpdate**.

28.5.1. Lab: useMemo

En este laboratorio vamos a ver como usar el hook de **useMemo** para crear una propiedad **nombreCompleto** formada a partir de las propiedades **nombre** y **apellido** del estado y que solo se calcule cuando cambie alguna de estas dos propiedades.

Vamos a copiar la plantilla del proyecto que se ha realizado en el laboratorio **reactjs-proyecto-inicial** y vamos a cambiarle el nombre del proyecto a **reactjs-hooks-usememo-lab**.



Es posible que sea necesario volver a instalar las dependencias del proyecto para que funcione correctamente. Para ello solo tenemos que lanzar el comando **npm install** dentro de la carpeta del proyecto.

Ahora vamos a levantar el servidor de desarrollo con el comando:

```
$ npm start
```

Lo siguiente es crear el componente **Persona** en el que vamos a añadir tres estados (nombre, apellido y email), los vamos a mostrar y añadiremos también tres inputs para poder cambiar estos datos.

/reactjs-hooks-usememo-lab/src/components/Persona.jsx

```
import React, { useState } from 'react'

const Persona = () => {
  const [nombre, setNombre] = useState('Charly')
  const [apellido, setApellido] = useState('Falco')
  const [email, setEmail] = useState('cfalco@gmail.com')

  return (
    <div>
      <input type="text" value={nombre} onChange={(e) => setNombre(e.target.value)}/>
      <input type="text" value={apellido} onChange={(e) => setApellido(e.target.value)}/>
      <input type="text" value={email} onChange={(e) => setEmail(e.target.value)}/>
      <hr/>
      <p>{nombre}</p>
      <p>{apellido}</p>
      <p>{email}</p>
    </div>
  )
}

export default Persona
```

Una vez que lo tenemos, vamos a añadir este componente dentro del componente raíz de la aplicación para mostrarlo.

```
import React, { Component } from 'react'
import Persona from './Persona'

class App extends Component {
  render() {
    return (
      <div>
        <Persona />
      </div>
    )
  }
}

export default App
```

Lo siguiente que vamos a hacer es crear una función que va a calcular el nombre completo a partir de las propiedades nombre y apellido del estado.

```
import React, { useState } from 'react'

const Persona = () => {
  const [nombre, setNombre] = useState('Charly')
  const [apellido, setApellido] = useState('Falco')
  const [email, setEmail] = useState('cfalco@gmail.com')

  const nombreCompleto = () => {
    return <p>{nombre} {apellido}</p>
  }

  return (
    <div>
      <input type="text" value={nombre} onChange={(e) => setNombre(e.target.value)} />
      <input type="text" value={apellido} onChange={(e) => setApellido(e.target.value)} />
      <input type="text" value={email} onChange={(e) => setEmail(e.target.value)} />
      <hr />
      <p>{nombreCompleto()}</p>
      <p>{email}</p>
    </div>
  )
}

export default Persona
```

Ahora deberíamos de ver los valores del estado y poder cambiarlos, pero todavía hay algo que podemos mejorar. ¿Que pasa si modificamos la propiedad del email? Vamos a mostrar por consola un mensaje dentro de la función que calcula el nombre completo, para ver que está ocurriendo.

```
import React, { useState } from 'react'

const Persona = () => {
  const [nombre, setNombre] = useState('Charly')
  const [apellido, setApellido] = useState('Falco')
  const [email, setEmail] = useState('cfalco@gmail.com')

  const nombreCompleto = () => {
    console.log('nombreCompleto')
    return <p>{nombre} {apellido}</p>
  }

  return (
    <div>
      <input type="text" value={nombre} onChange={(e) => setNombre(e.target.value)}>
      <input type="text" value={apellido} onChange={(e) => setApellido(e.target.value)}>
      <input type="text" value={email} onChange={(e) => setEmail(e.target.value)}>
      <hr/>
      <p>{nombreCompleto()}</p>
      <p>{email}</p>
    </div>
  )
}

export default Persona
```

Si modificamos el email, vemos que se va calculando también el nombre completo, aunque no hayan cambiado ni el nombre ni el apellido, y esto es algo que podríamos evitar con el hook **useMemo**.

Vamos a meter dentro de este hook la función de **nombreCompleto**, y como este hook nos devuelve el valor de la función ya memorizado vamos a dejar de ejecutar la función dentro de nuestro código JSX.

```
import React, { useState, useMemo } from 'react'

const Persona = () => {
  const [nombre, setNombre] = useState('Charly')
  const [apellido, setApellido] = useState('Falco')
  const [email, setEmail] = useState('cfalco@gmail.com')

  const nombreCompleto = useMemo(() => {
    console.log('nombreCompleto')
    return <p>{nombre} {apellido}</p>
  })

  return (
```

```

    <div>
      <input type="text" value={nombre} onChange={(e) => setNombre(e.target.value)}>/>
      <input type="text" value={apellido} onChange={(e) => setApellido(e.target.value)}>/>
      <input type="text" value={email} onChange={(e) => setEmail(e.target.value)}>/>
      <hr/>
      <p>{nombreCompleto}</p>
      <p>{email}</p>
    </div>
  )
}

export default Persona

```

Ahora si volvemos a probar a modificar el campo, vemos que sigue ocurriendo lo mismo. El problema está en que no le estamos indicando cuando tiene que ejecutar la función para volver a calcular el nuevo valor, y por tanto la está ejecutando todas las veces que se renderiza el componente.

Para solucionarlo, tenemos que pasarle una lista de dependencias como segundo parámetro para indicarle que solo tiene que volver ejecutarla cuando alguna de esas dependencias haya cambiado. Por tanto, en este caso, vamos a pasarle un array con **nombre** y **apellido** como dependencias.

/reactjs-hooks-usememo-lab/src/components/Persona.jsx

```

import React, { useState, useMemo } from 'react'

const Persona = () => {
  const [nombre, setNombre] = useState('Charly')
  const [apellido, setApellido] = useState('Falco')
  const [email, setEmail] = useState('cfalco@gmail.com')

  const nombreCompleto = useMemo(() => {
    console.log('nombreCompleto')
    return <p>{nombre} {apellido}</p>
  }, [nombre, apellido])

  return (
    <div>
      <input type="text" value={nombre} onChange={(e) => setNombre(e.target.value)}>/>
      <input type="text" value={apellido} onChange={(e) => setApellido(e.target.value)}>/>
      <input type="text" value={email} onChange={(e) => setEmail(e.target.value)}>/>
      <hr/>
      <p>{nombreCompleto}</p>
      <p>{email}</p>
    </div>
  )
}

export default Persona

```

Ahora ya podemos observar que cuando cambiamos el email, no se pasa por la función **nombreCompleto**, sino que usa el valor memorizado la primera vez, y en el caso de que cambiemos el nombre o el apellido, vuelve a ejecutar la función u memoriza el nuevo valor para evitar tener que hacerlo en los siguientes renders.

28.6. useRef

El hook **useRef** nos permite crear una referencia a etiquetas HTML del componente para así poder acceder a sus propiedades y métodos internos.

Este hook recibe como parámetro el valor inicial con el cual queremos inicializar la propiedad **current** de la referencia. Y devuelve la referencia para asignarla a aquella etiqueta HTML con la que queramos interactuar.

```
const ref = useRef(valorInicial)
```

Para asignarla a una etiqueta y poder acceder a sus atributos internos, solo hay que darle esta variable como valor del atributo **ref** de la etiqueta.

```
<video src="video.mp4" ref={ref} />
```

El objeto que nos devuelve el hook es mutable y cualquier cambio sobre el no hace que se renderice el componente de nuevo.

28.6.1. Lab: useRef

En este laboratorio vamos a crear un panel de control para trabajar con la etiqueta audio y con el cual vamos a poder reproducir, pausar y modificar el volumen.

Vamos a copiar la plantilla del proyecto que se ha realizado en el laboratorio **reactjs-proyecto-inicial** y vamos a cambiarle el nombre del proyecto a **reactjs-hooks-useref-lab**.



Es posible que sea necesario volver a instalar las dependencias del proyecto para que funcione correctamente. Para ello solo tenemos que lanzar el comando **npm install** dentro de la carpeta del proyecto.

Para este laboratorio necesitamos usar cualquier archivo de audio, que hay que meter dentro de la carpeta **dist/assets** del proyecto.

Una vez que tenemos el proyecto vamos a empezar creando un nuevo componente funcional **AudioPlayer** en la carpeta **components**.

/reactjs-hooks-useref-lab/src/components/AudioPlayer.jsx

```
import React, { useRef } from 'react'

const AudioPlayer = () => {
  return (
    <div>
      <audio src="/assets/sonido-piolin.m4a" />
      <button type="button">Play</button>
      <button type="button">Pause</button>
      <input type="range" name="volume" id="volume" />
    </div>
  )
}

export default AudioPlayer
```

El siguiente paso es añadir el componente que hemos creado en el componente raíz de la aplicación.

/reactjs-hooks-useref-lab/src/components/App.jsx

```
import React, { Component } from 'react'
import AudioPlayer from './AudioPlayer'

class App extends Component {
  render() {
    return (
      <div>
        <AudioPlayer />
      </div>
    )
  }
}
```

```

    }
  }

  export default App

```

Ahora vamos a usar el hook de **useRef** para añadir una referencia a la etiqueta de audio del componente para poder acceder a todas sus funcionalidades. Para ello solo tenemos que importar el hook, y pasarle a una propiedad **ref** de la etiqueta audio la referencia que nos ha devuelto el hook.

/reactjs-hooks-useref-lab/src/components/AudioPlayer.jsx

```

import React, { useRef } from 'react'

const AudioPlayer = () => {
  const audioRef = useRef(null);

  return (
    <div>
      <audio src="/assets/sonido-piolin.m4a" ref={audioRef} />
      <button type="button">Play</button>
      <button type="button">Pause</button>
      <input type="range" name="volume" id="volume" />
    </div>
  )
}

export default AudioPlayer

```

Pues el siguiente paso es añadir la funcionalidad a los botones y al input para poder controlar el sonido que hayamos puesto.

Empezamos por crearnos una función para reproducir la canción cuando pulsemos el primer botón. En esta función vamos a coger la referencia que nos ha devuelto el hook, accederemos a **current** y ejecutaremos la función **play** de la etiqueta audio para reproducir el audio.

/reactjs-hooks-useref-lab/src/components/AudioPlayer.jsx

```

import React, { useRef } from 'react'

const AudioPlayer = () => {
  const audioRef = useRef(null);

  const play = () => {
    audioRef.current.play();
  }

  return (
    <div>
      <audio src="/assets/sonido-piolin.m4a" ref={audioRef} />

```

```

    <button type="button">Play</button>
    <button type="button">Pause</button>
    <input type="range" name="volume" id="volume" />
  </div>
)
}

export default AudioPlayer

```

Ahora haremos lo mismo pero creando una función **pause** en la que llamaremos al método **pause** de la referencia a la etiqueta audio.

/reactjs-hooks-useref-lab/src/components/AudioPlayer.jsx

```

import React, { useRef } from 'react'

const AudioPlayer = () => {
  const audioRef = useRef(null);

  const play = () => {
    audioRef.current.play();
  }

  const pause = () => {
    audioRef.current.pause();
  }

  return (
    <div>
      <audio src="/assets/sonido-piolin.m4a" ref={audioRef} />
      <button type="button">Play</button>
      <button type="button">Pause</button>
      <input type="range" name="volume" id="volume" />
    </div>
  )
}

export default AudioPlayer

```

El siguiente paso es pasarle una referencia de estas dos funciones a los eventos **onClick** de cada uno de los botones.

/reactjs-hooks-useref-lab/src/components/AudioPlayer.jsx

```

import React, { useRef } from 'react'

const AudioPlayer = () => {
  const audioRef = useRef(null);

  const play = () => {

```

```

    audioRef.current.play();
  }

  const pause = () => {
    audioRef.current.pause();
  }

  return (
    <div>
      <audio src="/assets/sonido-piolin.m4a" ref={audioRef} />
      <button type="button" onClick={play}>Play</button>
      <button type="button" onClick={pause}>Pause</button>
      <input type="range" name="volume" id="volume" />
    </div>
  )
}

export default AudioPlayer

```

Ahora ya deberíamos de poder reproducir y pausar el sonido.

Lo siguiente es añadir la funcionalidad para poder cambiar el volumen del audio. Lo primero que necesitamos es añadir un estado para guardar el valor del volumen y cambiarlo al mover el slider del input. El valor inicial del estado lo vamos a obtener de la propiedad **volume** que obtenemos a partir de la referencia.



La propiedad volume de las etiquetas audio y video coge un valor entre 0 y 1.

/reactjs-hooks-useref-lab/src/components/AudioPlayer.jsx

```

import React, { useRef } from 'react'

const AudioPlayer = () => {
  const audioRef = useRef(null);
  const [volume, setVolume] = useState(audioRef.current.volume * 100)

  const play = () => {
    audioRef.current.play();
  }

  const pause = () => {
    audioRef.current.pause();
  }

  return (
    <div>
      <audio src="/assets/sonido-piolin.m4a" ref={audioRef} />
      <button type="button" onClick={play}>Play</button>
      <button type="button" onClick={pause}>Pause</button>
      <input type="range" name="volume" id="volume" />
    </div>
  )
}

```

```

    </div>
  )
}

export default AudioPlayer

```

Antes de continuar, si miramos la consola del navegador, nos mostrará un error indicando que no puede acceder a la propiedad **volume** de **null**. Esto se debe a que la referencia la hemos inicializado con **null**, y por tanto no tiene propiedades a las que acceder. Para solucionarlo, vamos a inicializar esta referencia con una instancia de la clase **Audio** que es con la que se construyen las etiquetas de este tipo, y la cual nos proporcionará acceso a todas las propiedades iniciales que tiene esta etiqueta.

/reactjs-hooks-useref-lab/src/components/AudioPlayer.jsx

```

import React, { useRef } from 'react'

const AudioPlayer = () => {
  const audioRef = useRef(new Audio());
  const [volume, setVolume] = useState(audioRef.current.volume * 100)

  const play = () => {
    audioRef.current.play();
  }

  const pause = () => {
    audioRef.current.pause();
  }

  return (
    <div>
      <audio src="/assets/sonido-piolin.m4a" ref={audioRef} />
      <button type="button" onClick={play}>Play</button>
      <button type="button" onClick={pause}>Pause</button>
      <input type="range" name="volume" id="volume" />
    </div>
  )
}

export default AudioPlayer

```

Solucionado este error, vamos a darle el valor al input y vamos a añadir el evento onChange que llamará a la función changeVolume que vamos a crear a continuación.

/reactjs-hooks-useref-lab/src/components/AudioPlayer.jsx

```

import React, { useRef } from 'react'

const AudioPlayer = () => {
  const audioRef = useRef(new Audio());

```

```

const [volume, setVolume] = useState(audioRef.current.volume * 100)

const play = () => {
  audioRef.current.play();
}

const pause = () => {
  audioRef.current.pause();
}

return (
  <div>
    <audio src="/assets/sonido-piolin.m4a" ref={audioRef} />
    <button type="button" onClick={play}>Play</button>
    <button type="button" onClick={pause}>Pause</button>
    <input type="range" name="volume" id="volume" min="0" max="100" value={volume} onChange={changeVolume} />
  </div>
)
}

export default AudioPlayer

```

Esta función que se va a ejecutar cuando vayamos moviendo el slider del volumen lo que tiene que hacer es acceder al valor del input a partir del evento que recibimos como parámetro y cambiar la propiedad volume de la referencia y el estado.

/reactjs-hooks-useref-lab/src/components/AudioPlayer.jsx

```

import React, { useRef } from 'react'

const AudioPlayer = () => {
  const audioRef = useRef(new Audio());
  const [volume, setVolume] = useState(audioRef.current.volume * 100)

  const play = () => {
    audioRef.current.play();
  }

  const pause = () => {
    audioRef.current.pause();
  }

  const changeVolume = (e) => {
    const newVolume = e.target.value;
    audioRef.current.volume = newVolume / 100;
    setVolume(newVolume);
  }

  return (
    <div>
      <audio src="/assets/sonido-piolin.m4a" ref={audioRef} />
      <button type="button" onClick={play}>Play</button>
      <button type="button" onClick={pause}>Pause</button>
      <input type="range" name="volume" id="volume" min="0" max="100" value={volume} onChange={changeVolume} />
    </div>
  )
}

export default AudioPlayer

```

Y con esto ya deberíamos de ser capaces de cambiar el volumen del audio.

28.7. useReducer

El hook **useReducer** funciona como una función reducer de la librería Redux. Nos va a permitir gestionar el estado o parte del estado de los componentes.

Este hook es una función que recibe como parámetros la función de reducer, que a su vez recibe el estado actual y una acción, y el estado inicial. Y este hook devuelve un array de dos posiciones en la que en la primera nos encontramos con el estado actual y en la segunda con un método **dispatch** que usaremos para emitir las acciones que le llegarán al reducer.

```
const initialState = {}

function reducer(state, action) {
  // ...
}

const [state, dispatch] = useReducer(reducer, initialState)
```

Pero entonces, ¿cuándo usamos **useState** y cuándo **useReducer**?

La diferencia entre usar `useState` y `useReducer` es que este último suele usarse cuando para cambiar el estado necesitamos añadir algo de lógica.

28.7.1. Lab: useReducer

En este laboratorio vamos a ver como mover un cuadrado por la ventana usando las teclas de las flechas y gestionando el estado con el hook **useReducer**.

Vamos a copiar la plantilla del proyecto que se ha realizado en el laboratorio **reactjs-proyecto-inicial** y vamos a cambiarle el nombre del proyecto a **reactjs-hooks-usereducer-lab**.



Es posible que sea necesario volver a instalar las dependencias del proyecto para que funcione correctamente. Para ello solo tenemos que lanzar el comando **npm install** dentro de la carpeta del proyecto.

Ahora vamos a levantar el servidor de desarrollo con el comando:

```
$ npm start
```

Empezamos por crear un componente **Caja** que será el componente que vamos a mover por la pantalla.

/reactjs-hooks-usereducer-lab/src/components/Caja.jsx

```
import React from 'react'

const Caja = () => {

  return (
    <div style={{width: '100px', height: '100px', backgroundColor: 'darkred', position: 'absolute', left: '0px', top: '0px'}}>
      </div>
    )
  }

export default Caja
```

Ahora que tenemos la caja, vamos a mostrarla en el componente **App** de la aplicación.

/reactjs-hooks-usereducer-lab/src/components/App.jsx

```
import React, { Component } from 'react'
import Caja from './Caja'

class App extends Component {
  render() {
    return (
      <div>
        <Caja />
      </div>
    )
  }
}
```

Ahora vamos a crear nuestro reducer para gestionar la posición de la caja en la pantalla. Para esto vamos a usar el hook **useReducer** al que le vamos a pasar una función de reducir y el estado inicial como parámetros.

El estado será un objeto con la posición X e Y en la pantalla e inicialmente pondremos la caja centrada en la pantalla.

En cuanto al reducer, es la función que va a recibir el estado actual y una acción que indica que cambios hay que realizar al estado. En este caso en la acción recibiremos la tecla que se ha pulsado y dentro de un bloque switch sumaremos o restaremos un valor a las coordenadas del estado dependiendo de las teclas pulsadas.

/reactjs-hooks-usereducer-lab/src/components/Caja.jsx

```
import React from 'react'

const initialState = {
  posX: (window.innerWidth / 2) - 50,
  posY: (window.innerHeight / 2) - 50
}

function reducer(state, action) {
  switch(action.type) {
    case 'ArrowLeft':
      return {...state, posX: `${parseInt(state.posX)-10}px`};
    case 'ArrowRight':
      return {...state, posX: `${parseInt(state.posX)+10}px`};
    case 'ArrowUp':
      return {...state, posY: `${parseInt(state.posY)-10}px`};
    case 'ArrowDown':
      return {...state, posY: `${parseInt(state.posY)+10}px`};
    default:
      return state;
  }
}

const Caja = () => {
  return (
    <div style={{width: '100px', height: '100px', backgroundColor: 'darkred', position: 'absolute', left: '0px', top: '0px'}}>
    </div>
  )
}

export default Caja
```

Ahora que tenemos el reducer y el estado inicial, vamos a añadir el hook dentro del componente al que le vamos a pasar estos dos argumentos y nos va a devolver el estado actual que serán las coordenadas y la función de **dispatch** para poder emitir acciones al reducer.

Ya que tenemos las coordenadas, vamos a darselas como valor a las propiedades **left** y **top** de los estilos del componente.

```
import React, { useReducer } from 'react'

const initialState = {
  posX: (window.innerWidth / 2) - 50,
  posY: (window.innerHeight / 2) - 50
}

function reducer(state, action) {
  switch(action.type) {
    case 'ArrowLeft':
      return {...state, posX: `${parseInt(state.posX)-10}px`};
    case 'ArrowRight':
      return {...state, posX: `${parseInt(state.posX)+10}px`};
    case 'ArrowUp':
      return {...state, posY: `${parseInt(state.posY)-10}px`};
    case 'ArrowDown':
      return {...state, posY: `${parseInt(state.posY)+10}px`};
    default:
      return state;
  }
}

const Caja = () => {
  const [coords, dispatch] = useReducer(reducer, initialState)

  return (
    <div style={{width: '100px', height: '100px', backgroundColor: 'darkred', position: 'absolute', left: coords.posX, top: coords.posY}}>
    </div>
  )
}

export default Caja
```

Con estos cambios, la caja debería de aparecer centrada en la página, y solo nos queda poder moverla.

Empezamos por añadir una función que recibe un evento y despacha la acción en la que se pasa el nombre de la tecla que se ha pulsado.

```
import React, { useReducer } from 'react'

const initialState = {
  posX: (window.innerWidth / 2) - 50,
  posY: (window.innerHeight / 2) - 50
}

function reducer(state, action) {
  switch(action.type) {
    case 'ArrowLeft':
      return {...state, posX: `${parseInt(state.posX)-10}px`};
    case 'ArrowRight':
      return {...state, posX: `${parseInt(state.posX)+10}px`};
    case 'ArrowUp':
      return {...state, posY: `${parseInt(state.posY)-10}px`};
    case 'ArrowDown':
      return {...state, posY: `${parseInt(state.posY)+10}px`};
    default:
```

```

    return state;
  }
}

const Caja = () => {
  const [coords, dispatch] = useReducer(reducer, initialState)

  const dispatchAction = (e) => {
    dispatch({type: e.key})
  }

  return (
    <div style={{width: '100px', height: '100px', backgroundColor: 'darkred', position: 'absolute', left: coords.posX,
top: coords.posY}}>
      </div>
    )
  }
}

export default Caja

```

Para moverla con las flechas del teclado, necesitamos añadir un listener del evento **keydown** sobre la página del navegador cuando se cargue este componente, además de que tendremos que quitar el listener si el componente se elimina.

Vamos a usar el hook de **useEffect** donde añadiremos estos listeners y como solo queremos que este hook se ejecute una sola vez, le pasaremos como segundo parámetro un array vacío.

Y para poder eliminar el listener tendremos que devolver dentro del **useEffect** una función en la que llamemos al **removeEventListener**.

/reactjs-hooks-usereducer-lab/src/components/Caja.jsx

```

import React, { useReducer, useEffect } from 'react'

const initialState = {
  posX: (window.innerWidth / 2) - 50,
  posY: (window.innerHeight / 2) - 50
}

function reducer(state, action) {
  switch(action.type) {
    case 'ArrowLeft':
      return {...state, posX: `${parseInt(state.posX)-10}px`};
    case 'ArrowRight':
      return {...state, posX: `${parseInt(state.posX)+10}px`};
    case 'ArrowUp':
      return {...state, posY: `${parseInt(state.posY)-10}px`};
    case 'ArrowDown':
      return {...state, posY: `${parseInt(state.posY)+10}px`};
    default:
      return state;
  }
}

const Caja = () => {
  const [coords, dispatch] = useReducer(reducer, initialState)

  const dispatchAction = (e) => {
    dispatch({type: e.key})
  }

  useEffect(() => {

```

```

    window.addEventListener('keydown', dispatchAction)
    return () => {
      window.removeEventListener('keydown', dispatchAction)
    }
  }, [])

  return (
    <div style={{width: '100px', height: '100px', backgroundColor: 'darkred', position: 'absolute', left: coords.posX,
top: coords.posY}}>
      </div>
    )
  }
}

export default Caja

```

Al añadir el listener, ya deberíamos de poder mover la caja por la pantalla usando las teclas de las flechas del teclado.

28.8. Crear un hook

Hemos visto algunos de los hooks que ya nos trae React, pero cualquiera puede crear sus propios hooks, pues al final son funciones que reciben unos parámetros y devuelven o no unos datos o métodos que nos permiten interactuar con la lógica que encapsula el hook.

El nombre de un hook creado por nosotros (al igual que los que no) tiene que empezar por **use**, y dentro de el podremos llamar a otros hooks ya existentes.

Podemos encontrar muchos hooks que han ido desarrollando otras personas en la siguiente url: <https://nikgraf.github.io/react-hooks/>.

28.8.1. Lab: Crear un hook

En este laboratorio vamos a ver como crear nuestro propio hook para mantener los campos de un formulario.

Vamos a copiar la plantilla del proyecto que se ha realizado en el laboratorio **reactjs-proyecto-inicial** y vamos a cambiarle el nombre del proyecto a **reactjs-hooks-custom-hook-lab**.



Es posible que sea necesario volver a instalar las dependencias del proyecto para que funcione correctamente. Para ello solo tenemos que lanzar el comando **npm install** dentro de la carpeta del proyecto.

Ahora vamos a levantar el servidor de desarrollo con el comando:

```
$ npm start
```

Vamos a empezar creando un componente **Form** dentro de la carpeta **components** en el que añadiremos un campo de texto y un parrafo para mostrar si el campo es válido o no.

/reactjs-hooks-custom-hook-lab/src/components/Form.jsx

```
import React from 'react'

const Form = () => {
  return (
    <div>
      <input type="text" />
      <p>Válido o inválido</p>
    </div>
  )
}

export default Form
```

Una vez que tenemos el componente, vamos a añadirlo a nuestra aplicación, dentro del componente **App**.

/reactjs-hooks-custom-hook-lab/src/components/App.jsx

```
import React, { Component } from 'react'
import Form from './Form'

class App extends Component {
  render() {
    return (
      <div>
        <Form />
      </div>
    )
  }
}
```

```
}  
}  
  
export default App
```

Ahora ya deberíamos de ver el componente en el navegador, pero no tiene ninguna funcionalidad.

Lo siguiente que vamos a hacer es empezar a crear nuestro propio hook en **hooks/index.js**.

El hook es una función que vamos a llamar **useInputForm** y exportaremos para usarlo en nuestra aplicación.

/reactjs-hooks-custom-hook-lab/src/hooks/index.js

```
export const useInputForm = () => {  
  
}
```

Dentro de la función vamos a crear un estado que inicializaremos con un valor que vamos a recibir como parámetro en la función del hook.

/reactjs-hooks-custom-hook-lab/src/hooks/index.js

```
import { useState } from 'react';  
  
export const useInputForm = (initValue) => {  
  const [value, setValue] = useState(initValue)  
}
```

Con esto ya tendríamos el valor de cada input, pero para poder pasarlo al input correspondiente vamos a devolver:

- El **value**, que añadiremos como valor del input
- Y un método **onChange**, que se encargará de cambiar el estado cuando se detecte dicho evento en el input.

/reactjs-hooks-custom-hook-lab/src/hooks/index.js

```
import { useState } from 'react';  
  
export const useInputForm = (initValue, regexps) => {  
  const [value, setValue] = useState(initValue)  
  
  return {  
    value,  
    onChange: (e) => setValue(e.target.value)  
  }  
}
```


Ahora vamos a añadir estas propiedades al campo de texto que tenemos en el componente, y lo primero que hay que hacer es importar el hook y ejecutarlo para obtener las propiedades que hemos devuelto.

/reactjs-hooks-custom-hook-lab/src/components/Form.jsx

```
import React from 'react'
import { useInputForm } from '../hooks'

const Form = () => {
  const hexadecimal = useInputForm('#ffffff');

  return (
    <div>
      <input type="text" value={hexadecimal.value} onChange={hexadecimal.onChange} />
      <p>Válido o inválido</p>
    </div>
  )
}

export default Form
```

Con esto ya deberíamos de poder modificar el valor del campo de texto.

Pero no nos vamos a quedar aquí, ahora vamos a añadir algunas validaciones al campo. Estas validaciones las pasaremos como segundo parámetro del hook. Dentro de nuestro hook usaremos **useEffect** para pasar todas las validaciones cada vez que el **value** cambie de valor.

Tenemos que añadir un nuevo estado para controlar si el campo es válido o no lo es. Este valor lo vamos a devolver para poder mostrar el mensaje correcto en el campo de texto.

/reactjs-hooks-custom-hook-lab/src/hooks/index.js

```
import { useState, useEffect } from 'react';

export const useInputForm = (initValue, regexps) => {
  const [value, setValue] = useState(initValue)
  const [isValid, setValid] = useState(false)

  useEffect(() => {
    const valid = regexps.every(r => {
      const rexp = new RegExp(r);
      return rexp.test(value);
    })
    setValid(valid)
  }, [value])

  return {
    value,
    onChange: (e) => setValue(e.target.value),
```

```
    isValid
  }
}
```

Una vez que tenemos el hook, vamos a pasar las validaciones a nuestro hook, y mostraremos el mensaje que indica si las pasa o no.

/reactjs-hooks-custom-hook-lab/src/components/Form.jsx

```
import React from 'react'
import { useInputForm } from '../hooks'

const Form = () => {
  const hexadecimal = useInputForm('#ffffff', ['^#.*$', '^#([0-9A-Fa-f]{3}|[0-9A-Fa-f]{6})$']);

  return (
    <div>
      <input type="text" value={hexadecimal.value} onChange={hexadecimal.onChange} />
      <p>{ hexadecimal.isValid ? 'Válido' : 'Inválido' }</p>
    </div>
  )
}

export default Form
```

Una vez añadidas las validaciones, ya debe de mostrarnos si el valor introducido es válido.

Chapter 29. Redux

Redux es una librería que surge para entender como fluyen los cambios en los datos a través de la aplicación. Redux está influenciado por el patrón **Flux** propuesto por el equipo de Facebook.

Redux está pensado sobre todo para usarlo con React, pero no impide usarlo con cualquier otra librería o framework como podría ser Angular, Backbone...

Redux tiene **actions**, **action creators**, un **store** y **action objects** que usa para cambiar el **estado** de la aplicación. También introduce los **reducers** que son funciones puras que devuelven el **nuevo estado** basándose en los argumentos que recibe (el **estado actual** y una **action**).

Redux sigue 3 principios:

- El **estado** de la aplicación se almacena en un único objeto global inmutable.
- Los cambios en el estado se definen a través de las **actions**.
- Los **reducers** son los encargados de realizar los cambios al estado.

29.1. Instalación

Para instalar **Redux** en nuestro proyecto, vamos a lanzar el siguiente comando:

```
$ npm install redux
```

29.2. Estado

El **estado** de la aplicación es cualquier información necesaria para representar la interfaz de usuario en un momento dado. Pueden ser desde **datos externos** que podemos obtener del servidor al estado de la interfaz de usuario como si un modal se está mostrando o si un menú está desplegado.

Imaginad que tenemos una aplicación con varios componentes en la que cada componente tiene su propio estado, va a funcionar bien. Pero si empieza a crecer puede ser un poco difícil de seguir todas las actualizaciones o cambios ya que hay que ir mirando en cada componente el estado que tiene. Redux propone tener todos los estados en un mismo lugar, en un objeto global inmutable, de esta forma va a resultar mucho más fácil de ver todos los datos de la aplicación.

Cuando vamos a construir aplicaciones usando Redux, lo primero en lo que hay que pensar es en definir la estructura que vamos a necesitar para guardar todos los datos del estado (en un objeto JavaScript).

29.3. Actions

El primer principio que sigue Redux es el de tener el estado en un objeto global inmutable, pero el estado se va a ir actualizando con las acciones de los usuarios. Al ser inmutable no se puede modificar su valor, por lo que en lugar de hacer eso, se va a reemplazar por un nuevo estado. Para poder realizar estos cambios, vamos a necesitar las **actions**. Las actions son instrucciones que describen los cambios que se van a realizar.

Lo primero de todo es identificar las acciones que vamos a tener en la aplicación y definir el tipo de acción en un archivo de constantes.

```
export const ACCION = 'ACCION';
```

Todas las **actions** van a ser objetos, que como mínimo van a tener un atributo **type** cuyo valor va a ser una de las constantes que se han declarado antes y va a indicar que tiene que ocurrir.

Algunas actions necesitan saber más datos a parte de la action para poder cambiar el estado. Estos datos se conocen como **payload** y se añaden en el mismo objeto JavaScript.

29.3.1. Action creators

Los **action objects** son objetos JavaScript que definen como son las acciones. Y los **action creators** son funciones que crean y devuelven esos action objects, de esta forma usamos el action creator para crear la action que necesita emitir el método **dispatch**.

```
import { ACCION } from './action-types';

export function accion1() {
  return {
    type: ACCION
  }
}
```

29.4. Reducers

Los **reducers** son funciones puras que cogen el **estado actual** y una **action** y los usan para crear el **nuevo estado** y devolverlo. Este nuevo estado se inyecta en los componentes para refrescar la interfaz de usuario con los nuevos datos.

```
import { ACCION } from './action-types';

export default function reducer(state = 'hola mundo', action) {
  switch (action.type) {
    case ACCION:
      return state + '!';
    default:
      return state;
  }
}
```

29.5. Store

El **store** es la parte que se encarga de almacenar el estado de la aplicación y gestiona todas las actualizaciones de estado. El patrón Flux permitía tener más de un store en la aplicación, pero con Redux solo vamos a tener uno para la aplicación entera.

El store va a gestionar las actualizaciones del estado pasando el estado actual y la action a través de un reducer que va a resultar de la combinación de varios reducers más pequeños que se centran en parte más concretas. En principio, cuando se crea el **store**, este solo recibe un reducer como argumento. De esta forma se pierde la modularidad, porque al recibir solo un reducer, tendríamos que añadir en ese reducer toda la funcionalidad para manejar el estado. Pero nos permite evitar eso combinando todos los reducers en uno solo que va a ser el que le pasemos como argumento.

Para crear el store se va a usar la función **createStore(reducer, initial_state)** el cual puede recibir (opcionalmente) un segundo parámetro que sería un estado inicial.

```
import { createStore } from 'redux';
import miReducer from './contador';

export default function configStore() {
  return createStore(miReducer);
}
```

Y para combinar los distintos reducers en uno solo se usa el método **combineReducers()** que recibe como argumento un objeto con los reducers a combinar.

```
import { createStore, combineReducers } from 'redux';
import reducer1 from './red1';
import reducer2 from './red2';

export default function configStore() {
  return createStore(combineReducers({
    reducer1,
    reducer2
  }));
}
```


29.5.1. Funciones del Store

El objeto **store** que creamos con Redux nos da acceso a una serie de funciones para poder utilizar las funcionalidades de esta librería.

Estas funciones son:

- **getState()**: con esta función obtenemos el estado actual del store.
- **dispatch(actionObject)**: con esta función despachamos una acción para cambiar el estado de la aplicación.
- **subscribe(fn)**: el subscribe nos permite ejecutar la función de callback **fn** cada vez que cambia el estado que almacena el store. Esta función nos devuelve la referencia a una función para **eliminar la suscripción**.

```
const App = ({store}) => {
  useEffect(() => {
    const unsubscribe = store.subscribe(() => {
      console.log(store.getState())
    })

    return unsubscribe;
  }, [])

  return (
    <div>
      <button type="button" onClick={() => store.dispatch({type: 'ACCION'})}>Emitir acción</button>
    </div>
  )
}

export default App
```

29.6. Lab: Redux

En este laboratorio vamos a ver como utilizar la librería de Redux para gestionar el estado de un contador.

Vamos a copiar la plantilla del proyecto que se ha realizado en el laboratorio **reactjs-proyecto-inicial-lab** y vamos a cambiarle el nombre del proyecto a **reactjs-redux-lab**.



Es posible que sea necesario volver a instalar las dependencias del proyecto para que funcione correctamente. Para ello solo tenemos que lanzar el comando **npm install** dentro de la carpeta del proyecto.

Para poder usar **redux** vamos a necesitar instalar la siguiente dependencia en nuestro proyecto:

```
$ npm install redux
```

Ahora vamos a levantar el servidor de desarrollo con el comando:

```
$ npm start
```

Vamos a empezar creando la estructura del contador dentro de nuestro componente **App**.

/reactjs-redux-lab/src/components/App.jsx

```
const App = () => {  
  return (  
    <div>  
      <button>-</button>  
      <span>Cuenta: 0</span>  
      <button>+</button>  
    </div>  
  )  
}  
  
export default App
```

Ahora, vamos a empezar a crear los archivos que necesitaremos para utilizar Redux en nuestro proyecto.

Empezamos creando una carpeta **store** dentro de la carpeta **src**. Dentro de esta carpeta vamos a generar la siguiente estructura de archivos y carpetas:

- store/config-store.js
- store/contador/action-types.js
- store/contador/actions.js
- store/contador/index.js

Vamos a empezar definiendo los distintos tipos de acciones que vamos a poder despachar. Esto lo haremos en el archivo de **action-types.js**, en el que vamos a crear una serie de constantes que exportaremos para usarlas luego más tarde en el reducer y en el archivo de **actions.js**.

/reactjs-redux-lab/src/store/contador/action-types.js

```
export const INCREMENTAR = 'INCREMENTAR';
export const DECREMENTAR = 'DECREMENTAR';
```

Ahora vamos a crear las funciones **action creators** que se encargarán de crear los objetos de acciones que vamos a despachar.

Nos vamos al archivo **actions.js** en el que exportaremos dos funciones, una que devolverá la acción para incrementar la cuenta, y la otra para decrementarla.

/reactjs-redux-lab/src/store/contador/actions.js

```
import { INCREMENTAR, DECREMENTAR } from "./action-types"

export function incrementar() {
  return {
    type: INCREMENTAR,
  }
}

export function decrementar() {
  return {
    type: DECREMENTAR,
  }
}
```

Una vez tenemos este archivo, vamos a rellenar el **index.js** donde vamos a poner nuestro **reducer**, es decir, una función que va a recibir el estado actual y una acción, y donde vamos a crear el nuevo estado con los cambios indicados por la acción que llega. Esta función tiene que retornar el nuevo estado.

/reactjs-redux-lab/src/store/contador/index.js

```
import { DECREMENTAR, INCREMENTAR } from "./action-types";

export default function contador(state = 0, action) {
  switch (action.type) {
    case INCREMENTAR:
      return state + 1;
    case DECREMENTAR:
      return state - 1;
    default:
      return state;
  }
}
```

```
}
```

Con esto ya tenemos la parte de las acciones y el reducer, ahora solo tenemos que crear el **store**, y esto lo haremos en el archivo **config-store.js**.

Dentro de este archivo exportaremos otra función que va a retornar el store creado con la función **createStore** de Redux a la que le vamos a pasar nuestro **reducer**.

/reactjs-redux-lab/src/store/config-store.js

```
import { createStore } from 'redux'
import contadorReducer from './contador'

export const configStore = () => {
  return createStore(contadorReducer)
}
```

El siguiente paso es crear el store y pasarlo como propiedad a los componentes. En nuestro caso lo tenemos fácil porque solo tenemos un componente, por tanto, dentro del archivo **src/index.js** llamaremos a esta función que acabamos de crear y le pasaremos el **store** como propiedad al componente **App**.

/reactjs-redux-lab/src/index.js

```
import { createRoot } from 'react-dom/client';
import App from './components/App';
import './style.css';
import { configStore } from './store/config-store';

const store = configStore();

createRoot(document.getElementById('root')).render(<App store={store} />);
```

Ahora vamos a empezar por mostrar la cuenta actual en el componente **App**.

Como recibimos el store como propiedad, podemos pedir el estado actual llamando a la función **getState** de este y mostrarlo donde habíamos puesto el 0 dentro de la etiqueta **span**.

/reactjs-redux-lab/src/components/App.jsx

```
const App = ({store}) => {

  const cuenta = store.getState();

  return (
    <div>
      <button>-</button>
      <span>Cuenta: {cuenta}</span>
      <button>+</button>
    </div>
  )
}
```

```

    </div>
  )
}

export default App

```

Ahora vamos a importar las funciones de los **action creators** en el componente, y vamos a añadir dos funciones a los botones con las que ejecutaremos estos action creators para obtener las acciones a despachar.

/reactjs-redux-lab/src/components/App.jsx

```

import { incrementar, decrementar } from "../store/contador/actions";

const App = ({store}) => {

  const cuenta = store.getState();

  const incrementarCuenta = () => {
    const actionObj = incrementar();
  }

  const decrementarCuenta = () => {
    const actionObj = decrementar();
  }

  return (
    <div>
      <button type="button" onClick={decrementarCuenta}>-</button>
      <span>Cuenta: {cuenta}</span>
      <button type="button" onClick={incrementarCuenta}>+</button>
    </div>
  )
}

export default App

```

Ahora nos toca despachar estas dos acciones, y para ello, usaremos la función **dispatch** que nos ofrece el **store**.

/reactjs-redux-lab/src/components/App.jsx

```

import { incrementar, decrementar } from "../store/contador/actions";

const App = ({store}) => {

  const cuenta = store.getState();

  const incrementarCuenta = () => {
    const actionObj = incrementar();

```

```

    store.dispatch(actionObj);
  }

  const decrementarCuenta = () => {
    const actionObj = decrementar();
    store.dispatch(actionObj);
  }

  return (
    <div>
      <button type="button" onClick={decrementarCuenta}>-</button>
      <span>Cuenta: {cuenta}</span>
      <button type="button" onClick={incrementarCuenta}>+</button>
    </div>
  )
}

export default App

```

Si pulsamos sobre los botones, vemos que la cuenta no se actualiza. Esto se debe a que no nos hemos suscrito a los cambios de estado del store, por lo que no estamos recibiendo el nuevo estado cuando cambia.

Entonces, el siguiente paso es añadir la suscripción a estos cambios dentro del hook de **useEffect**, además tendremos que eliminar la suscripción cuando el componente se vaya a destruir, por lo que devolveremos la función que nos retorna el **store.subscribe**.

/reactjs-redux-lab/src/components/App.jsx

```

import { useEffect } from 'react';
import { incrementar, decrementar } from "../store/contador/actions";

const App = ({store}) => {
  const cuenta = store.getState();

  useEffect(() => {
    const unsubscribe = store.subscribe(() => {
      console.log(store.getState())
    })

    return unsubscribe;
  }, [])

  const incrementarCuenta = () => {
    const actionObj = incrementar();
    store.dispatch(actionObj);
  }

  const decrementarCuenta = () => {
    const actionObj = decrementar();
  }
}

```

```

    store.dispatch(actionObj);
  }

  return (
    <div>
      <button type="button" onClick={decrementarCuenta}>-</button>
      <span>Cuenta: {cuenta}</span>
      <button type="button" onClick={incrementarCuenta}>+</button>
    </div>
  )
}

export default App

```

Ahora que ya estamos recibiendo los cambios de estado, vamos a crear un estado para almacenar el valor de la cuenta y lo cambiaremos por el nuevo dentro de la suscripción.

/reactjs-redux-lab/src/components/App.jsx

```

import { useEffect, useState } from 'react';
import { incrementar, decrementar } from "../store/contador/actions";

const App = ({store}) => {
  const [cuenta, setCuenta] = useState(0)

  useEffect(() => {
    const unsubscribe = store.subscribe(() => {
      setCuenta(store.getState())
    })

    return unsubscribe;
  }, [])

  const incrementarCuenta = () => {
    const actionObj = incrementar();
    store.dispatch(actionObj);
  }

  const decrementarCuenta = () => {
    const actionObj = decrementar();
    store.dispatch(actionObj);
  }

  return (
    <div>
      <button type="button" onClick={decrementarCuenta}>-</button>
      <span>Cuenta: {cuenta}</span>
      <button type="button" onClick={incrementarCuenta}>+</button>
    </div>
  )
}

```

```
}
```

```
export default App
```

Ahora ya podemos ver que la cuenta se actualiza cada vez que pulsamos los botones.

Chapter 30. React-Redux

La librería **react-redux** nos permite reducir la complejidad a la hora de integrar Redux dentro de una aplicación de React.

30.1. Instalación

Para instalar **react-redux** en nuestro proyecto, vamos a lanzar el siguiente comando:

```
$ npm install react-redux
```

30.2. Provider

Esta librería nos provee de un componente **Provider** que recibe como propiedad el store y da acceso al store a todos los componentes que están por debajo de él en el árbol de componentes. De esta forma no tenemos que pasar, como propiedad, el store a todos los componentes para que estos tengan acceso a él.

```
import { createRoot } from 'react-dom/client'
// Importaciones
import { Provider } from 'react-redux';

const store = configStore();
createRoot(document.getElementById('root')).render(<Provider store={store}><App /></Provider>);
```

30.3. connect

Para poder acceder al Store y poder recibir el estado y emitir acciones, vamos a usar el método **connect()** que recibe dos parámetros, el primero **mapStateToProps** y el segundo **mapDispatchToProps**. Este método va a inyectarle al componente que indiquemos el estado y emisores de acciones que necesitemos.

- **mapStateToProps** es una función que recibe como parámetro el **estado** y lo mapea a props que va a pasar a nuestro componente. Tiene que devolver un objeto con la parte del estado que queremos usar en el componente.
- **mapDispatchToProps** es una función que recibe como parámetro el método **dispatch** y que devuelve un objeto con las funciones que van a emitir las acciones que nos devuelven los **action creators**, y lo va a pasar como propiedades al componente. En lugar de una función, podemos asignarle a **mapDispatchToProps** un objeto con los **actions creators** y el se encarga de emitir las acciones correctas.

Decir también que la ejecución del método **connect** devuelve un **HOC** (Higher Order Component) encargado de pasarle estos datos como propiedades al componente que se le va a pasar como parámetro.

```
import { accion1, accion2 } from 'store/app/actions';
import { connect } from 'react-redux';

const Cmp = (props) => {
  return (
    <div>
      <p>{props.dato1}</p>
      <button onClick={props.accion1}>Despachar acción 1</button>
      <button onClick={() => props.accion2(192)}>Despachar acción 2</button>
    </div>
  )
}

const mapStateToProps = (state) => {
  return {
    dato1: state.dato1
  }
}

// const mapDispatchToProps = (dispatch) => {
//   return {
//     accion1: () => dispatch(accion1()),
//     accion2: (n) => dispatch(accion2(n))
//   }
// }

const mapDispatchToProps = {
  return {
    accion1,
```

```
    accion2
  }
}

export default connect(mapStateToProps, mapDispatchToProps)(Cmp);
```

30.4. Hooks de react-redux

Con la llegada de los React Hooks muchas librerías se han actualizado para permitir usar la funcionalidad de ellas usando este tipo de funciones.

En el caso de React Redux se han añadido dos hooks que nos van a permitir sustituir a la función **connect** por ellos.

Estos hooks son:

- **useSelector**: este hook se encarga de obtener parte del estado que se almacena en el store. Este hook recibe como parámetro una función en la que obtenemos el estado y con la que tenemos que devolver un objeto con la parte de este que queramos usar en el componente.
- **useDispatch**: este hook nos devuelve una referencia a la función **dispatch** con la que vamos a poder despachar las acciones que nos devuelven los **action creators**.
- **useStore**: este hook nos devuelve el objeto **store** de Redux para trabajar directamente con las funciones que nos proporciona, como por ejemplo, **getState** o **dispatch**.

```
import { useSelector, useDispatch, useStore } from 'react-redux';
import { accion1, accion2 } from 'store/app/actions';

const Cmp = () => {
  const store = useStore();
  console.log(store.getState());

  const dato1 = useSelector(state => state.dato1);
  const dispatch = useDispatch();

  return (
    <div>
      <p>{dato1}</p>
      <button onClick={() => dispatch(accion1())}>Despachar acción 1</button>
      <button onClick={() => dispatch(accion2(192))}>Despachar acción 2</button>
    </div>
  )
}

export default Cmp;
```

30.5. Lab: react-redux

En este laboratorio vamos a ver como utilizar la librería de Redux para gestionar el estado de un contador pero esta vez utilizando la librería react-redux.

Vamos a copiar la plantilla del proyecto que se ha realizado en el laboratorio **reactjs-proyecto-inicial-lab** y vamos a cambiarle el nombre del proyecto a **reactjs-react-redux-lab**.



Es posible que sea necesario volver a instalar las dependencias del proyecto para que funcione correctamente. Para ello solo tenemos que lanzar el comando **npm install** dentro de la carpeta del proyecto.

Para poder usar **redux** y **react-redux** vamos a necesitar lanzar los siguientes comandos dentro de la carpeta del proyecto:

```
$ npm install redux
$ npm install react-redux
```

Ahora vamos a levantar el servidor de desarrollo con el comando:

```
$ npm start
```

Vamos a empezar creando la estructura del contador dentro de nuestro componente **App**.

/reactjs-react-redux-lab/src/components/App.jsx

```
const App = () => {
  return (
    <div>
      <button>-</button>
      <span>Cuenta: 0</span>
      <button>+</button>
    </div>
  )
}

export default App
```

Ahora, vamos a empezar a crear los archivos que necesitaremos para utilizar Redux en nuestro proyecto.

Empezamos creando una carpeta **store** dentro de la carpeta **src**. Dentro de esta carpeta vamos a generar la siguiente estructura de archivos y carpetas:

- store/config-store.js
- store/contador/action-types.js

- store/contador/actions.js
- store/contador/index.js

Vamos a empezar definiendo los distintos tipos de acciones que vamos a poder despachar. Esto lo haremos en el archivo de **action-types.js**, en el que vamos a crear una serie de constantes que exportaremos para usarlas luego más tarde en el reducer y en el archivo de **actions.js**.

/reactjs-react-redux-lab/src/store/contador/action-types.js

```
export const INCREMENTAR = 'INCREMENTAR';
export const DECREMENTAR = 'DECREMENTAR';
```

Ahora vamos a crear las funciones **action creators** que se encargarán de crear los objetos de acciones que vamos a despachar.

Nos vamos al archivo **actions.js** en el que exportaremos dos funciones, una que devolverá la acción para incrementar la cuenta, y la otra para decrementarla.

/reactjs-react-redux-lab/src/store/contador/actions.js

```
import { INCREMENTAR, DECREMENTAR } from "../action-types"

export function incrementar() {
  return {
    type: INCREMENTAR,
  }
}

export function decrementar() {
  return {
    type: DECREMENTAR,
  }
}
```

Una vez tenemos este archivo, vamos a rellenar el **index.js** donde vamos a poner nuestro **reducer**, es decir, una función que va a recibir el estado actual y una acción, y donde vamos a crear el nuevo estado con los cambios indicados por la acción que llega. Esta función tiene que retornar el nuevo estado.

/reactjs-react-redux-lab/src/store/contador/index.js

```
import { DECREMENTAR, INCREMENTAR } from "../action-types";

export default function contador(state = 0, action) {
  switch (action.type) {
    case INCREMENTAR:
      return state + 1;
    case DECREMENTAR:
      return state - 1;
  }
}
```



```

    default:
      return state;
  }
}

```

Con esto ya tenemos la parte de las acciones y el reducer, ahora solo tenemos que crear el **store**, y esto lo haremos en el archivo **config-store.js**.

Dentro de este archivo exportaremos otra función que va a retornar el store creado con la función **createStore** de Redux a la que le vamos a pasar nuestro **reducer**.

/reactjs-react-redux-lab/src/store/config-store.js

```

import { createStore } from 'redux'
import contadorReducer from './contador'

export const configStore = () => {
  return createStore(contadorReducer)
}

```

El siguiente paso es crear el store y pasarlo como propiedad a los componente. Aquí vamos a envolver nuestro componente **App** con el componente **Provider** de react-redux al cual le pasaremos el **store** y se encargará de hacerlo llegar a todos los componentes donde vayamos a necesitarlo.

/reactjs-react-redux-lab/src/index.js

```

import { createRoot } from 'react-dom/client';
import App from './components/App';
import { Provider } from 'react-redux';
import './style.css';
import { configStore } from './store/config-store';

const store = configStore();

createRoot(document.getElementById('root')).render(<Provider store={store}><App /></Provider>);

```

Ahora vamos a obtener el estado dentro del componente **App** para mostrarlo en lugar del 0 que hemos puesto hardcodeado anteriormente.

Necesitaremos llamar al hook de **useSelector** y retornar el estado del store, lo que nos devolverá la cuenta actual.

/reactjs-react-redux-lab/src/components/App.jsx

```

import { useSelector } from 'react-redux';

const App = () => {

  const cuenta = useSelector(state => state);

```

```

return (
  <div>
    <button>-</button>
    <span>Cuenta: {cuenta}</span>
    <button>+</button>
  </div>
)
}

export default App

```

Ahora nos toca añadir los eventos de **click** a los botones, con los que vamos a llamar a dos funciones que vamos a crear desde las que despacharemos las acciones.

/reactjs-react-redux-lab/src/components/App.jsx

```

import { useSelector } from 'react-redux';
import { incrementar, decrementar } from '../store/contador/actions';

const App = () => {
  const cuenta = useSelector(state => state);

  const incrementarCuenta = () => {
    const actionObj = incrementar();
  }

  const decrementarCuenta = () => {
    const actionObj = decrementar();
  }

  return (
    <div>
      <button type="button" onClick={decrementarCuenta}>-</button>
      <span>Cuenta: {cuenta}</span>
      <button type="button" onClick={incrementarCuenta}>+</button>
    </div>
  )
}

export default App

```

Para poder despachar las acciones que nos devuelven los action creators, vamos a necesitar ejecutar la función **dispatch** que nos devuelve el hook **useDispatch**. Por lo que lo importamos y ejecutaremos **dispatch(actionObj)** dentro de ambas funciones a las que se llaman desde los botones.

/reactjs-react-redux-lab/src/components/App.jsx

```

import { useSelector, useDispatch } from 'react-redux';

```

```
import { incrementar, decrementar } from '../store/contador/actions';

const App = () => {
  const cuenta = useSelector(state => state);
  const dispatch = useDispatch()

  const incrementarCuenta = () => {
    const actionObj = incrementar()
    dispatch(actionObj)
  }

  const decrementarCuenta = () => {
    const actionObj = decrementar()
    dispatch(actionObj)
  }

  return (
    <div>
      <button type="button" onClick={decrementarCuenta}>-</button>
      <span>Cuenta: {cuenta}</span>
      <button type="button" onClick={incrementarCuenta}>+</button>
    </div>
  )
}

export default App
```

Y con esto ya tenemos nuestro contador funcionando.



Si comparamos este laboratorio con el de **Lab: Redux**, podemos ver que es mucho más fácil crear la misma aplicación utilizando la librería de **react-redux** que usando solo **redux**.

30.6. Middleware

El middleware en Redux nos va a permitir ejecutar piezas de código cuando se despachen las acciones. Se pueden combinar múltiples middleware, que se irán ejecutando en cadena, y ninguno de ellos sabrá cual se ha ejecutado antes o cual se va a ejecutar después. Estos middlewares se suelen usar para trabajar con acciones asíncronas.

Para poder usar el middleware vamos a importar **applyMiddleware** desde **redux**. Al método de **createStore** se le va a pasar como segundo parámetro el método **applyMiddleware** que llevará como parámetro todos los middlewares que queramos aplicar. Las funciones de middleware reciben como parámetro el **store**, y esta función tiene que devolver a su vez otra función la cual recibe como parámetro el método **next**. El método next sirve para llamar a la siguiente función de middleware. Esta función devuelve otra función más que recibe como parámetro el **action** que se va a pasar como parámetro a la siguiente función middleware. En esta última función podemos acceder, tanto al store como al action, por lo que podremos mostrar por consola el action que se ha disparado y cual es el nuevo estado.

```
import { createStore, applyMiddleware } from 'redux';
import miReducer from './reducer';

export default function configStore() {
  const logger = (store) => {
    return (next) => {
      return (action) => {
        console.log('Dispatched action: ', action);
        const res = next(action);
        console.log('Next state: ', store.getState());
        return res;
      }
    }
  }

  return createStore(miReducer, applyMiddleware(logger));
}
```

Y de esta forma, cada vez que se lance una acción, nos va a mostrar la acción despachada y el siguiente estado de la aplicación.

30.6.1. redux-thunk

En nuestras aplicaciones seguramente que vamos a necesitar ejecutar código asíncrono, por ejemplo para pedir los datos al servidor, y al usar redux puede no quedar muy claro como hacerlo. Esto es lo que vamos a ver con **redux-thunk**. Redux thunk es un middleware que nos va a permitir exactamente eso, ejecutar código asíncrono antes de despachar la acción. Para poder usarlo tenemos que instalar la dependencia:

```
$ npm install redux-thunk
```

Una vez instalado, como es un middleware, lo tendremos que pasar en la función de **applyMiddleware** que se encuentra en el archivo de configuración del store.

```
import { createStore, applyMiddleware } from 'redux';
import miReducer from './reducer';
import thunk from 'redux-thunk';

export default function configStore() {
  const logger = (store) => {
    return (next) => {
      return (action) => {
        console.log('Dispatched action: ', action);
        const res = next(action);
        console.log('Next state: ', store.getState());
        return res;
      }
    }
  }

  return createStore(miReducer, applyMiddleware(logger, thunk));
}
```

Redux-Thunk nos va a permite crear **action creators** que van a devolver una función en lugar de la acción, por lo que antes de que se modifique el estado se va a haber ejecutado ese código asíncrono. La función que devuelven estos action creators recibe como parámetros el método de **dispatch** para despachar la acción una vez haya terminado el código asíncrono de ejecutarse y el método de **getState** que nos da el estado actual del store. De esta forma una vez que ya tenemos los datos del servidor, usamos el método **dispatch** para despachar la acción (que devolverá otro action creator).

```
import * as ActionTypes from './action-types';

function accion() {
  return dispatch => {
    setTimeout(() => {
      dispatch(getAccionObj());
    }, 2000)
  }
}
```

```
}  
  
function getAccionObj() {  
  return {  
    type: ActionTypes.ACCION  
  }  
}  
  
export { accion };
```

Chapter 31. React Router

El routing es lo que nos va a permitir cambiar entre las distintas páginas de nuestra aplicación, pero recordad que estamos ante aplicaciones de una sola página (SPAs), es decir, aplicaciones que solo tienen una única página de HTML, por lo que no tiene sentido cambiar de página cada vez que cambiamos de URL. En este caso, el routing funciona un poco diferente a como se hacía con las MPAs (Multi-Page Applications), ahora, lo que se cambia es el contenido a mostrar por la página **index.html**, que serán nuestros componentes.

React al ser una librería y no un framework no lo trae consigo, pero para poder añadirlo a nuestras aplicaciones usaremos la librería de **React Router** que nos permitirá añadir rutas a nuestra aplicación y se encargará de cambiar entre los distintos componentes asociados a estas rutas.

31.1. Instalación

Para instalar **React Router** en nuestro proyecto hay que instalar el siguiente paquete de NPM:

```
$ npm install react-router-dom
```

Una vez instalada vamos a ver como funciona.

31.2. BrowserRouter

Una vez que se han instalado las librerías tenemos que permitir el uso del routing en nuestra aplicación para poder usar las funcionalidades que este nos va a proporcionar. Para ello tenemos que envolver toda nuestra aplicación con el componente **BrowserRouter** que importaremos desde el paquete de **react-router-dom**.

Este componente se va a encargar de interactuar con la API del historial del navegador para poder poner las URLs en la barra de direcciones, además de ir añadiendo/eliminando estas direcciones en la pila del historial.

/src/app.js

```
import { createRoot } from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './components/App';

createRoot(document.getElementById('root')).render(<BrowserRouter><App /><
/BrowserRouter>);
```

Una vez envuelta nuestra aplicación con ese componente, todos aquellos componentes que se encuentren por debajo de este en el árbol de componentes, serán capaces de usar las rutas.

31.3. Routes y Route

React Router nos proporciona dos componentes necesarios para definir las rutas de nuestra aplicación, el componente **Routes** y el **Route**.

Con el componente **Route** vamos a definir las distintas rutas de la aplicación. Este componente recibe como propiedades:

- **path**: el path con el que tiene que coincidir la url.
- **element**: el elemento HTML/JSX a renderizar cuando el path coincida con la URL a la que hemos navegado.

El componente **Routes** es un contenedor para los componentes **Route**, y de lo que se va a encargar es de recorrer todas las rutas que se le pasan como **children** para mostrar aquella que tenga la mayor coincidencia con la URL. Esto es algo nuevo que se ha añadido en esta última versión de react-router (la v6), ya que en las anteriores versiones se mostraban según el orden en el que se encontraban definidas, algo que puede llevar a darnos algún quebradero de cabeza.

```
import { Routes, Route } from 'react-router-dom'

const App = (props) => {
  return (
    <div>
      <Routes>
        <Route path="/" element={<Inicio />} />
        <Route path="/fin" element={<Fin />} />
      </Routes>
    </div>
  )
}

export default App
```

Necesitamos añadir la opción **devServer.historyApiFallback: true** en la configuración de webpack para que funcione correctamente al cambiar entre las distintas páginas.



Si no la añadimos, cada vez que se cambie la URL se hará una petición al servidor pidiendo un archivo nuevo de HTML para descargar. El problema es que solo tenemos un único index.html en nuestra aplicación. Con esta opción, le estamos indicando que devuelva ese único archivo para todas las peticiones que le lleguen al servidor.

31.4. Link

Cuando queremos cambiar de ruta en nuestra aplicación podemos utilizar la etiqueta **a** de HTML, pero nos encontramos con un problema, y es que cuando cambiamos de página con esta etiqueta, se refresca la aplicación, cargandose desde 0 el index.html y por tanto perdiendo el estado que no se hubiese guardado hasta ese momento de pulsar el enlace.

React Router nos proporciona un componente **Link** que se va a encargar de evitar refrescar la página al cambiar de una ruta a otra. Este componente pinta un enlace normal de HTML, solo que cuando se pulsa sobre el no refresca la página. Le indicamos a donde queremos navegar con la propiedad **to**.

```
import { Link } from 'react-router-dom'

const Header = (props) => {
  return (
    <ul>
      <li>
        <Link to="/">Inicio</Link>
        <Link to="/otra-pagina">Otra página</Link>
      </li>
    </ul>
  )
}

export default Header
```



Podemos hacer que refresque la página, es decir, que actúe como un enlace normal y corriente, si le añadimos la propiedad **reloadDocument**.

31.5. Navegación por código (useNavigate)

Hemos visto como navegar usando los componentes **Link** de React Router, pero hay veces en las que queremos cambiar de componente una vez se ha realizado una acción, como por ejemplo, cuando guardamos los datos de un formulario. Esta navegación la haríamos a través de código.

Para realizar la navegación por código, tendremos que utilizar el hook de **useNavigate** que nos proporciona React Router. Este hook es una función que nos va a devolver una función para realizar la navegación a otra ruta.

Esta función se puede usar de dos formas:

- Le pasamos la ruta a la que queremos navegar como parámetro.
- O le pasamos un número (llamado **delta**) con el que le indicamos cuantas páginas queremos avanzar o retroceder en la pila del historial, por ejemplo, pasarle un -1 significa que nos vamos a la página anterior.

```
import { useNavigate } from 'react-router-dom'

const App = (props) => {
  const navigate = useNavigate()

  return (
    <div>
      <button onClick={() => navigate('/mi-perfil')}>Mi perfil</button>
      <button onClick={() => navigate(-1)}>Volver atrás</button>
    </div>
  )
}

export default App
```

31.6. Rutas con parámetros

Cuando tenemos listados de datos, es muy probable que necesitemos ver más en detalle la información de cada elemento en la lista, y aquí es donde entran las rutas con parámetros. Estas rutas tienen una parte dinámica que va a cambiar dependiendo de los datos que queramos mostrar, donde normalmente se usa el identificador del dato.

Para indicar que una ruta tiene un valor dinámico o parámetro vamos a utilizar `:` delante del nombre del parámetro en el path.

```
import { Routes, Route } from 'react-router-dom'

const App = (props) => {
  return (
    <div>
      <Routes>
        <Route path="/noticias/:id" element={<Noticia />} />
      </Routes>
    </div>
  )
}

export default App
```

De esta forma ya podemos crear rutas como:

- /noticias/1
- /noticias/12893
- /noticias/-M73ljd39udHS92_J0ss2
- ...

Luego en el componente asociado a dicha ruta necesitaremos recuperar el valor del parámetro, y aquí entra el hook **useParams** de React Router, con el que obtendremos un objeto de JS con el nombre del parámetro como clave, y asociado a este, el valor que le hemos puesto en la ruta.

```
import { useParams } from 'react-router-dom'

const Noticia = (props) => {
  const { id } = useParams()

  return (
    <div>
      ...
    </div>
  )
}
```


31.7. Rutas anidadas

Por ahora solo hemos creado rutas que se encuentran al mismo nivel, pero en algunos casos vamos a necesitar mostrar componentes que comparten parte de la misma url, y en estos casos es cuando podremos usar las rutas anidadas.

Por ejemplo, podríamos usarlo con un listado de datos para mostrar al mismo tiempo tanto la lista como la información de alguno de estos datos:

- /usuarios: esta mostrará la lista de los usuarios.
- /usuarios/2: esta mostrará la información del usuario que tiene el id 2.

Otro caso podría ser en el que tenemos un listado de series/películas y podemos filtrarlas por el género:

- /categorias: esta mostrará la lista de las categorías.
- /categorias/comedia: esta mostrará las series/películas de la categoría comedia.
- /categorias/comedia/-M1s1hXmn6_UopYgiIy6: esta mostrará la información de la serie/película que tiene el id -M1s1hXmn6_UopYgiIy6 y pertenece a la categoría comedia.

Con las rutas anidadas conseguiremos mostrar los componentes asociados a cada parte de la ruta al mismo tiempo, por lo que tendremos que añadir los componentes **Route** anidados según vayamos a definir las rutas.

```
import { Routes, Route } from 'react-router-dom'

const App = (props) => {
  return (
    <div>
      <Routes>
        <Route path="/" element={<Layout />}>
          <Route path="categorias" element={<ListaCategorias />}>
            <Route path=":categoria" element={<Categoria />}>
              <Route path=":serieId" element={<Serie />} />
            </Route>
          </Route>
        </Route>
      </Routes>
    </div>
  )
}

export default App
```

Una vez definidas las rutas, en aquellas que hemos puesto como rutas anidadas tendremos que indicar en que posición dentro del elemento que se va a renderizar se tiene que renderizar el elemento perteneciente a la ruta anidada. Para esto usaremos el componente **Outlet** que viene de la librería **react-router-dom**.

```
import { Outlet, Link } from 'react-router-dom'

const ListaCategoria = (props) => {
  return (
    <div>
      <h2>Categorias</h2>
      <ul>
        <li><Link to="/categorias/comedia">Comedia</Link></li>
        <li><Link to="/categorias/terror">Terror</Link></li>
        <li><Link to="/categorias/drama">Drama</Link></li>
      </ul>
      <div>
        <Outlet />
      </div>
    </div>
  )
}

export default ListaCategoria
```


31.8. Redirigir rutas

El componente **Navigate** de React Router se encarga de cambiar de ruta en el momento en el que se pinta, por lo que podemos encontrarle varios usos:

- Nos puede servir para redirigir de una ruta a otra.
- Nos servirá para cambiar de rutas dentro de los componentes de clase, ya que en ellos no podemos utilizar el hook de **useNavigate**.

```
import { Routes, Route, Navigate } from 'react-router-dom'

const App = (props) => {
  return (
    <div>
      <Routes>
        <Route path="/" element={<Inicio />} />
        <Route path="/redirigeme-al-inicio" element={<Navigate to="/" replace={true} />} />
      </Routes>
    </div>
  )
}

export default App
```

31.9. Guards

Las **guardas** nos permiten entrar en los componentes o no hacerlo dependiendo de alguna condición. Por ejemplo, en una aplicación solo se podrá ver la información de un usuario en el caso de que este usuario se haya logueado. Para añadir una guarda a nuestra aplicación es tan sencillo como renderizar la ruta o no hacerlo dependiendo de la condición.

```
import { Routes, Route } from 'react-router-dom'
import NuevoProyecto from './components/NuevoProyecto'

const App = (props) => {
  const [logueado, setLogueado] = useState(false)

  return (
    <div>
      <Routes>
        <Route path="/" element={<Inicio />} />
        {logueado && <Route path="/crear-proyecto" element={<NuevoProyecto />} />}
      </Routes>
    </div>
  )
}

export default App
```

De esta forma, como **logueado** es false, la ruta no se renderiza por lo que el componente asociado a ella no se puede mostrar y no se cambia la url.

31.10. Ruta comodín

La **ruta comodín** es aquella que se va a ejecutar siempre que no se ejecute ninguna otra ruta. Esta ruta se suele usar para mostrar una página de error, o para redireccionar a la página de inicio, siempre que introduzcamos una URL que no existe en nuestra aplicación.

Para usar esta ruta, solo hay que poner el path `*` e indicar el componente a mostrar. Además, hay que tener en cuenta que siempre la tenemos que poner en la última posición dentro de las rutas, ya que se ejecutará solo si no se ha ejecutado antes de ella otra ruta.

```
import { Routes, Route } from 'react-router-dom';
import { Error } from './components/error';

const App = (props) => {
  return (
    <div>
      <Routes>
        {/* otras rutas */}
        <Route path="*" element={<Error />} />
      </Routes>
    </div>
  )
}

export default App;
```

31.11. Query Params

Otro tipo de parámetros con los que nos podemos encontrar en las rutas de la aplicación son los parámetros de consulta, o **query params**, que son aquellos que van detrás del **?** al final de la URL.

Para poder acceder a estos parámetros de query, React Router nos proporciona un hook **useSearchParams** que funciona de forma similar al **useState**, devolviendonos un array de dos posiciones, donde la primera va a contener los parámetros, y la segunda es un método para poder modificarlos.

```
import { useSearchParams } from 'react-router-dom'

const ListaDatos = (props) => {
  const [queryParams, setQueryParams] = useSearchParams()
  const categoria = queryParams.get('categoria')

  const datos = props.lista.filter(d => d.categoria === categoria).map(d => <li key={d.id}>{d.titulo}</li>)

  return (
    <ul>
      {datos}
    </ul>
  )
}

export default ListaDatos
```

Si quisiéramos modificar los queryParams, tendríamos que llamar a la función de **setQueryParams** pasándole como parámetro un objeto de JavaScript con los datos de los parámetros.

```
import { useSearchParams } from 'react-router-dom'

const Cmp = (props) => {
  const [queryParams, setQueryParams] = useSearchParams()

  const fn = () => {
    setQueryParams({
      param1: 'val1',
      param2: 'val2',
    })
  }

  return (
    <div>
      ...
    </div>
  )
}
```

```
export default Cmp
```

31.12. Lab: React Router

En este laboratorio vamos a ver como utilizar la librería de React Router con todas sus funcionalidades.

Vamos a copiar la plantilla del proyecto que se ha realizado en el laboratorio **reactjs-proyecto-inicial-lab** y vamos a cambiarle el nombre del proyecto a **reactjs-react-router-lab**.



Es posible que sea necesario volver a instalar las dependencias del proyecto para que funcione correctamente. Para ello solo tenemos que lanzar el comando **npm install** dentro de la carpeta del proyecto.

Necesitaremos instalar la dependencia de React Router en nuestro proyecto. Para ello, vamos a lanzar el comando:

```
$ npm install react-router-dom
```

Ahora vamos a levantar el servidor de desarrollo con el comando:

```
$ npm start
```

Vamos a empezar por crear dos componentes iniciales dentro de la carpeta **components**, uno al que vamos a llamar **Inicio** y otro **NuevoUsuario**.

/reactjs-react-router-lab/src/components/Inicio.jsx

```
const Inicio = () => {  
  return (  
    <div>  
      <h2>Inicio</h2>  
    </div>  
  )  
}  
  
export default Inicio
```

/reactjs-react-router-lab/src/components/NuevoUsuario.jsx

```
const NuevoUsuario = () => {  
  return (  
    <div>  
      <h2>Nuevo usuario</h2>  
    </div>  
  )  
}
```

```
export default NuevoUsuario
```

Una vez que tenemos nuestros dos primeros componentes, vamos a añadir las rutas iniciales para poder navegar entre ellos. En el componente **App** vamos a añadir los componentes **Route** para definir las rutas y que aquellos componentes que se tienen que mostrar.

/reactjs-react-router-lab/src/components/App.jsx

```
import { Routes, Route } from 'react-router-dom';
import Inicio from './Inicio';
import NuevoUsuario from './NuevoUsuario';

const App = () => {
  return (
    <div>
      <h1>React Router v6</h1>

      <Routes>
        <Route path="/" element={<Inicio />} />
        <Route path="/nuevo-usuario" element={<NuevoUsuario />} />
      </Routes>
    </div>
  )
}

export default App
```

Si probamos a entrar en las siguientes dos URLs podremos ver que los componentes se pintan correctamente:

- <http://localhost:8080>
- <http://localhost:8080/nuevo-usuario>

Vamos a añadir un par de enlaces para poder cambiar entre estas dos rutas sin tener que escribirlas a mano en la barra de direcciones del navegador.

Para ello vamos a crearnos un componente nuevo **Header** en el que pondremos dos enlaces que apuntarán a las dos rutas que hemos definido.

/reactjs-react-router-lab/src/components/Header.jsx

```
const Header = () => {
  return (
    <div>
      <a href="/">Inicio</a> |
      <a href="/nuevo-usuario">Nuevo usuario</a>
    </div>
  )
}
```

```
export default Header
```

Tenemos que importar y usar este componente en nuestro App para poder verlo en nuestra aplicación y así poder pulsar sobre los enlaces para navegar entre las rutas.

/reactjs-react-router-lab/src/components/App.jsx

```
import { Routes, Route } from 'react-router-dom';
import Header from './Header';
import Inicio from './Inicio';
import NuevoUsuario from './NuevoUsuario';

const App = () => {
  return (
    <div>
      <h1>React Router v6</h1>
      <Header />

      <Routes>
        <Route path="/" element={<Inicio />} />
        <Route path="/nuevo-usuario" element={<NuevoUsuario />} />
      </Routes>
    </div>
  )
}

export default App
```

Si probamos a cambiar entre las distintas páginas de la aplicación pulsando los enlaces vemos que funciona correctamente, pero con un pequeño pero... La página se refresca cada vez que cambiamos de ruta. Esto lo vamos a solucionar cambiando las etiquetas **a** por componentes **Link** en el componente **Header**.

/reactjs-react-router-lab/src/components/Header.jsx

```
import { Link } from 'react-router-dom';

const Header = () => {
  return (
    <div>
      <Link to="/">Inicio</Link> |
      <Link to="/nuevo-usuario">Nuevo usuario</Link>
    </div>
  )
}

export default Header
```

Ahora ya lo tenemos solucionado, ya no se refresca la página, por lo que no perderemos el estado

de la aplicación que no se haya llegado a guardar en algún sitio.

El siguiente paso es ir al componente **NuevoUsuario** y añadir un botón que navegue automáticamente a la página de inicio cuando se guarden los datos del formulario (algo que vamos a simular con un **setTimeout**).

/reactjs-react-router-lab/src/components/NuevoUsuario.jsx

```
const NuevoUsuario = () => {

  const guardar = (e) => {

  }

  return (
    <div>
      <h2>Nuevo usuario</h2>
      <button onClick={guardar}>Crear usuario</button>
    </div>
  )
}

export default NuevoUsuario
```

Para navegar desde el código, vamos a utilizar la función **navigate** que nos devuelve el hook de **useNavigate**. Por tanto vamos a poner esta llamada dentro de un **setTimeout** para que la navegación no sea inmediata al igual que ocurriría si estuviésemos haciendo una petición POST para guardar los datos en una BBDD.

/reactjs-react-router-lab/src/components/NuevoUsuario.jsx

```
import { useNavigate } from 'react-router-dom'

const NuevoUsuario = () => {
  const navigate = useNavigate()

  const guardar = (e) => {
    setTimeout(() => {
      navigate('/')
    }, 2000)
  }

  return (
    <div>
      <h2>Nuevo usuario</h2>
      <button onClick={guardar}>Crear usuario</button>
    </div>
  )
}
```

```
export default NuevoUsuario
```

Ahora ya podemos pulsar el botón y comprobar que cuando pasan 2 segundos, nos vamos hasta la página inicial.

Vamos a cambiar un poco las rutas de la aplicación, y vamos a empezar por cambiar la ruta inicial por **/usuarios** ya que es lo que vamos a mostrar al pintar el componente **Inicio**.

/reactjs-react-router-lab/src/components/App.jsx

```
import { Routes, Route } from 'react-router-dom';
import Header from './Header';
import Inicio from './Inicio';
import NuevoUsuario from './NuevoUsuario';

const App = () => {
  return (
    <div>
      <h1>React Router v6</h1>
      <Header />

      <Routes>
        <Route path="/usuarios" element={<Inicio />} />
        <Route path="/nuevo-usuario" element={<NuevoUsuario />} />
      </Routes>
    </div>
  )
}

export default App
```

Con este cambio nuestros enlaces han dejado de apuntar a la nueva ruta, y por tanto cuando vamos a la ruta / no vemos nada salvo el menú de navegación. Esto lo vamos a solucionar haciendo que cuando se pinte la ruta inicial, se haga una navegación a la ruta **/usuarios**.

Para realizar esto vamos a utilizar el componente **Navigate** de React Router que vamos usar como elemento del componente **Route**. Le vamos a pasar la propiedad **replace** con un valor **true** para que esta ruta intermedia no quede en el historial, sino que la reemplace por aquella a la que vamos a navegar.

/reactjs-react-router-lab/src/components/App.jsx

```
import { Routes, Route, Navigate } from 'react-router-dom';
import Header from './Header';
import Inicio from './Inicio';
import NuevoUsuario from './NuevoUsuario';

const App = () => {
  return (
    <div>
```

```

    <h1>React Router v6</h1>
    <Header />

    <Routes>
      <Route path="/usuarios" element={<Inicio />} />
      <Route path="/nuevo-usuario" element={<NuevoUsuario />} />
      <Route path="/" element={<Navigate to="/usuarios" replace={true} />} />
    </Routes>
  </div>
)
}

export default App

```



Aunque esto corrige el problema de los enlaces que van hasta /, no es la solución definitiva, tendríamos que cambiar las rutas para evitar esa redirección. La redirección solo debería de ejecutarse cuando entramos a la página inicial desde la barra de navegación.

Una vez tenemos esto, vamos a hacer que en el componente **Inicio** se cargue una lista de usuarios para mostrarlos en nuestra aplicación. Estos datos los vamos a obtener de la API de **jsonplaceholder**.

Por tanto, nos vamos al componente de Inicio, donde vamos a añadir como estado un array vacío que luego rellenaremos con los datos que vengan de la API.

/reactjs-react-router-lab/src/components/Inicio.jsx

```

import { useState } from 'react';

const Inicio = () => {
  const [usuarios, setUsuarios] = useState([])

  const listaUsuarios = usuarios.map(u => (
    <li key={u.id}>
      <span>{u.username}</span>
    </li>
  ))

  return (
    <div>
      <h2>Inicio</h2>
      <ul>
        {listaUsuarios}
      </ul>
    </div>
  )
}

```

```
export default Inicio
```

Ahora dentro del **useEffect** vamos a realizar la petición GET a <http://jsonplaceholder.typicode.com/users> para obtener la lista de usuarios y guardarla dentro del estado. Como esta petición solo queremos hacerla una vez, vamos a pasarle al **useEffect** como lista de dependencias un array vacío.

/reactjs-react-router-lab/src/components/Inicio.jsx

```
import { useEffect, useState } from 'react';

const Inicio = () => {
  const [usuarios, setUsuarios] = useState([])

  useEffect(() => {
    fetch('http://jsonplaceholder.typicode.com/users')
      .then(resp => resp.json())
      .then(usuarios => setUsuarios(usuarios))
  }, [])

  const listaUsuarios = usuarios.map(u => (
    <li key={u.id}>
      <span>{u.username}</span>
    </li>
  ))

  return (
    <div>
      <h2>Inicio</h2>
      <ul>
        {listaUsuarios}
      </ul>
    </div>
  )
}

export default Inicio
```

Con esto ya deberíamos de ver una lista de nombres de usuarios en la página inicial. El siguiente paso es poder ver más en detalle los datos de cada usuario, por lo que vamos a añadir una nueva ruta, pero esta vez con parámetros.

Antes de añadir la ruta, vamos a añadir por cada usuario un enlace a la nueva ruta que vamos a añadir. Para ello vamos a utilizar el componente **Link** y pondremos como valor de la propiedad **to** el path **/usuarios/** seguido del identificador de cada uno de ellos.

/reactjs-react-router-lab/src/components/Inicio.jsx

```
import { useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
```

```

const Inicio = () => {
  const [usuarios, setUsuarios] = useState([])

  useEffect(() => {
    fetch('http://jsonplaceholder.typicode.com/users')
      .then(resp => resp.json())
      .then(usuarios => setUsuarios(usuarios))
  }, [])

  const listaUsuarios = usuarios.map(u => (
    <li key={u.id}>
      <span>{u.username}</span>
      <Link to={'/usuarios/' + u.id}>Ver + info</Link>
    </li>
  ))

  return (
    <div>
      <h2>Inicio</h2>
      <ul>
        {listaUsuarios}
      </ul>
    </div>
  )
}

export default Inicio

```

Ahora vamos a crear un nuevo componente **Usuario** en el que mostraremos la información de un usuario en concreto.

/reactjs-react-router-lab/src/components/Usuario.jsx

```

const Usuario = () => {
  return (
    <div>
      <h3>Usuario con ID: ...</h3>
    </div>
  )
}

export default Usuario

```

Ahora que tenemos el componente, vamos a añadir la nueva ruta **/usuarios/:usuarioId** en el componente **App**.

/reactjs-react-router-lab/src/components/App.jsx

```

import { Routes, Route, Navigate } from 'react-router-dom';

```

```

import Header from './Header';
import Inicio from './Inicio';
import NuevoUsuario from './NuevoUsuario';
import Usuario from './Usuario';

const App = () => {
  return (
    <div>
      <h1>React Router v6</h1>
      <Header />

      <Routes>
        <Route path="/usuarios" element={<Inicio />} />
        <Route path="/usuarios/:usuarioId" element={<Usuario />} />
        <Route path="/nuevo-usuario" element={<NuevoUsuario />} />
        <Route path="/" element={<Navigate to="/usuarios" replace={true} />} />
      </Routes>
    </div>
  )
}

export default App

```

Si pulsamos sobre cualquier enlace de los usuarios deberíamos ver el componente Usuario que acabamos de crear, pero todavía nos falta una cosa. Tenemos que obtener de alguna forma el identificador de la ruta para poder hacer una petición a la API y así mostrar los datos de este usuario.

Aquí usaremos el hook de **useParams** del que vamos a extraer el **usuarioId** y que de momento vamos a mostrar en el componente.

/reactjs-react-router-lab/src/components/Usuario.jsx

```

import { useParams } from 'react-router-dom';

const Usuario = () => {
  const { usuarioId } = useParams()

  return (
    <div>
      <h3>Usuario {usuarioId}</h3>
    </div>
  )
}

export default Usuario

```

El siguiente paso sería realizar la petición GET correspondiente para obtener los datos a mostrar. Al igual que antes, la realizaremos en el hook **useEffect** y vamos a guardar el resultado en un estado.

```
import { useState, useEffect } from 'react'
import { useParams } from 'react-router-dom';

const Usuario = () => {
  const { usuarioId } = useParams()
  const [infoUsuario, setInfoUsuario] = useState('')

  useEffect(() => {
    fetch(`https://jsonplaceholder.typicode.com/users/${usuarioId}`)
      .then(resp => resp.json())
      .then(datos => setInfoUsuario(JSON.stringify(datos, null, 2)))
  }, [usuarioId])

  return (
    <div>
      <h3>Usuario {usuarioId}</h3>
      <pre>{infoUsuario}</pre>
    </div>
  )
}

export default Usuario
```

Con esto ya deberíamos de ver la información de los usuarios cuando pulsamos sobre los enlaces que habíamos puesto para cada uno de ellos.

Pero, si nos fijamos, es un poco tedioso pasar de la información de uno de ellos a la de otro, ya que tenemos que volver a la página de inicio y pulsar sobre el enlace de otro usuario.

Esto podemos mejorarlo utilizando las rutas anidadas. Vamos a hacer que al mismo tiempo que se muestra la información de un usuario, también se muestra la lista de estos. Para ello anidaremos la ruta **/usuarios/:usuarioId** dentro de la ruta de **/usuarios**.

Por tanto, vamos al componente **App** y vamos a modificar estas dos rutas, metiendo como propiedad children a la ruta de **/usuarios/:usuarioId** cuyo path pasará a ser solo **:usuarioId**.

```
import { Routes, Route, Navigate } from 'react-router-dom';
import Header from './Header';
import Inicio from './Inicio';
import NuevoUsuario from './NuevoUsuario';
import Usuario from './Usuario';

const App = () => {
  return (
    <div>
      <h1>React Router v6</h1>
```

```

    <Header />

    <Routes>
      <Route path="/usuarios" element={<Inicio />}>
        <Route path=":usuarioId" element={<Usuario />} />
      </Route>
      <Route path="/nuevo-usuario" element={<NuevoUsuario />} />
      <Route path="/" element={<Navigate to="/usuarios" replace={true} />} />
    </Routes>
  </div>
)
}

export default App

```

Ahora ya deberíamos de ver la lista, pero si pulsamos sobre cualquiera de los enlaces de los usuarios veremos que el componente de **Usuario** no se muestra. Esto se debe a que nos falta una última cosa por añadir.

Cuando usamos rutas anidadas, React Router no sabe donde tiene que colocar el componente a pintar para las rutas anidadas, por lo que tenemos que ayudarle añadiendo el componente **Outlet** dentro del componente principal, que en este caso sería nuestro componente **Inicio**.

/reactjs-react-router-lab/src/components/Inicio.jsx

```

import { useEffect, useState } from 'react';
import { Link, Outlet } from 'react-router-dom';

const Inicio = () => {
  const [usuarios, setUsuarios] = useState([])

  useEffect(() => {
    fetch('http://jsonplaceholder.typicode.com/users')
      .then(resp => resp.json())
      .then(usuarios => setUsuarios(usuarios))
  }, [])

  const listaUsuarios = usuarios.map(u => (
    <li key={u.id}>
      <span>{u.username}</span>
      <Link to={'/usuarios/' + u.id}>Ver + info</Link>
    </li>
  ))

  return (
    <div>
      <h2>Inicio</h2>
      <ul>
        {listaUsuarios}
      </ul>
    </div>
  )
}

```



```

        <Outlet />
      </div>
    )
  }

export default Inicio

```

Para ir terminando, vamos a añadir una página de error para cuando los usuarios intenten acceder a una ruta que no existe.

Vamos a crear un componente **Error** dentro de la carpeta **components**.

/reactjs-react-router-lab/src/components/Error.jsx

```

const Error = () => {
  return (
    <div>
      <h2 style={{color: 'red'}}>Error 404: Page not found!</h2>
    </div>
  )
}

export default Error

```

Una vez tengamos el componente, añadimos la ruta comodín dentro del componente **App**.

/reactjs-react-router-lab/src/components/App.jsx

```

import { Routes, Route, Navigate } from 'react-router-dom';
import Header from './Header';
import Inicio from './Inicio';
import NuevoUsuario from './NuevoUsuario';
import Usuario from './Usuario';
import Error from './Error';

const App = () => {
  return (
    <div>
      <h1>React Router v6</h1>
      <Header />

      <Routes>
        <Route path="/usuarios" element={<Inicio />}>
          <Route path=":usuarioId" element={<Usuario />} />
        </Route>
        <Route path="/nuevo-usuario" element={<NuevoUsuario />} />
        <Route path="/" element={<Navigate to="/usuarios" replace={true} />} />
        <Route path="*" element={<Error />} />
      </Routes>
    </div>
  )
}

```

```
)  
}  
  
export default App
```

Si probamos a entrar en una ruta que no existe, entonces deberíamos de ver el error.